

Unit 12

SOFTWARE DEVELOPMENT

163 items · 924 marks

Syllabus sections in this unit

- 12.1 Program Development Life cycle
- 12.2 Program Design
- 12.3 Program Testing and Maintenance

Unit 12: Software Development

1. M/J 26/P21/Q3/a,b

- 3 A business operates five call centres. The software currently in use in each call centre is to be replaced; a new system will be developed by a software company.

(a) The analysis stage included a feasibility study.

State **three** other activities carried out during the analysis stage.

- 1
- 2
- 3

[3]

(b) The analysis stage has ended.

Complete the statements about the development of the new system.

Before any programming is started, the stage for the proposed new system must be completed.

For this large project, the tasks will be broken down into several ; each will be coded by a different programming team.

Once each team has completed its work, testing must be carried out.

Before the new system is delivered to the business, testing will be carried out in-house.

It is decided to install the software initially in only one of the call centres where testing will be carried out.

The new software is finally rolled out to all five call centres. In the first week of its use, staff report that the software still requires extra features which were not originally agreed. The software is withdrawn until maintenance is completed.

[6]

2. M/J 26/P21/Q8/a,b,c

- 8 Global array `MyData` contains the values shown.

Study the pseudocode algorithm and array MyData

```

FUNCTION MyTrace(Target : INTEGER) RETURNS INTEGER
    DECLARE LeftPointer, RightPointer, MidPointer : INTEGER
    LeftPointer ← 1
    RightPointer ← 15
    WHILE LeftPointer ≤ RightPointer
        MidPointer ← (LeftPointer + RightPointer) DIV 2
        IF MyData[MidPointer] = Target THEN
            OUTPUT "Found " & NUM_TO_STR(MidPointer)
            RETURN MidPointer
        ELSE
            IF Target > MyData[MidPointer] THEN
                LeftPointer ← MidPointer + 1
            ELSE
                RightPointer ← MidPointer - 1
            ENDIF
        ENDIF
    ENDWHILE
    OUTPUT "Not found"
    RETURN -1
ENDFUNCTION

```

	MyData
1	2
2	8
3	9
4	12
5	13
6	15
7	18
8	19
9	20
10	23
11	25
12	26
13	39
14	38
15	40

- (a) Complete the trace table for the function `MyTrace` when the following pseudocode statement is executed.

`Result ← MyTrace(15)`

LeftPointer	RightPointer	MidPointer

[4]

- (b) Give the output.

[1]

(c) Give the value assigned to `Result`

[1]

3. M/J 26/P22/Q1/b,c,d,e,f

- 1 A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a) When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (i) Complete the pseudocode to declare the variable to be used to hold the password.

```
DECLARE Pass : .....
```

[1]

- (ii) Complete the pseudocode to check that a password contains between **six** and **eight** characters inclusive (rule 1).

```
IF ..... (Pass) < 6 OR ..... (Pass) > ..... THEN
    OUTPUT "Error"
    OUTPUT "The password has to contain between 6 and 8 characters."
ENDIF
```

[2]

- (iii) Complete the pseudocode to check that a password entered contains at least one letter and one digit character (rule 2).

```

DECLARE Index : INTEGER
DECLARE Character : CHAR
DECLARE LetterFlag, DigitFlag : .....
DigitFlag ← FALSE
LetterFlag ← .....
FOR Index ← 1 TO ..... (Pass)
    Character ← ..... (Pass, ..... , ..... )
    IF ..... (Character) = TRUE THEN
        DigitFlag ← .....
    ELSE
        LetterFlag ← .....
    ENDIF
NEXT Index
IF ..... (LetterFlag AND DigitFlag) THEN
    OUTPUT "Error"
    OUTPUT "Password needs at least one letter and one digit."
ENDIF

```

[7]

- (b) A state-transition diagram is being created as part of the design process for the module, which will control the alarm system once it has been successfully installed. Two of the states already identified are that the alarm is in:

- standby mode
- on mode (alarm system is active).

- (i) Identify **two** other states that should be part of the state-transition diagram.

1

2

[2]

- (ii) Two events that will cause a change of state are:

- entering the password when the alarm system is in standby mode
- entering the password when the alarm system is in on mode.

Identify **three** other events that would cause a change of state in the state-transition diagram for the alarm system.

1

2

3

[3]

- (c) The validation rules from 1(a) are given again:

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

Complete the table with test data that could be used to test the validation rules.

Each example of test data given must:

- be invalid for the specific rule being tested
- be valid for the other **two** rules **not** being tested
- test a different aspect of the rule being tested.

Two of the tests have been completed for you.

rule being tested	example of test data	reason for invalid test data
rule 1	1A	password is too short
rule 1		
rule 2		
rule 2		
rule 3	PASSWRD2	letters are repeated
rule 3		
rule 3		

[3]

(d) Five outcomes that could be produced at the design stage in the program development lifecycle include:

- state-transition diagrams
- pseudocode
- test strategy
- test plan
- structured English.

Identify **three** other outcomes that could be produced at the design stage.

1

2

3

[3]

(e) The design and testing stages are two of the stages in the program development lifecycle used when developing the program to control the alarm system.

Identify **two** other stages in the program development lifecycle.

1

2

[2]

- (f) All the individual modules for the alarm system have now been written, tested and debugged as necessary. All these modules are combined to form the program, and further testing is now needed.

- (i) Identify this method of testing.

[1]

- (ii) One of the modules does not work as expected when added to the program.

Identify and describe a testing method that can be used to address this problem, so that the program can continue to be tested.

method

description

[3]

4. M/J 26/P23/Q3/a,b,c

- 3** A company wishes to replace the point-of-sale software that each shop uses at its checkout. A software company is developing a new system.

(a) Complete the summary about the development of the new system.

The initial stage is carried out by studying in detail the current system.

A design for the proposed new system is produced.

Following the design and coding stages, it is decided to install the completed new software initially in only two of the shops. testing is carried out.

The final stage of the project uses testing to test all programs on the company's hardware. After successful testing, the company approves the new system.

The company then installs the new software in all shops. In the first week of its use, unexpected bugs are reported. maintenance has to be carried out.

[4]

(b) For this large project, the task was divided between two programming teams.

Team A were responsible for the screen designs and all outputs; they involved the company at the earliest possible stage and obtained feedback.

Team B used a more traditional approach to the program development life cycle; the stages in the development followed an agreed sequence.

Identify the program development life cycle model adopted by each team, and describe **one** additional feature of each approach.

team A model

feature

team B model

feature

[4]

(c) State **three** activities carried out during the design stage of the program development life cycle.

1

2

3

[3]

5. M/J 26/P23/Q7/a,b

7 Study the pseudocode algorithm. Numbers will be copied into the *Data* array.

```
// STAGE 1
Num1 ← 12
Index ← 1
FOR i ← 2 TO Num1 DIV 2
    IF Num1 MOD i = 0 THEN
        Data[Index] ← i
        Index ← Index + 1
    ENDIF
NEXT i
Index ← Index - 1
OUTPUT Index, " values were found"
```

(a) (i) Complete the trace table for STAGE 1

Num1	Index	i	Data					
			1	2	3	4	5	6

[4]

(ii) State how the numbers in the *Data* array are related.

[1]

(b) The complete algorithm has a second stage.

```
// STAGE 2

Num2 ← 8
Match ← FALSE

REPEAT
    IF Num2 MOD Data[Index] = 0 THEN
        Match ← TRUE
    ELSE
        Index ← Index - 1
    ENDIF
UNTIL Match = TRUE OR Index = 0

IF Match = TRUE THEN
    OUTPUT Data[Index]
ELSE
    OUTPUT "Nothing found"
ENDIF
```

One line of the STAGE 1 code is changed:

```
Num1 ← 20
```

The programmer creates a final program with the STAGE 1 code followed immediately by the STAGE 2 code.

The STAGE 1 trace table shows the relevant final variable values after the completion of STAGE 1

STAGE 1				Data					
Num1	Num2	Index	Match	1	2	3	4	5	6
20		4		2	4	5	10		

This is followed by running of the STAGE 2 code.

(i) Complete the trace table for STAGE 2

STAGE 2				Data					
Num1	Num2	Index	Match	1	2	3	4	5	6
20		4		2	4	5	10		

[3]

(ii) State the purpose of the STAGE 2 algorithm.

[1]

6. O/N 25/P21/Q1/b

1 A program is being developed to meet a particular customer requirement.

(b) Different test methods will be used at different stages of the program development.

Complete the table by identifying the test method that matches the test description.

The first row has been completed for you.

Test description	Test method
carried out as soon as a program module has been coded	alpha
carried out as program modules are being combined	
completed by the developers without referring to the code	
completed by the customer	

[3]

7. O/N 25/P21/Q7/a,b

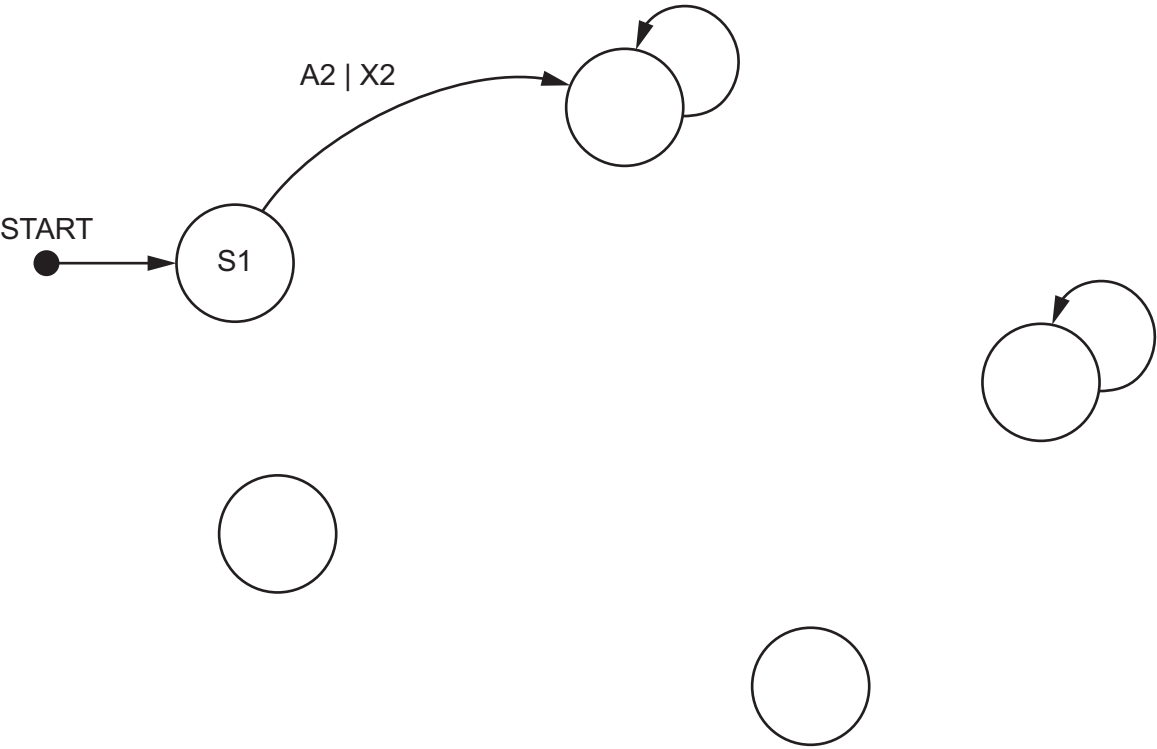
7 There are several different ways to express an algorithm during the design of a program.

(a) One part of the program contains an algorithm which is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A2	X2	S3
S1	A1	X1	S2
S2	A4	X4	S5
S3	A1		S3
S3	A3	X3	S2
S3	A2	X4	S4
S4	A1	X1	S4
S4	A3		S2
S4	A4	X4	S5

Complete the state-transition diagram to represent the information given in the table.



(b) A structure chart is used to document a different part of the program.

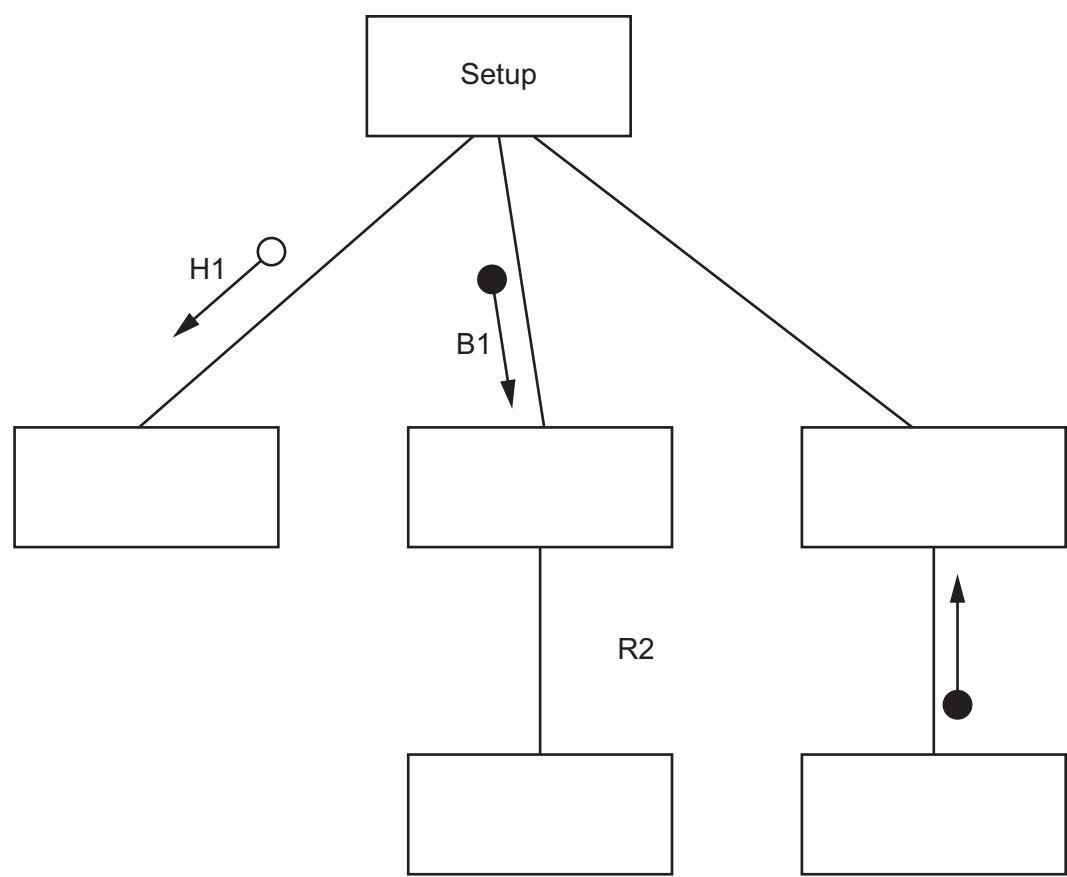
[5]

This part of the program contains six modules:

Pseudocode module header
PROCEDURE Setup()
PROCEDURE Restart(H1 : STRING, C1 : INTEGER)
FUNCTION Modify(B1 : BOOLEAN) RETURNS INTEGER
PROCEDURE Final(T1 : INTEGER)
PROCEDURE Update(BYREF R2 : STRING)
FUNCTION Confirm() RETURNS BOOLEAN

Module Setup will repeatedly call **three** of the modules.

Complete the structure chart to document the information given in the table.



8. O/N 25/P21/Q8/c

- 8 A program is being developed to manage student book loans from a college library. Students may borrow up to five books at a time from the library.

The programmer has defined a record type to define each loan.

The record data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 5000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined the first program module:

Module	Description
<code>OKToBorrow()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - search the array for loan records for the specified student - output a suitable message to say whether the student may, or may not, borrow another book

- (c) It is decided to introduce a system of fines for books that have been borrowed for too long.

Two new requirements are defined:

- Each loan has a maximum length, represented as a number of days.
- Each book in the library will be assigned one of three categories. Each category has a different maximum loan length.

Record structure and program design changes are needed to meet these two requirements.

Outline the changes that are necessary to meet the **two** requirements.

[2]

9. O/N 25/P22/Q5/a,b

- 5** A program is developed to satisfy a specific customer requirement.

The project follows a program development life cycle model. This model divides the development process into several different stages.

- (a)** The table lists some of the development activities.

Complete the table by writing the name of the life cycle stage for each activity:

Activity	Name of life cycle stage
a structure chart is produced	
a program is modified to allow it to run on new hardware	
the programmer identifies the customer's requirements	
an Integrated Development Environment (IDE) provides context-sensitive help such as 'auto-complete'	

[4]

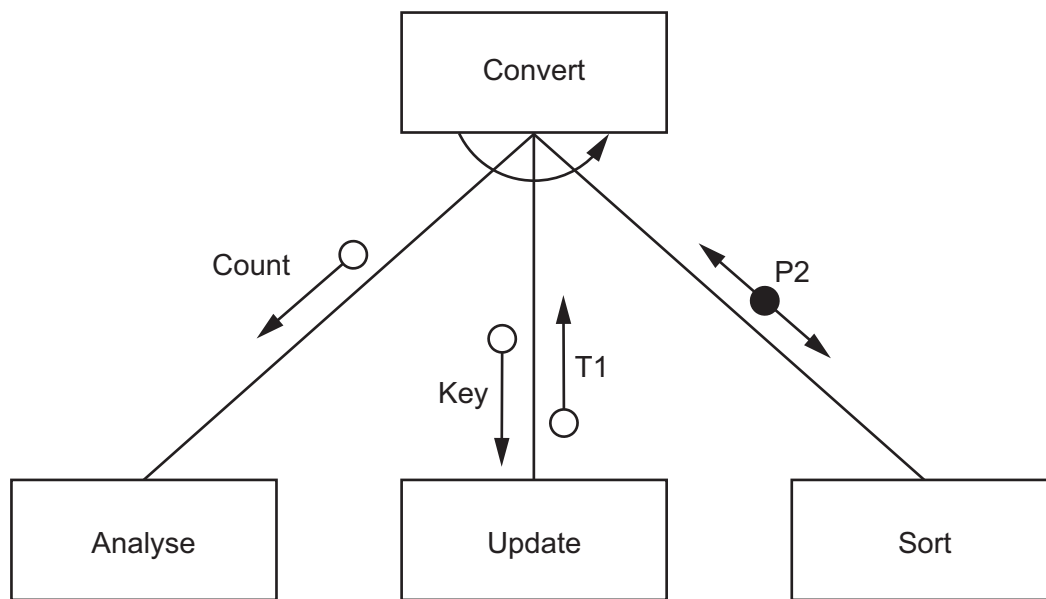
- (b)** Alpha and beta testing has been completed. The final testing stage is carried out by the customer.

Identify this final testing stage.

[1]

10. O/N 25/P22/Q7/a,b

7 The structure chart shows part of a program design:



(a) Explain the meaning of the curved arrow symbol in this structure chart.

[2]

(b) The program designer has noted that:

- Count is the number of elements in an array
- T1 is a value representing the number of elements in an array that have been modified
- Key is of type `STRING`

Write the pseudocode module headers for analyse, update and sort.

Analyse:

Update:

Sort:

[4]

11. O/N 25/P22/Q8/d

- 8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 7000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
<code>CountLoans()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - counts the number of books currently on loan to the specified student - counts the number of books that the student has already returned - output both counts together with a suitable message

- (d) It is decided to extend the program so that a new module `Reminder()` will send an email to the student three days before the book is due to be returned.

Outline the changes that will need to be made to the data stored and how this data will be used to generate the reminder email.

Data

Use

[2]

12. O/N 25/P23/Q5/a

- 5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
Num[2] ← 2
Num[3] ← 5
Num[4] ← 3
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE CaseVar, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  WHILE Count <= 20
    CaseVar ← Num[Index]
    CASE OF CaseVar
      1 : Num[Index] ← Num[Index] + Index //clause one
        Index ← Index + 1
        Count ← Count + 1

      2 : Num[Index] ← Num[Index] + Index //clause two
        Index ← Index + 2
        Count ← Count + 2

      3 : Num[Index] ← Num[Index] * 2      //clause three
        Index ← Index + 3
        Count ← Count - 1

      4 : Count ← Count + 4                //clause four
    OTHERWISE : Count ← 20
    ENDCASE

    Index ← (Index MOD 4) + 1

  ENDWHILE
ENDPROCEDURE
```


13. O/N 25/P23/Q6/a

- 6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The original number is always at least three digits in length.

When the new number is input, the check digit is used to validate the new number.

- (a) A function `Generate()` is written to take an original number as a parameter and to return the check digit.

Outline a test plan that could be used to fully test function `Generate()`

Assume that the parameter is valid.

[2]

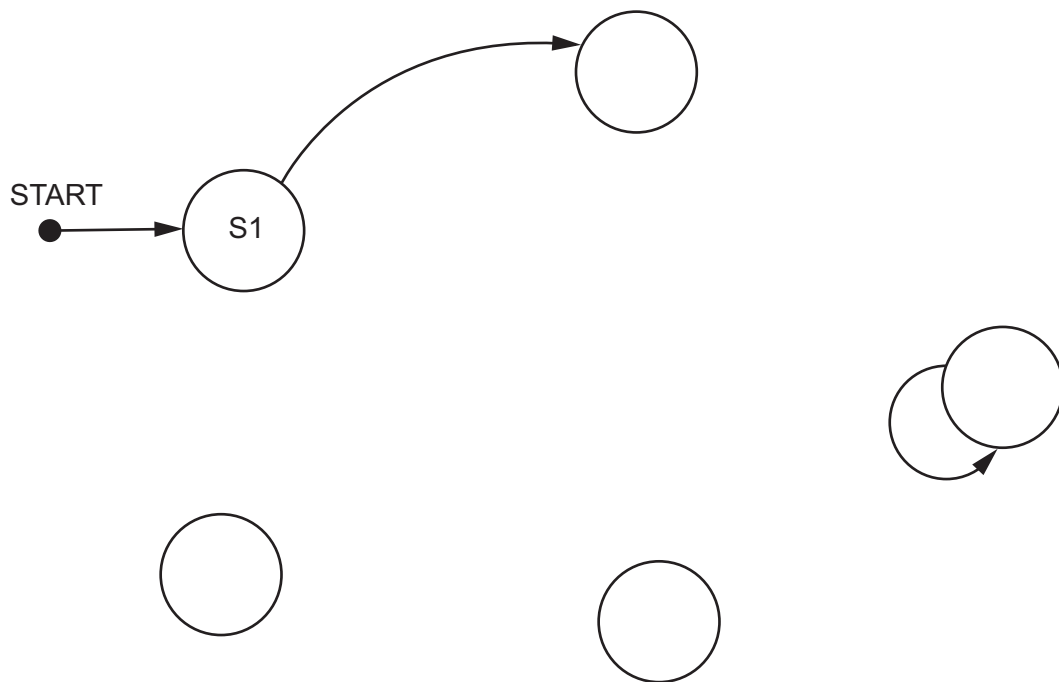
14. O/N 25/P23/Q7/a,b

- 7 There are several different ways to express an algorithm during the design of a program.
- (a) One part of the program contains an algorithm which is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A1		S2
S2	A2	X4	S3
S3	A1	X1	S3
S3	A2	X1	S3
S3	A3	X3	S4
S3	A4		S2
S4	A3	X4	S5
S4	A4	X4	S5
S4	A1		S2

Complete the state-transition diagram to represent the information given in the table:



[5]

- (b) A structure chart is used to document a different part of the program, made up of five modules.

Program notes:

- module Setup calls either module Restart, or module Confirm
- module Confirm takes a string as a parameter and returns an integer
- module Modify has no parameters and returns a Boolean
- module Update takes a string as a parameter
- module Restart repeatedly calls module Update followed by module Modify
- module Restart takes a string as a parameter that is passed by reference.

Draw a structure chart to represent the relationship between the **five** modules, including all parameters and return values.

[4]

15. M/J 25/P21/Q2/a,b,c

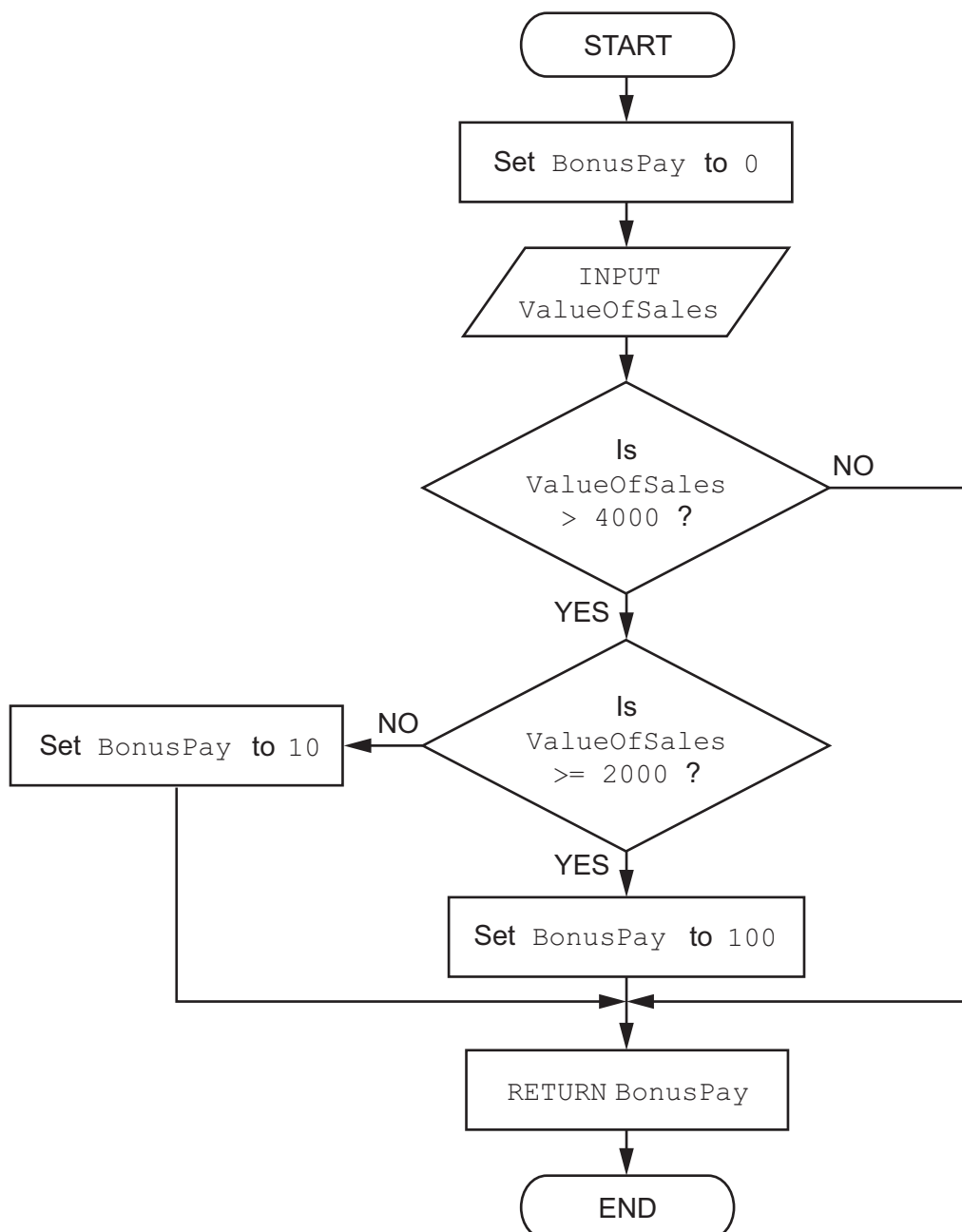
2 A program is being developed to calculate the pay of employees working for a company.

A function `CalculateBonus()` calculates bonus pay based on the value of sales.

Bonus pay is calculated as shown in the table.

Value of sales (in dollars)	Bonus pay (in dollars)
below 2000	0
between 2000 and 4000 inclusive	10
above 4000	100

A flowchart for the function `CalculateBonus()` has been designed.



(a) The flowchart contains logic errors.

One logic error is that `BonusPay` will **not** be set to 10 although the `ValueOfSales` input is between 2000 and 4000 inclusive.

(i) Explain why this error occurs.

[1]

(ii) Explain how this error could be corrected.

[1]

(b) There are different ways to reduce the risk of errors when developing the new program, such as the use of constants.

(i) State a value that could be replaced by a constant in the function `CalculateBonus()`

[1]

(ii) Explain how the use of constants helps to reduce the risk of programming errors.

[2]

(iii) One other way that can reduce the risk of errors when writing the program is the use of library routines.

Explain how library routines can reduce the risk of programming errors.

[1]

(c) All the logic errors have been corrected, and the program has been coded.

Identify and describe **two** other types of error that the program could contain.

Type of error

Description

Type of error

Description

[4]

16. M/J 25/P21/Q4/b**4** Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I ≤ 4                                     //Outer loop
    Key ← Chars[I]
    J ← I - 1
    WHILE J ≥ 0 AND Chars[J] > Key             //Inner loop
        Chars[J + 1] ← Chars[J]
        J ← J - 1
    ENDWHILE
    Chars[J + 1] ← Key
    I ← I + 1
ENDWHILE
```

(b) Complete the trace table by dry running the algorithm.

17. M/J 25/P21/Q6/a

- 6** A program monitors the speed of vehicles as they move around a large building site.

Each vehicle contains a sensor which reads an integer value that represents the speed of the vehicle. The value is expected to be in the range 0 to 60 inclusive.

The sensors cannot read values less than 0.

A program module has been written to validate the values read by the sensors.

- (a)** A test plan is needed to fully test the module.

Complete the table. The first line has been completed for you.

Assume the sensors generate only integer values.

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted

[4]

18. M/J 25/P21/Q7/b(ii)

- 7** A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	- called with two integer parameters: - the number of coffees ordered - the current number of points - output a suitable message giving the number of free coffees - return a value for the new points total.

- (b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme. The data items for each customer will be stored on a separate line of the text file where each data item is separated by a comma:

```
<CustomerID>,<Points>
```

The contents of the text file `Loyalty.txt` will always be stored in ascending order by customer ID.

When the data items are read from or written to the text file `Loyalty.txt`, they may need to be converted to the appropriate data type.

Each customer has a unique customer ID starting at "100001" with this value increasing by one each time a new customer joins the loyalty scheme.

When a customer joins the loyalty scheme, they are assigned the next customer ID and value of points is set to 0.

For example, if the loyalty scheme has 204 customers and a new customer joins the loyalty scheme, the following line is added to the text file `Loyalty.txt`:

```
"100205,0"
```

You can assume that the number of customers in the loyalty scheme will never be more than 9000.

The programmer has defined a program module as follows:

Module	Description
<code>AddNewCustomers()</code>	- called with an integer parameter representing the number of new customers to be added to the loyalty scheme - adds a new line, containing the required information, to the text file <code>Loyalty.txt</code> for each customer added to the loyalty scheme - outputs each new customer ID added to the loyalty scheme

- (ii) The requirements for `AddNewCustomers()` are changed. There may be no existing customers in the loyalty scheme.

Explain the changes that will need to be made to the module `AddNewCustomers()`

[4]

19. M/J 25/P22/Q1/a,b

1 A program is being developed to control the production line in a factory.

(a) A number of different program life cycles are available for the development of a program.

(i) Explain the need for different program development life cycles.

[1]

(ii) Coding is a stage in a program development life cycle.

State **one** consideration that would influence the choice of programming language.

[1]

(b) The program has been in use for a number of months and adaptive maintenance is required.

(i) Give **three** reasons why adaptive maintenance may be required.

1

2

3

[3]

(ii) As well as adaptive maintenance, other types of program maintenance may be needed.

Identify **one** other type of program maintenance.

[1]

20. M/J 25/P22/Q2/b

2 A program to calculate the pay of employees working for a company is being designed.

- (b)** One of the program modules used to calculate employees' weekly bonus pay has been completed. The amount of bonus pay they receive is based on the number of hours worked and the value of sales made as shown in the table.

Bonus pay and value of sales are both in dollars.

Hours worked	Value of sales	Bonus pay
between 1 and 40 inclusive	2000 or less	0
between 1 and 40 inclusive	above 2000	50
above 40	2000 or less	10
above 40	above 2000	100

- (i)** The module is tested using white-box testing.

State **two** tests, using valid data, that can be used to test different paths through the program.

Test one:

Hours worked

Value of sales

Bonus pay

Test two:

Hours worked

Value of sales

Bonus pay

[2]

- (ii)** The program to calculate pay uses a number of modules which are each called from different places in the program.

The program is to be tested using stub testing before all the program modules have been completed.

Describe stub testing.

[2]

- (iii)** The program has been completed and compiles successfully.

The program is tested using black-box testing.

Identify and describe **one** type of error that black-box testing could detect.

Type of error

Description

[2]

21. M/J 25/P22/Q5/a

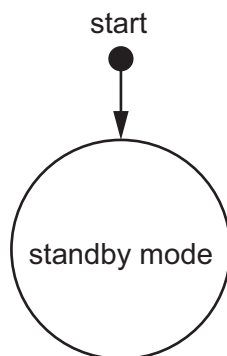
- 5 An automated digital camera system is used to take a sequence of pictures of animals in a garden. During the design of the system, a state-transition diagram is produced.

The table details the states in the automated digital camera system along with the events which cause the states to change.

The system starts in standby mode. The sequence of pictures is taken when in active mode.

Current state	Event	Next state
standby mode	turn on	detect mode
detect mode	turn off	standby mode
detect mode	movement detected	active mode
active mode	20 seconds elapsed	sequence complete
active mode	turn off	standby mode
sequence complete	time saved	detect mode

- (a) Complete the state-transition diagram for the automated digital camera system.



[4]

22. M/J 25/P22/Q6/a

also Unit 10 (3 marks); this unit 4 marks

- 6 Two arrays `Data` and `Pointer` are accessed by the procedure `Place()`

`Data` and `Pointer` are both global arrays of type `INTEGER`

The contents of these two arrays are shown:

Data		Pointer	
1	1018	1	7
2	1007	2	3
3	1010	3	1
4	1056	4	6
5	1092	5	-1
6	1062	6	5
7	1034	7	4
8	0	8	9
9	0	9	10
10	0	10	-1

Study the pseudocode:

```

PROCEDURE Place(Value : INTEGER, Start : INTEGER, Unused : INTEGER)
  DECLARE New, Current, Last : INTEGER
  CONSTANT NullPointer = -1
  New ← Unused
  Last ← NullPointer
  Current ← Start
  WHILE Current <> NullPointer AND Data[Current] < Value
    Last ← Current
    Current ← Pointer[Current]
  ENDWHILE
  Pointer[New] ← Pointer[Last]
  Pointer[Last] ← New
  Data[New] ← Value
ENDPROCEDURE

```

- (a) (i) Complete the trace table below by dry running the procedure `Place()` when it is called by the statement:

`CALL Place(1043, 2, 8)`

Value	Start	Unused	New	Last	Current

[4]

- (ii) Complete the diagram showing the contents of the global arrays `Data` and `Pointer` after the procedure `Place()` has run to completion when called as shown in part (a)(i).

Data		Pointer	
1	1018	1	
2	1007	2	
3	1010	3	
4	1056	4	
5	1092	5	
6	1062	6	
7	1034	7	
8		8	
9		9	
10		10	

[3]

23. M/J 25/P23/Q4/b

- 4 A programmer is developing a new stock control program for a shop owner to produce sales reports.

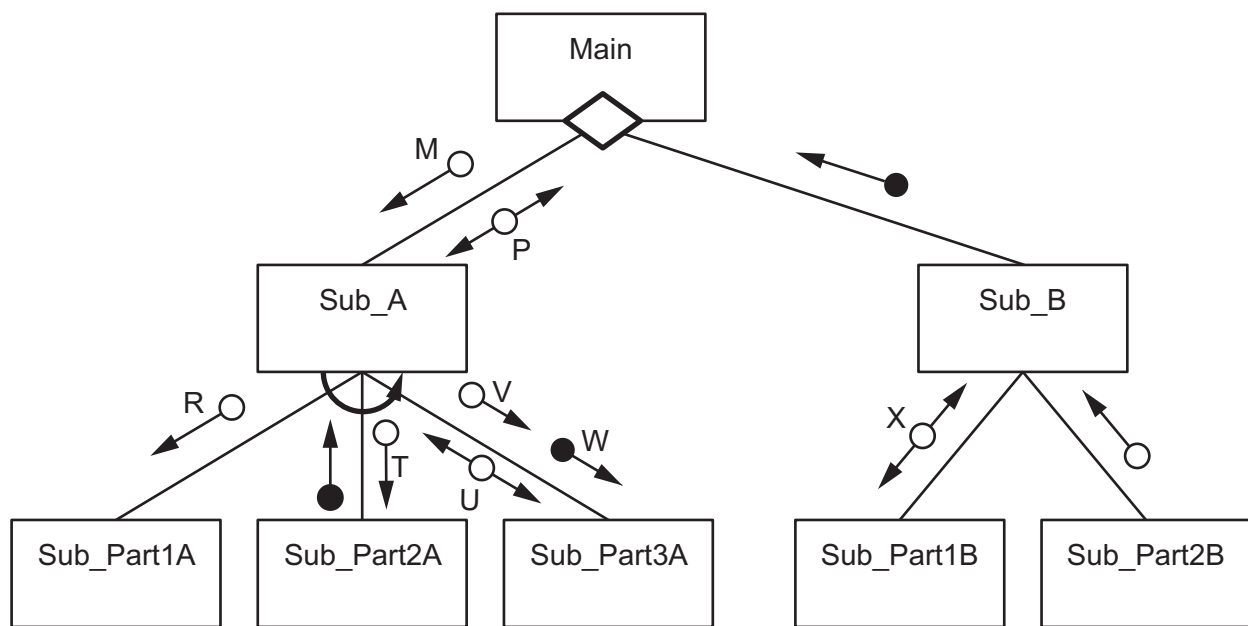
- (b) Once the program has been completed, it is tested using the walkthrough method.

Describe the walkthrough method and explain how it can be used to identify errors.

[3]

24. M/J 25/P23/Q6/b,c

6 Study the structure chart:



Some of the parameter data types are:

- R and V are of type `INTEGER`
- T is of type `REAL`
- U is of type `STRING`
- W is of type `BOOLEAN`.

Some of the modules in the structure chart are functions, others are procedures.

(b) Write the pseudocode to define the module headers:

Sub_Part1A

Sub_Part2A

Sub_Part3A

[5]

(c) The structure chart uses the two symbols shown.

Complete the table to explain what each symbol means and how the relevant modules in the structure chart are affected.

Symbol	Explanation
	
	

[3]

25. O/N 24/P21/Q1/a

- 1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```

HighRate ← FALSE
CASE OF ItemCost
    <= 50 : TaxRate ← 3.75      // tax rate of 3.75%
    <= 200 : TaxRate ← 5.23    // tax rate of 5.23%
    > 200 : TaxRate ← 6.25     // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable

```

- (a) The pseudocode contains a logical error.

Identify the error **and** suggest a correction.

Error

Correction

[2]

26. O/N 24/P21/Q3/c

- 3** An algorithm will output the **last** three lines from a text file `Result.txt`

The lines need to be output in the same order as they appear in the file.

Assume:

- Three variables `LineX`, `LineY` and `LineZ` will store the three lines. These are of type string and all three variables have been initialised to an empty string.
- The file exists and contains **at least** three lines.

- (a)** The algorithm to output the lines is expressed in eight steps.

Complete the steps.

1. Open the file
2. Loop until
3. and store in `ThisLine`
4. Assign `LineY` to `LineX`
5. Assign `LineZ` to `LineY`
6. Assign `ThisLine` to `LineZ`
7. After the loop,
8. Output `LineX`, `LineY`, `LineZ`

[4]

- (c)** The requirement changes, and the algorithm will now output three lines from the file, starting from a **given** line number.

The modified algorithm will be implemented as a function which will:

- be called with an integer parameter representing the given line number
- output three lines, starting at the given line
- return `TRUE` if the 3 lines are output, or `FALSE` if it was not possible to output the 3 lines.

Describe the changes that need to be made to **steps 2 to 8** of the algorithm given in part **(a)**.

[4]

27. O/N 24/P21/Q4/a

- 4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

- (a) Identify the type of error that could occur if these checks are **not** carried out **and** state a cause of this error.

Type

Cause

[2]

28. O/N 24/P21/Q5/a,b

- 5 A software developer follows a program development life cycle. The life cycle divides the development process into various stages.

- (a) The following table lists some development activities.

Complete the table by writing the name of the life cycle stage for each activity.

Activity	Name of life cycle stage
The walkthrough method is used.	
An algorithm is implemented in a programming language.	
The client is interviewed about problems with the current system.	
The program is modified to run on new hardware.	
Records and file structures are defined.	

[5]

- (b) The program contains a validation function.

(i) The function will:

- take an integer value as a parameter
- return `TRUE` if the value is within the range 24 to 37, inclusive
- otherwise return `FALSE`.

Complete the table to define a test plan to thoroughly test the operation of the function.

Type of test data	Test data value	Expected result
Normal	30	TRUE

[4]

(ii) The function is to be tested on its own. When it is shown to work correctly the function will be combined with other modules and testing will continue.

Identify the type of testing that this represents.

[1]

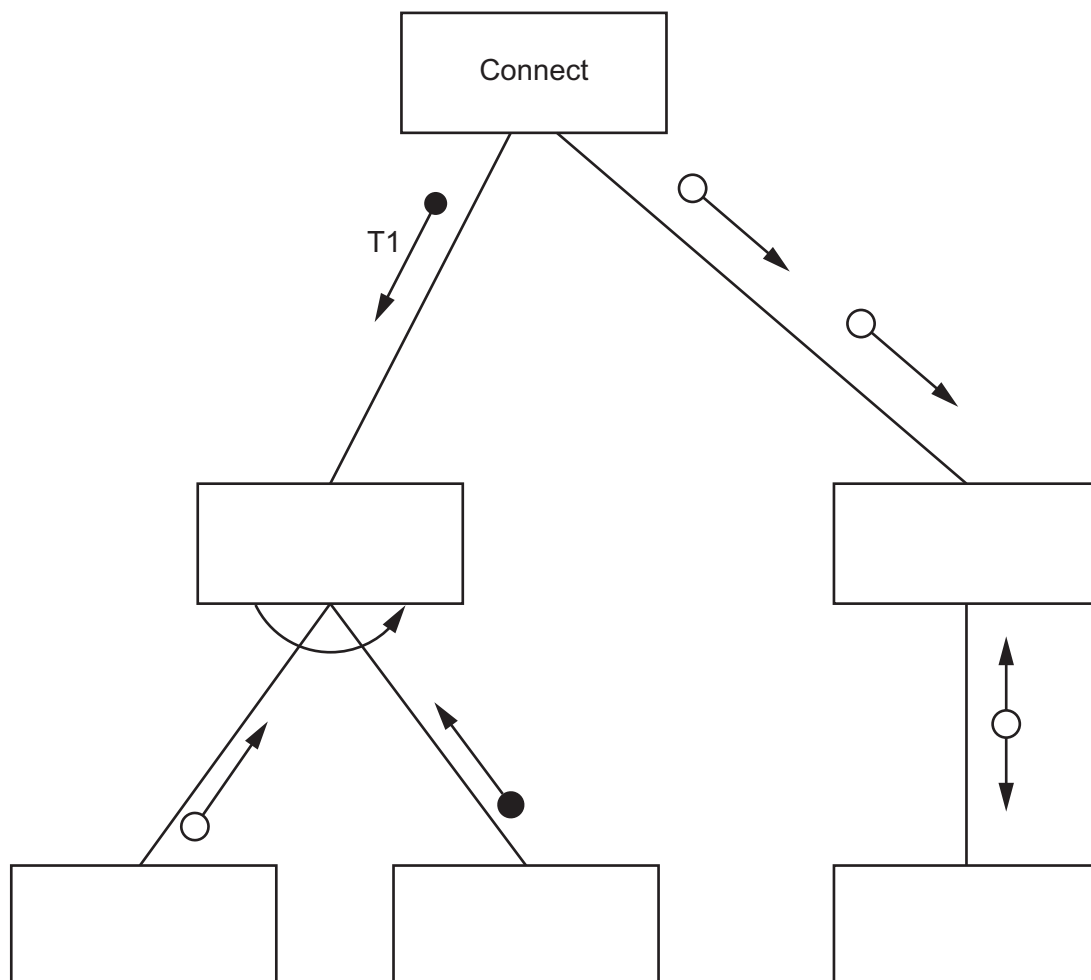
29. O/N 24/P21/Q7/a,b

7 A program contains six modules with headers as follows:

Pseudocode module header
PROCEDURE Connect ()
FUNCTION Activate (H1 : STRING, Code : INTEGER) RETURNS BOOLEAN
FUNCTION Sync (T1 : BOOLEAN, S2 : REAL) RETURNS INTEGER
PROCEDURE Initialise (BYREF ID : INTEGER, BYVAL CC : INTEGER)
FUNCTION Reset (RA : STRING) RETURNS INTEGER
FUNCTION Enable (SA : INTEGER) RETURNS BOOLEAN

Module Connect () will call either Activate () or Sync (). This is decided at run-time.

(a) Complete the structure chart for these modules.



[5]

(b) Explain the meaning of the curved arrow symbol used in the diagram in part (a).

[2]

30. O/N 24/P22/Q1/a

- 1 A program has been developed and released for general use. After a few months of use an error is detected where under certain circumstances it outputs an unexpected value.

(a) The error in the program needs to be corrected.

Identify the stage of the program development life cycle that this correction is made in.

[1]

31. O/N 24/P22/Q5/a,b

- 5 A program contains a global 1D array `Data` with 100 elements of type `INTEGER`.

The program contains a function `Process()` expressed in pseudocode as follows:

```
FUNCTION Process(Number : INTEGER, Label : STRING) RETURNS STRING
  DECLARE Index, Count : INTEGER
  DECLARE ReturnValue : STRING

  Count ← INT(100 / Number)
  Index ← Data[Number]

  CASE OF (Index MOD 2)
    0 : ReturnValue ← TO_UPPER(RIGHT(Label, Count))
    1 : ReturnValue ← "*****"
  ENDCASE

  RETURN RetVal
ENDFUNCTION
```

- (a) Run-time errors can be generated in different ways. For example, a run-time error will be generated if a function is called with invalid parameters.

The pseudocode contains three statements that could generate a run-time error.

Write the **three** statements **and** explain how each could generate a run-time error.

Statement 1

Explanation

Statement 2

Explanation

Statement 3

Explanation

[3]

- (b) One type of run-time error can cause a program to stop responding ('freezing').

Identify a particular type of programming construct that can generate this type of error **and** explain why it occurs.

Construct

Explanation

[2]

32. O/N 24/P22/Q6/b

- 6 A shop sells sandwiches and snacks. The owner chooses a 'daily special' sandwich which is displayed on a board outside the shop. Each 'daily special' has two different fillings and is made with one type of bread.

The owner wants a program to randomly choose the 'daily special' sandwich.

The program designer decides to store the possible sandwich fillings in a 1D array of type string.

The array is declared in pseudocode as follows:

```
DECLARE Filling : ARRAY [1:35] OF STRING
```

Each element contains the name of one filling.

An example of the first five elements is as follows:

Index	Element value
1	"Cheese"
2	"Onion"
3	"Salmon"
4	"Anchovies"
5	"Peanut Butter"

A second 1D array stores the possible bread used:

```
DECLARE Bread : ARRAY [1:10] OF STRING
```

Each element contains the name of one type of bread.

An example of the first three elements is as follows:

Index	Element value
1	"White"
2	"Brown"
3	"Pitta"

Both arrays may contain unused elements. The value of these will be an empty string and they may occur anywhere in each array.

A procedure `Special()` will output a message giving the 'daily special' sandwich made from **two** randomly selected different fillings and **one** randomly selected bread.

Unused array elements must **not** be used when creating the 'daily special' sandwich.

Using the above examples, the output could be:

```
"The daily special is Cheese and Onion on Brown bread."
```

- (b) The owner decides that some combinations of fillings do not go well together. For example, anchovies and peanut butter.

Describe how the design could be changed to prevent certain combinations being selected.

33. O/N 24/P22/Q7/c,d

- 7** A coffee shop runs a computerised loyalty card system.

Customers are issued with a loyalty card with their name together with a unique customer ID.

Loyalty points are added to their card each time they spend money at the shop.

The following information is stored for each customer: ID, name, home address, email address, mobile phone number, date of birth, number of points, date of last visit and amount of money spent at last visit.

A new module will generate a personalised email message to each loyalty card customer who has not visited the coffee shop in the last three months. The message will include a unique voucher code which can be used to authorise a discount if the customer goes to the shop within the next two weeks.

- (c)** It is decided to adopt a formal program development life cycle model for the development of the new module.

The analysis of the new module is complete and the project moves on to the design stage. During this stage all the necessary algorithms and module designs will be defined.

State **three other** items that will be defined for the new module during the design stage.

1

2

3

[3]

- (d)** Part of the coffee shop program contains three program modules as follows:

- Module `Init()` has no parameters and returns a Boolean.
- Module `Reset()` takes a string as a parameter and returns an Integer.
- Module `Check()` repeatedly calls `Init()` followed by `Reset()`.

Draw a structure chart to represent the relationship between the **three** modules, including all parameters and return values.

[3]

34. O/N 24/P23/Q5/a,b

- 5 A software developer follows a program development life cycle. The life cycle divides the development process into various stages.

(a) The following table lists some development activities.

Complete the table by writing the name of the life cycle stage for each activity.

Activity	Name of life cycle stage
A compiler is used.	
A program that has been released for general use is modified.	
The dry run method is used.	
The program structure is specified.	

[4]

- (b) A software developer has written modules `Test_A()` and `Test_B()`. These have been written but contain errors. These modules are called from several places in the main program and testing of the main program (integration testing) has to stop.

Identify a method that can be used to continue testing the main program **before** the errors in these modules have been corrected **and** describe how this would work.

Method

Description

[3]

35. O/N 24/P23/Q7/a

- 7 A coffee shop owner wants to introduce a computerised loyalty card system.

A programmer discusses the details of the system with the shop owner.

- (a) Identify the stage of the program development life cycle that this discussion is part of **and** give a document that will be produced during this stage.

Stage

Document

[2]

36. M/J 24/P21/Q5/a,b

- 5 A global 1D array of strings contains three elements which are assigned values as shown:

```
Data[1] ← "aaaaaa"  
Data[2] ← "bbbbbb"  
Data[3] ← "cccccc"
```

Procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```

PROCEDURE Process (Format : STRING)
    DECLARE Count, Index, L : INTEGER
    DECLARE Result : STRING
    DECLARE C : CHAR

    Result ← "*****"

    FOR Count ← 1 TO LENGTH (Format) STEP 2
        C ← MID (Format, Count, 1)
        L ← STR_TO_NUM (MID (Format, Count + 1, 1))

        Index ← (Count + 1) DIV 2

        CASE OF C
            'X' : Result ← TO_UPPER (Data[Index])
            'Y' : Result ← TO_LOWER (Data[Index])
            'Z' : Result ← "***" & Data[Index]
        ENDCASE

        Data[Index] ← LEFT (Result, L)
    NEXT Count

ENDPROCEDURE

```

(a) Complete the trace table by dry running the procedure when it is called as follows:

CALL Process ("X3Y2W4")

Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]

- (b) The procedure is to be modified. If variable `C` is assigned a value other than 'X', 'Y' or 'Z', then procedure `Error()` is called and passed the value of variable `C` as a parameter.

This modification can be implemented by adding a **single line** of pseudocode.

- (i) Write the single line of pseudocode.

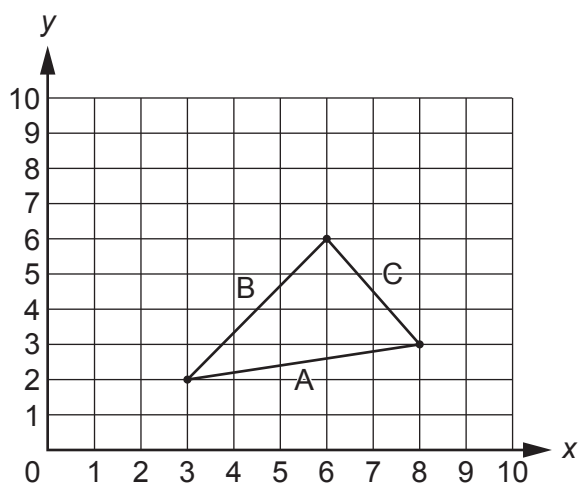
[1]

- (ii) State where this new line should be placed.

[1]

37. M/J 24/P21/Q6/b

- 6 Three points on a grid form a triangle with sides of length A, B and C as shown in the example:



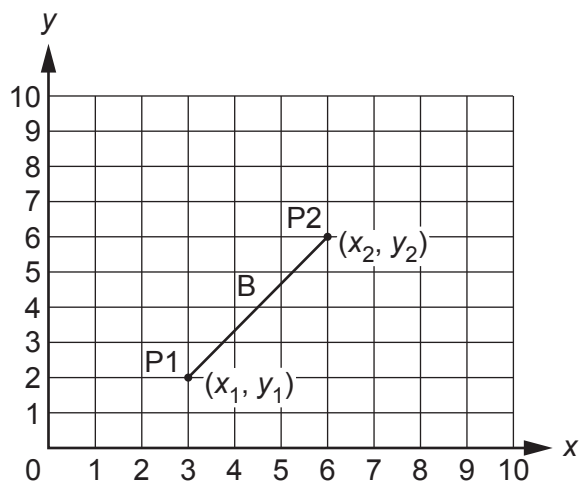
A triangle is said to be right-angled if the following test is true (where A is the length of the longest side):

$$A^2 = B^2 + C^2$$

A^2 means A multiplied by A, for example 3^2 means 3×3 which evaluates to 9

You can calculate A^2 , B^2 and C^2 by using the coordinates of the endpoints of each line.

For example, B^2 is calculated as follows:



The endpoints, P1 and P2, have the coordinates (3, 2) and (6, 6).

The value B^2 is given by the formula:

$$B^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

In this example:

$$B^2 = (3 - 6)^2 + (2 - 6)^2$$

$$B^2 = (-3)^2 + (-4)^2$$

$$B^2 = 9 + 16$$

$$B^2 = 25$$

(a) A function `IsRA()` will:

- take three sets of integers as parameters representing the coordinates of the three endpoints that form a triangle
- return `TRUE` if the endpoints form a right-angled triangle, otherwise return `FALSE`.

In pseudocode, the operator '^' represents an exponent, which is the number of times a value is multiplied by itself. For example, the expression Value^2 may be written in pseudocode as `Value ^ 2`

Complete the pseudocode for the function `IsRA()`.

```
FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN
ENDFUNCTION
```

[6]

(b) The test used to check if a triangle is right-angled can be written in two ways:

$$A^2 = B^2 + C^2$$

or

$$A = \sqrt{B^2 + C^2}$$

The symbol $\sqrt{}$ represents the square root operation. For example, $\sqrt{81} = 9$

A new function `SQRT()` is written to perform the square root operation. The function takes an integer number as a parameter and returns a positive real value representing the square root of the number.

During testing it is found that the `SQRT()` function returns a value that is only accurate to 4 decimal places.

For example, `SQRT(25)` returns 5.0000125 rather than the correct value of 5.0

The function `IsRA()` from part (a) is modified to use the new `SQRT()` function to test if a triangle is right-angled.

Describe a problem that might occur when using the modified `IsRA()` function **and** suggest a solution that still allows the `SQRT()` function to be used.

Problem

Solution

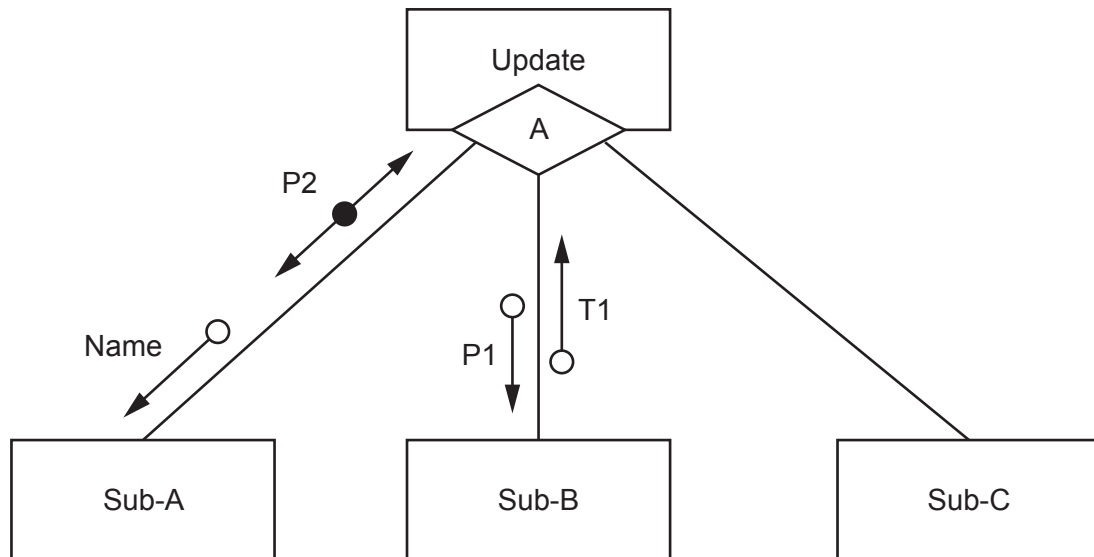
[2]

38. M/J 24/P21/Q7/b

- 7** A fitness club has a computerised membership system. The fitness club offers a number of different exercise classes.

The following information is stored for each club member: name, home address, email address, mobile phone number, date of birth and the exercise(s) they are interested in.

(b) The structure chart illustrates part of the membership program:



Data item notes:

- Name contains the name of a club member
- P1 and T1 are of type real.

(i) Explain the meaning of the diamond symbol (labelled with the letter A) in the chart.

[2]

(ii) Write the pseudocode module headers for Sub-A and Sub-B.

Sub-A

Sub-B

[4]

39. M/J 24/P22/Q5/a(i),b

5 A program is being designed in pseudocode.

The program contains a global 1D array `Data` of type string containing 200 elements.

The first element has the index value 1.

A procedure `Process()` is written to initialise the values in the array:

```
PROCEDURE Process(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 0
  INPUT Data[Index]
  WHILE Index < 200
    Index ← Index + 1
    CASE OF (Index MOD 2)
      0 : Data[Index] ← TO_UPPER(Label)
      1 : Data[Index] ← TO_LOWER(Label)
      OTHERWISE : OUTPUT "Alarm 1201"
    ENDCASE
  NEXT Index
  OUTPUT "Completed " & Index & " times"
ENDPROCEDURE
```

(a) (i) The pseudocode contains **two** syntax errors and **one** other error.

Identify the errors.

Syntax error 1

Syntax error 2

Other error

[3]

(b) After correcting all syntax errors, the pseudocode is translated into program code which compiles without generating any errors.

When the program is executed it unexpectedly stops responding.

Identify the type of error that has occurred.

[1]

40. M/J 24/P22/Q7/b

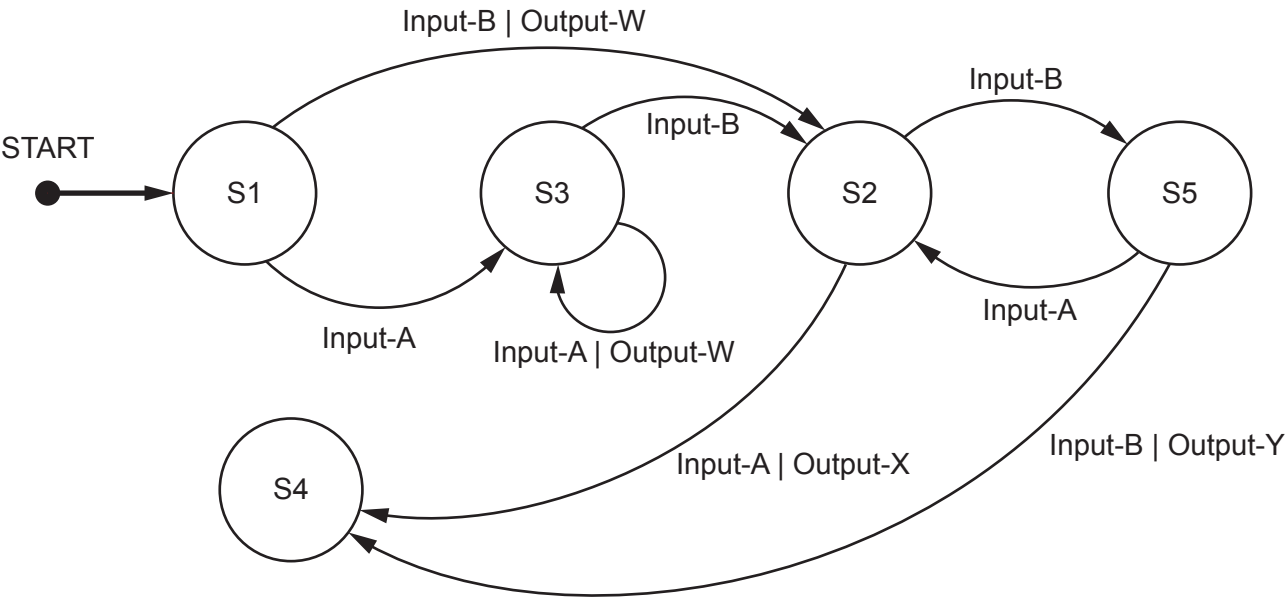
7 A fitness club has a computerised membership system.

The system stores information for each club member: name, home address, email address, mobile phone number, date of birth and exercise preferences.

Many classes are full, and the club creates a waiting list for each class. The club adds details of members who want to join a class that is full to the waiting list for that class.

When the system identifies that a space is available in one of the classes, a new module will send a text message to each member who is on the waiting list.

(b) A different part of the program is represented by the following state-transition diagram.



(i) Complete the table to show the inputs, outputs and next states.

Assume that the current state for each row is given by the ‘Next state’ on the previous row. For example, the first Input-A is made when in state S1.

If there is no output for a given transition, then the output cell should contain ‘none’.

The first two rows have been completed.

Input	Output	Next state
		S1
Input-A	none	S3
	Output-W	
	none	
Input-B		
Input-A		
		S4

[5]

(ii) Identify the input sequence that will cause the minimum number of state changes in the transition from S1 to S4.

[1]

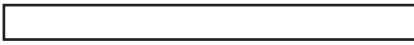
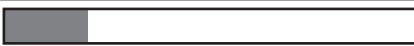
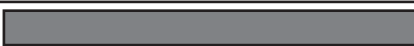
41. M/J 24/P23/Q6/b(ii)

- 6 A program displays a progress bar to inform the user of the progress of tasks that take a significant time to complete, such as those involving file transfer operations.

Task progress is divided into 11 steps. Each step represents the amount of progress as a percentage. An image is associated with each step and each image is stored in a different file.

Different progress bar images may be selected. For a given image, files all have the same filename root, with a different suffix.

The table illustrates the process for using the image with filename root `BargraphA`

Step	Percentage progress	Image filename	Image
1	< 10	BargraphA-1.bmp	
2	>= 10 and < 20	BargraphA-2.bmp	
3	>= 20 and < 30	BargraphA-3.bmp	
...			
9	>= 80 and < 90	BargraphA-9.bmp	
10	>= 90 and < 100	BargraphA-10.bmp	
11	100	BargraphA-11.bmp	

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

(b) The definition of procedure `Progress()` is provided here for reference:

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

`Progress()` will be rewritten and a new module `Progress2()` produced with these requirements:

- an additional parameter of type integer will specify the total number of steps
- the image filename will be returned (procedure `Display()` will **not** be called from within `Progress2()`).

(ii) State **one** benefit of increasing the number of steps.

42. M/J 24/P23/Q7/a(i),b

- 7 Seven program modules form part of a program. A description of the relationship between the modules is summarised below. Any return values are stated in the description.

Module name	Description
Mod-A	calls Mod-B followed by Mod-C
Mod-B	- called with parameters Par1 and Par2 - calls either Mod-D or Mod-E, determined when the program runs - returns a Boolean value
Mod-C	- called with parameters Par1 and Par3 - Par3 is passed by reference - repeatedly calls Mod-F followed by Mod-G
Mod-D	called with parameter Par2
Mod-E	- called with parameter Par3 - returns an integer value
Mod-F	called with parameter Par3
Mod-G	- called with parameter Par3 - Par3 is passed by reference

Parameters in the table are as follows:

- Par1 and Par3 are of type string.
- Par2 is of type integer.

- (a) (i) Identify the modules that would be implemented as functions.

[1]

- (b) Draw a structure chart to show the relationship between the seven modules and the parameters passed between them.

[6]

43. O/N 23/P21/Q5/a,b

- 5 An algorithm is designed to find the smallest numeric value from an input sequence and count how many numeric values have been input.

An example of an input sequence is:

23, AB56, 17, 23ZW, 4, 10, END

Numeric input values are all integers and non-numeric input is ignored, except for the string "END" which is used to terminate the sequence.

The algorithm is expressed in pseudocode as shown:

```
DECLARE NextInput : STRING
DECLARE Min, Count, Num : INTEGER

Min ← 999
Count ← 0

REPEAT
    INPUT NextInput
    IF IS_NUM(NextInput) = TRUE THEN
        Num ← STR_TO_NUM(NextInput)
        IF Num > Min THEN
            Min ← Num
        ENDIF
        Count ← Count & 1
    ENDIF
UNTIL NextInput ← "END"

OUTPUT "The minimum value is ", Min, " and the count was ", Count
```

(a) The pseudocode contains three errors due to the incorrect use of **operators**.

Identify each error **and** state the correction required.

1

2

3

[3]

(b) The operator errors are corrected and the algorithm is tested as follows:

The input sequence:

18, 4, ONE, 27, 189, ERIC, 3, 65, END

produces the output:

The minimum value is 3 and the count was 6

The algorithm is tested with a different test data sequence. The sequence contains a mix of integer and non-numeric values. It is terminated correctly but the algorithm produces unexpected results.

(i) Explain the problem with the algorithm.

[2]

(ii) Give a sequence of **four** test data values that could be input to demonstrate the problem.

Value 1

Value 2

Value 3

Value 4

[2]

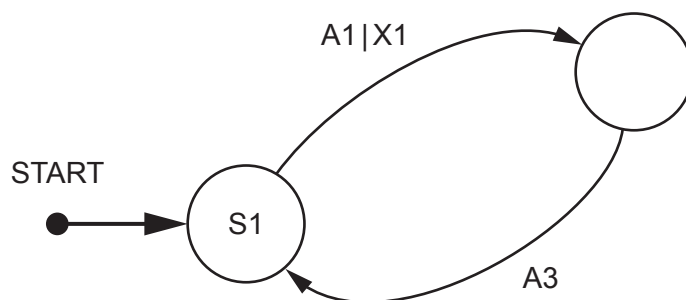
44. O/N 23/P21/Q7

7 An algorithm is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A1	X1	S2
S2	A3	none	S1
S2	A2	X4	S5
S5	A1	X1	S5
S5	A4	X2	S2
S5	A3	none	S3
S1	A9	X9	S3
S3	A9	X9	S4

Complete the state-transition diagram to represent the information given in the table.



[5]

45. O/N 23/P21/Q8/c

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

<STX>"232101Hello Kevin"<ETX>

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string containing a message ◦ an integer containing a port number • transmits the message using the given port
<code>SendFile()</code>	• takes three parameters: ◦ a string containing a text file name ◦ a string containing a Destination ID ◦ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "*****"

(c) One of the text files to be sent contains several blank lines (lines that do not contain any text).

(i) Explain why this is a problem.

[2]

(ii) Explain how the message format could be changed to allow a blank line to be sent.

[2]

46. O/N 23/P22/Q4/b

4 A procedure `Count ()` will:

1. input a value (all values will be positive integers)
2. count the number of odd values and count the number of even values
3. repeat from step 1 until the value input is 99
4. output the two count values, with a suitable message.

The value 99 must not be counted.

(b) The procedure `Count ()` is to be tested.

Typical test data would consist of odd and even values, for example:

23, 5, 64, 100, 2002, 1, 8, 900, 99

The purpose of this test would be to test a typical mix of even and odd values and check the totals.

Give **three** test data sequences that may be used to test **different** aspects of the procedure.

Do **not** include invalid data.

Sequence 1:

Test data

Purpose of test.

Sequence 2:

Test data

Purpose of test.

Sequence 3:

Test data

Purpose of test.

[3]

47. O/N 23/P22/Q5

5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 1  
Mix[2] ← 3  
Mix[3] ← 4  
Mix[4] ← 2
```

A procedure `Process ()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE Value, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  REPEAT
    Value ← Mix[Index]
    Mix[Index] ← Mix[Index] - 1
    Index ← Value
    Count ← Count + 1
  UNTIL Count = 5

  Mix[4] ← Count * Index

ENDPROCEDURE
```

Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(3)
```

[illegible]

[6]

48. O/N 23/P22/Q7/a,b

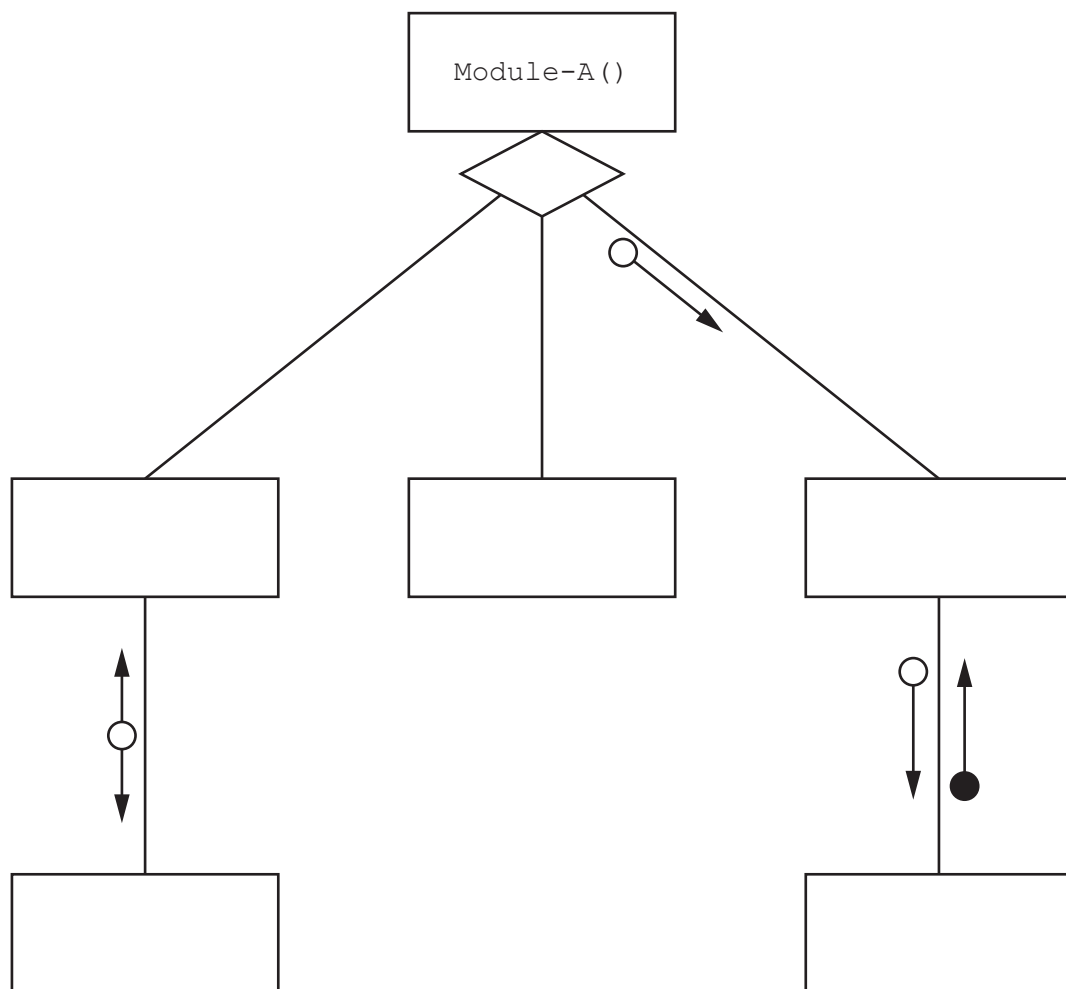
7 A program contains six modules:

Pseudocode module header
PROCEDURE Module-A()
PROCEDURE Module-X(T1 : INTEGER, S2 : REAL)
PROCEDURE Reset(BYREF Code : INTEGER)
FUNCTION Restore(OldCode : INTEGER) RETURNS BOOLEAN
FUNCTION Module-Y(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN
FUNCTION Module-Z(SA : INTEGER) RETURNS INTEGER

Module-X() **calls** Reset()

Module-Y() **calls** Restore()

(a) Complete the structure chart for these modules.



[4]

(b) Explain the meaning of the diamond symbol as used in the diagram in part (a).

[2]

49. O/N 23/P23/Q1/d

- 1 A program is being developed in pseudocode before being converted into a programming language.
- (d) The hardware that runs the program is changed and the program needs to be modified so that it works with the new hardware.

Identify the type of maintenance that this represents **and** give **one other** reason why this type of maintenance may be needed.

Type

Reason

[2]

50. O/N 23/P23/Q5

- 5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 4
Mix[2] ← 2
Mix[3] ← 3
Mix[4] ← 5
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE Value, Index, Total : INTEGER

  Index ← Start
  Total ← 0

  WHILE Total < 20
    Value ← Mix[Index]
    Total ← Total + Value

    IF Index < 4 THEN
      Mix[Index] ← Mix[Index] + Mix[Index+1]
    ELSE
      Mix[Index] ← Mix[Index] + Mix[1]
    ENDIF
    Index ← (Value MOD 4) + 1

  ENDWHILE

  Mix[1] ← Total * Index

ENDPROCEDURE
```

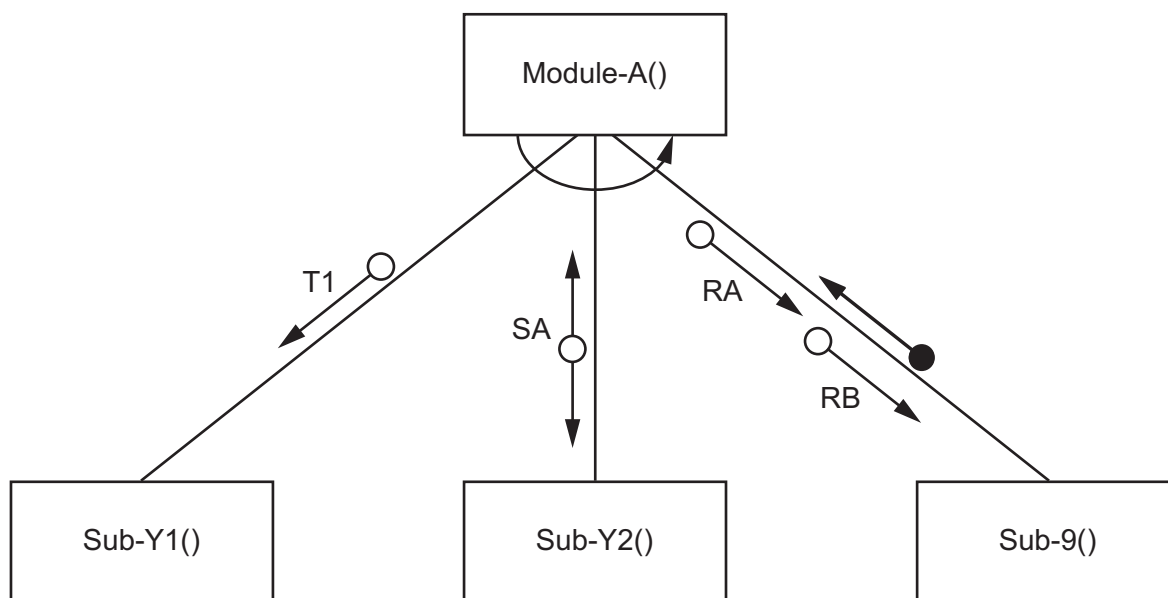
Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(2)
```

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]

51. O/N 23/P23/Q7/a

7 A structure chart shows the modular structure of a program:



(a) Explain the meaning of the curved arrow symbol which begins and ends at Module-A().

[2]

52. O/N 23/P23/Q8/b,d

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

```
<STX>"454202Hello Jack"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	• returns the data field from a message that has been received • If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	• takes a file name as a parameter of type string • creates a text file with the given file name (no checking required) • writes the data field returned by <code>GetData()</code> to the file • repeats until the data field is "*****", which is not written to the file • outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

- (b) The use of the string "*****" as explained in the module description for `ReceiveFile()` may cause a problem.

Explain the problem and suggest a solution.

Problem

Solution

(c) Two new modules are defined, which will allow two users to exchange messages.

Module	Description
<code>Transmit()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing a port number - transmits the message using the given port
<code>Chat()</code>	- takes two parameters: - a string representing a Destination ID - an integer representing a port number - extracts data from a received message using <code>GetData()</code> and outputs it - forms a message using data input by the user and sends it using <code>Transmit()</code> - repeats until either the output string or the sent string is "Bye"

Reminders:

- Each program contains a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.
- Messages are sent between computers as a string of characters organised into fields as shown:

`<STX><DestinationID><SourceID><Data><ETX>`

Write pseudocode for module `Chat()`.

Modules `GetData()` and `Transmit()` must be used.

[7]

(d) Module `GetData()` returns the data field from a message that has been received. If no message is available, the module waits until one has been received.

Explain the limitation of this on module `Chat()` from part **(c)**.

Describe a modification to `GetData()` to address this limitation.

Limitation

Modification

[3]

53. M/J 23/P21/Q5/a,b

- 5** Several companies are developing websites to market a new type of games console. The company that is first to create a website that can demonstrate the interactive features of the games console will have an advantage over the others. The requirements for the website are likely to change as more information about the features of the console are made available.

One company has decided to develop their website using a program development life cycle based on the waterfall model.

- (a) (i)** Give **two** reasons why this may **not** be the most appropriate model to use in this case.

Reason 1

Reason 2

[2]

- (ii)** Identify a **more appropriate** program development life cycle model for this scenario.

[1]

- (b)** The website has been running in test mode for several weeks.

Identify **and** describe a final stage of testing that should take place before the website is made available to all customers.

Stage

Description

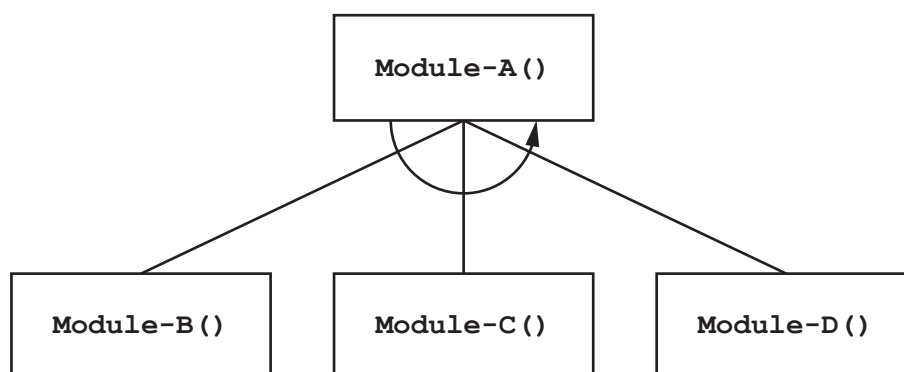
[3]

54. M/J 23/P21/Q7/b

- 7 A school has a computerised library system that allows students to borrow books for a fixed length of time. The system uses text files to store details of students, books and loans.

A new module is to be written which will generate emails to each student who has an overdue book.

- (b) The program designer produces a structure chart for the new module. Part of the structure chart is shown:



- (i) Explain the relationship between the four modules shown.

[2]

- (ii) Two new modules are added: `Module-X()` and `Module-Y()`.

- `Module-X()` has no parameters.
- `Module-Y()` will take a string and a real number as parameters and return a Boolean value.
- `Module-D()` will call either `Module-X()` or `Module-Y()`.

Draw **only** the part of the structure chart that represents the relationship between `Module-X()`, `Module-Y()` and `Module-D()`.

[3]

55. M/J 23/P21/Q8/c

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

<ItemNum><SupplierCode><Description>

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

(b) A new module has been defined:

Module	Description
<code>CheckNewItem()</code>	- called with a parameter of type string representing a line of item information - checks to see whether an item with the same <code>ItemNum</code> already exists in the file - returns <code>TRUE</code> if the <code>ItemNum</code> is not already in the file, otherwise returns <code>FALSE</code>

Write **efficient** pseudocode for module `CheckNewItem()`.

[7]

(c) The program modules `SuppExists()`, `IsNewSupp()` and `CheckNewItem()` are part of a group of modules that are combined to create a complete stock control program.

Each module in the program is tested individually during development and is debugged as necessary. It is then added to the program and further testing performed.

(i) Identify this method of testing.

[1]

(ii) One of the modules does not work properly when it is added to the program.

Describe a testing method that can be used to address this problem so that testing can continue and other modules can be added.

[2]

56. M/J 23/P22/Q5/a,b,d

- 5** A programmer has produced the following pseudocode to output the square root of the numbers from 1 to 10.

Line numbers are for reference only.

```
10 DECLARE Num : REAL
11 Num ← 1.0
...
40 REPEAT
41     CALL DisplaySqrt(Num)
42     Num ← Num + 1.0
43 UNTIL Num > 10
...
50 PROCEDURE DisplaySqrt(BYREF ThisNum : REAL)
51     OUTPUT ThisNum
52     ThisNum ← Sqrt(ThisNum)           // Sqrt returns the square root
53     OUTPUT " has a square root of ", ThisNum
54 ENDPROCEDURE
```

The pseudocode is correctly converted into program code.

Function `Sqrt()` is a library function and contains no errors.

The program code compiles without errors, but the program gives unexpected results. These are caused by a design error.

- (a)** Explain why the program gives unexpected results.

[3]

- (b)** Explain why the compiler does **not** identify this error.

[1]

- (d)** The pseudocode is converted into program code as part of a larger program.

During compilation, a complex statement generates an error.

The programmer does not want to delete the complex statement but wants to change the statement so that it is ignored by the compiler.

State how this may be achieved.

[1]

57. M/J 23/P22/Q7/b

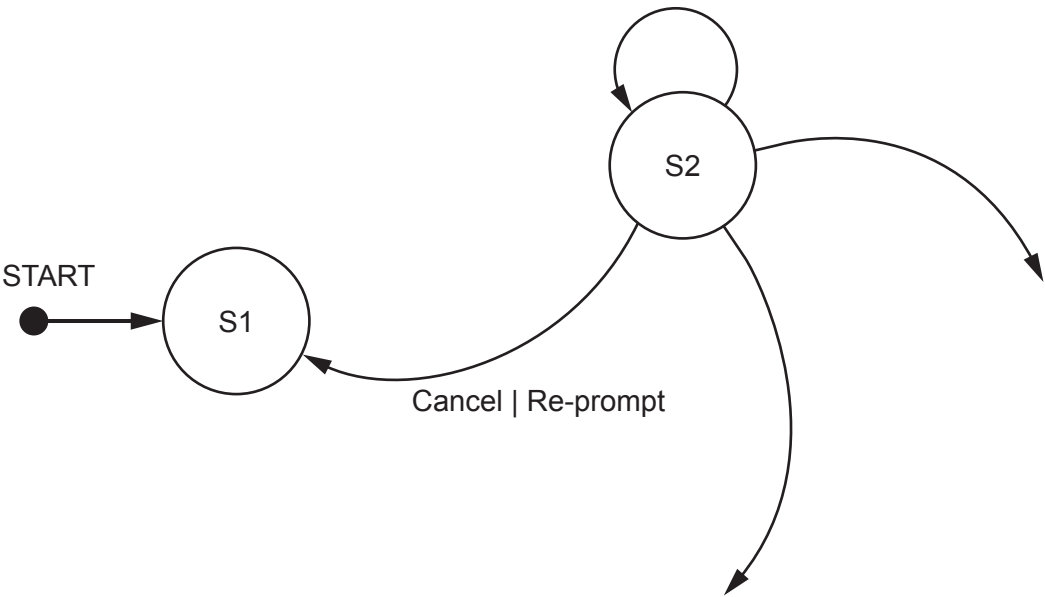
- 7 A computer system for a shop stores information about each customer. The items of information include name and address (both postal and email) together with payment details and order history. The system also stores the product categories they are interested in and how they would like to be contacted.
- (b) The program includes a module to validate a Personal Identification Number (PIN). This is used when customers pay for goods using a bank card.

A state-transition diagram has been produced for this module.

The table show the inputs, outputs and states for this part of the program:

Current state	Input	Output	Next state
S1	Input PIN		S2
S2	Re-input PIN	Display error	S2
S2	Cancel	Re-prompt	S1
S2	Valid PIN	Enable payment	S4
S2	Too many tries	Block Account	S3

Complete the state-transition diagram to represent the information given in the table.



58. M/J 23/P22/Q8/c

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

- (c) The program contains modules `SuppExists()` and `CheckSupplier()`. These have been written but contain errors. These modules are called from several places in the main program and testing of the main program (integration testing) has had to stop.

Identify a method that can be used to continue testing the main program before the errors in these modules have been corrected **and** describe how this would work.

Method

Description

59. M/J 23/P23/Q1/b

- 1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```
10  DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
...
100  FOR PCount ← 0 TO 49
101      Sheet4[1, PCount] ← 0
102      Sheet4[2, PCount] ← 47
103  NEXT PCount
```

- (b) The pseudocode contains two errors. One error is that variable `PCount` has not been declared.

Identify the **other** error **and** state the line number where it occurs.

Error

Line number

[2]

60. M/J 23/P23/Q5/a,b

- 5 A program is designed, coded and compiled without errors. The compiled code is sent for testing.

- (a) The program will be tested using the walkthrough method.

Additional information will be needed before this method can be used.

Identify this additional information **and** explain why it is needed.

Additional information

Explanation

[3]

- (b) Testing is completed and the program is made available to users.

Some time later, changes are made to the program to improve the speed of response.

State the type of maintenance that has been applied to the program.

[1]

61. M/J 23/P23/Q7/b

- 7 A school has a library system which allows students to borrow books for a length of time. Information relating to students and books is stored in text files. Student information includes name, home address, email address, date of birth, tutor and subject choices. Book information includes author, title, subject category, library location and the date that the book was borrowed.

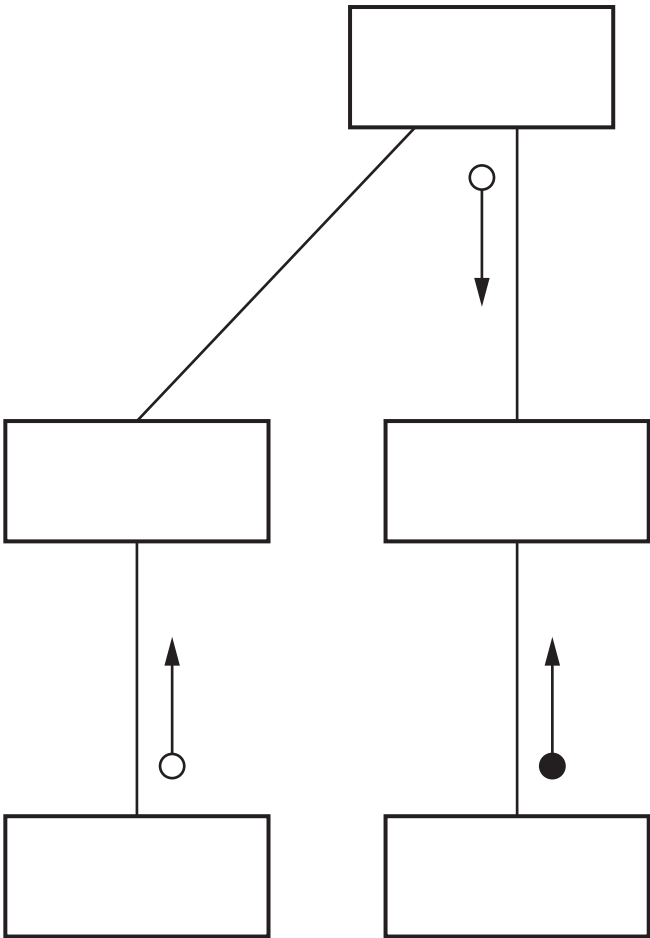
A program helps the staff to manage the borrowing of books.

(b) Part of the library program contains program modules with headers as follows:

Pseudocode module header
PROCEDURE Module-X()
PROCEDURE Module-Y(BYREF RA : INTEGER, SA : REAL)
PROCEDURE Overlay()
FUNCTION Replace(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN
FUNCTION Reset(TA : STRING) RETURNS INTEGER

Module-X() and Module-Y() are both called from module Overlay().

Complete the structure chart.



62. M/J 23/P23/Q8/c

- 8 A computer shop assembles desktop computers, using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range “0001” to “5999” inclusive
SupplierCode	3 alphabetic characters	code to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

The programmer has defined the first program module as follows:

Module	Description
<code>ChangeSupp()</code>	- called with two parameters <code>Code1</code> and <code>Code2</code> of type string that represent valid supplier codes - creates a new file <code>NewStock.txt</code> from the contents of the file <code>Stock.txt</code> where any reference to <code>Code1</code> is replaced by <code>Code2</code> - returns a count of the number of items that have had their supplier code changed

(b) A new module is required:

Module	Description
<code>Report_1()</code>	- takes a parameter of type string that represents a <code>SupplierCode</code> - searches the <code>Stock.txt</code> file for each line of item information that contains the given <code>SupplierCode</code> - produces a formatted report of items for the given <code>SupplierCode</code>, for example, for supplier DRG, the output could be: <pre>Report for Supplier: DRG Item Description 1234 USB Printer Cable 3m 1273 32GB USB Flash Drive 1350 Mouse Mat 320 x 240mm Number of items listed: 3</pre>

Write pseudocode for module `Report_1()`.

- (c) The format of the output from module `Report_1()` from **part (b)** is changed. The number of items listed is moved to the top of the report as shown in the example:

```
Report for Supplier: DRG
Number of items listed: 3
```

Item	Description
1234	USB Printer Cable 3m
1273	32GB USB Flash Drive
1350	Mouse Mat 320 x 240mm

- (i) Explain why this new layout would increase the complexity of the algorithm.

[2]

- (ii) The algorithm will be modified to produce the report in the new format. The modified algorithm will be implemented so that the file `Stock.txt` is only read once.

Describe the modified algorithm.

[3]

63. O/N 22/P21/Q6/b

- 6** The following pseudocode algorithm attempts to check whether a string is a valid email address.

```
FUNCTION IsValid(InString : STRING) RETURNS BOOLEAN
  DECLARE Index, Dots, Ats, Others : INTEGER
  DECLARE NextChar : CHAR
  DECLARE Valid : BOOLEAN

  Index ← 1
  Dots ← 0
  Ats ← 0
  Others ← 0
  Valid ← TRUE

  REPEAT
    NextChar ← MID(InString, Index, 1)
    CASE OF NextChar
      '.' : Dots ← Dots + 1
      '@' : Ats ← Ats + 1
            IF Ats > 1 THEN
              Valid ← FALSE
            ENDIF
      OTHERWISE : Others ← Others + 1
    ENDCASE

    IF Dots > 1 AND Ats = 0 THEN
      Valid ← FALSE
    ELSE
      Index ← Index + 1
    ENDIF

  UNTIL Index > LENGTH(InString) OR Valid = FALSE

  IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
    Valid ← FALSE
  ENDIF

  RETURN Valid

ENDFUNCTION
```

- (b) (i)** Complete the trace table by dry running the function when it is called as follows:

Result $\leftarrow$ IsValid("Liz.123@big@net")

Index	NextChar	Dots	Ats	Others	Valid

[5]

- (ii)** State the value returned when IsValid() is called using the expression shown in part (b)(i).

[1]

64. O/N 22/P21/Q7/b

7 A simple arithmetic expression is stored as a string in the format:

<Value1><Operator><Value2>

An operator character is one of the following: '+' '-' '*' '/'

Example arithmetic expression strings:

"803+1904"

"34/7"

(b) A string representing an arithmetic expression could be in the correct format but be impossible to evaluate.

Give an example of a correctly formatted string and explain why evaluation would be impossible.

Example string

Explanation

[2]

65. O/N 22/P21/Q8/d

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	- takes the name of an existing project file as a parameter of type string - returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	- takes a line from a project file as a parameter of type string - returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	- takes two parameters: - the name of a project file as a string - the maximum number of errors as an integer - uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid - calls <code>CheckLine()</code> for each line in the project file - counts the number of errors - outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (d) Module `CheckLine()` includes a check for syntax errors.

Two examples of syntax error that **cannot** be detected from examining a **single** line are those involving selection and iteration.

Give **two other** examples.

1

2

[2]

66. O/N 22/P22/Q1/b

- 1 (a) A programmer is developing an algorithm to solve a problem. Part of the algorithm would be appropriate to implement as a subroutine (a procedure or a function).

(i) State **two** reasons why the programmer may decide to use a subroutine.

1

2

[2]

(ii) A procedure header is shown in pseudocode:

```
PROCEDURE MyProc (Count : INTEGER, Message : STRING)
```

Give the correct term for the identifiers `Count` and `Message` **and** explain their use.

Term

Use

[2]

- (b) The algorithm in **part (a)** is part of a program that will be sold to the public.
All the software errors that were identified during in-house testing have been corrected.

Identify **and** describe the additional test stage that may be carried out before the program is sold to the public.

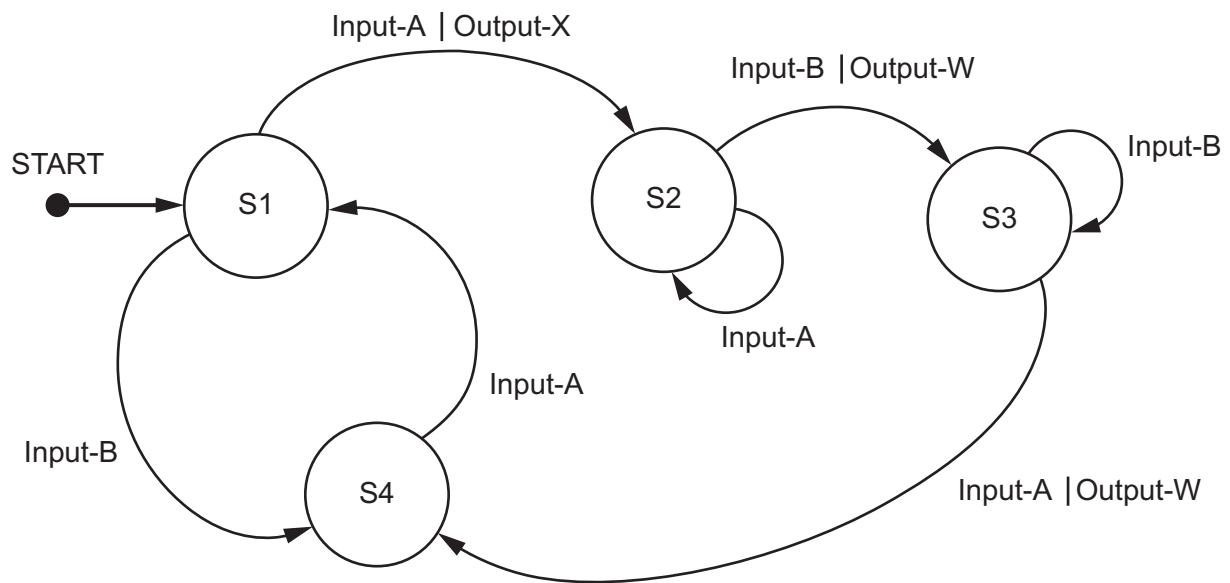
Test stage

Description

[4]

67. O/N 22/P22/Q2/b

(b) Examine the following state-transition diagram.



Complete the table to show the inputs, outputs and next states.

Input	Output	Next state
		S1
Input-A		
		S2
	Output-W	
	Output-W	

[4]

68. O/N 22/P22/Q6/b

- 6 (a) The factorial of an integer number is the product of all the integers from that number down to 1.

In general, the factorial of n is $n \times (n-1) \times \dots \times 2 \times 1$

For example, the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1 = 120$

In this question, n will be referred to as the `BaseNumber`.

A function `FindBaseNumber()` will:

- be called with a positive, non-zero integer value as a parameter

- return `BaseNumber` if the parameter value is the factorial of the `BaseNumber`
- return `-1` if the parameter value **is not** a factorial.

For example:

Parameter value	Value returned
120	5
12	-1
6	3
1	1

`FindBaseNumber(12)` will return `-1` because 12 is not a factorial.

You may use the rest of this page for rough working.

Write pseudocode for the function `FindBaseNumber()`.

[7]

- (b) A program is written to allow a user to input a sequence of values to be checked using the function `FindBaseNumber()`.

The user will input one value at a time. The variable used to store the user input has to be of type string because the user will input 'End' to end the program.

Valid input will be converted to an integer and passed to `FindBaseNumber()` and the return value will be output.

Complete the table by giving **four** invalid strings that may be used to test distinct aspects of the required validation. Give the reason for your choice in each case.

Input	Reason for choice

[4]

69. O/N 22/P23/Q1/b,c,d

1 A program is required for a shopping website.

(b) The designer considers the use of a development life cycle to split the development of the website into several stages.

(i) State **one** benefit of a development life cycle when developing the website.

[1]

(ii) Analysis is one stage of a development life cycle.

State **one** document that may be produced from the analysis stage of the website project.

[1]

(c) The program will be developed using the Rapid Application Development (RAD) life cycle.

(i) State **one** principle of this life cycle.

[1]

(ii) Give **two** benefits and **one** drawback of its use compared to the waterfall life cycle.

Benefit 1

Benefit 2

Drawback

[3]

(d) Adaptive maintenance needs to be carried out on the website program.

Give **two** reasons why adaptive maintenance may be required.

1

2

[2]

70. O/N 22/P23/Q5/d

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (d) The program that includes `Unpack()` is tested using the walkthrough method.

Describe this method **and** explain how it can be used to identify an error.

[3]

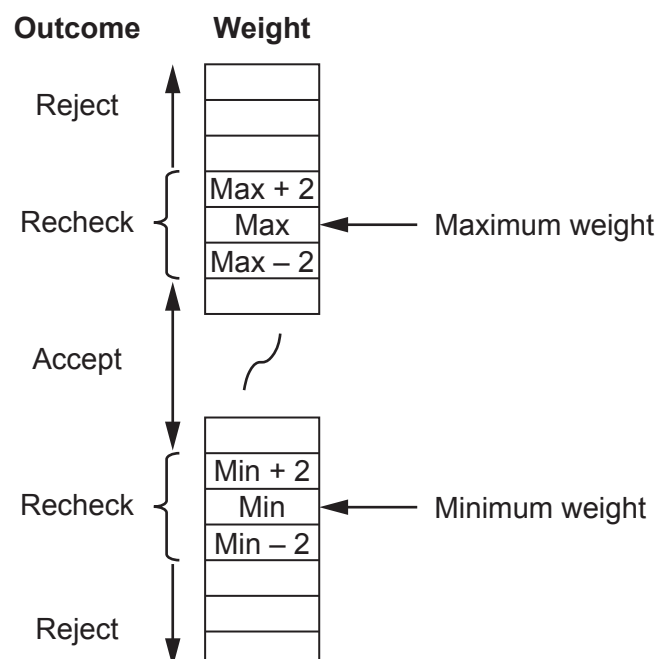
71. O/N 22/P23/Q6/a

- 6 Components are weighed during manufacture. Weights are measured to the nearest whole gram.

Components that weigh at least 3 grams more than the maximum weight, or at least 3 grams less than the minimum weight, are rejected.

A component is rechecked if it weighs within 2 grams of either the maximum or minimum weight.

The final outcome of weighing each component is shown below:



A function `Status()` will be called with three parameters. These are integers representing the weight of an individual component together with the minimum and maximum weights.

The value returned from the function will be as follows:

Outcome	Return value
Accept	'A'
Reject	'R'
Recheck	'C'

- (a) Complete the following test plan for **five** tests that could be performed on function `Status()`. The tests should address all possible outcomes.

Test number	Component weight	Min	Max	Expected return value
1				'A'
2				
3				
4				
5				

[5]

72. O/N 21/P21/Q1/a

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

- (a) (i) State what her colleague means by “does not contain any syntax errors”.

[1]

- (ii) Identify **and** describe **one** other type of error that the program may contain.

Type of error

Description

[2]

73. O/N 21/P21/Q4/a,b

4 A program controls the heating system in a sports hall.

Part of the program involves reading a value from a sensor. The sensor produces a numeric value that represents the temperature. The value is an integer, which should be in the range 0 to 40 inclusive.

A program function has been written to validate the values from the sensor.

(a) A test plan is needed to test the function.

Complete the table. The first line has been completed for you.

You can assume that the sensor will generate only integer data values.

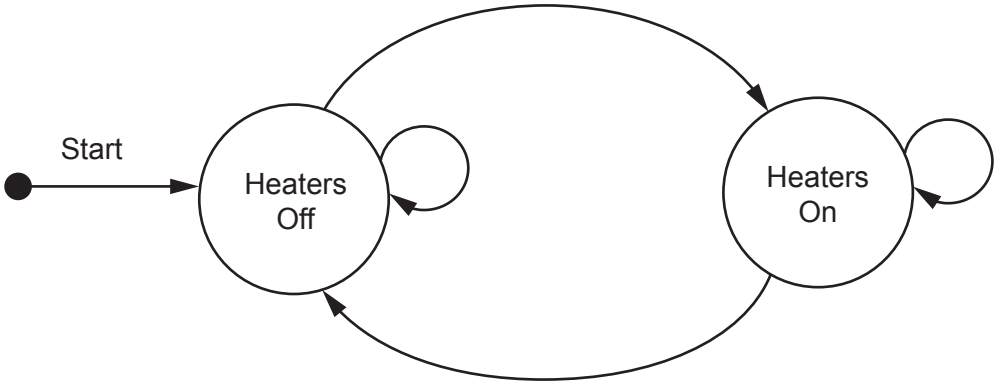
Test	Test data value	Explanation	Expected outcome
1	23	Normal data	Data is accepted
2			
3			
4			
5			

[4]

(b) A program module controls the heaters. This module operates as follows:

- If the temperature is below 10, switch the heaters on.
- If the temperature is above 20, switch the heaters off.

Complete the following state-transition diagram for the heating system:



[3]

74. O/N 21/P21/Q5/b

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (b) The choice of data structure was made during one stage of the program development life cycle.

Identify this stage.

[1]

75. O/N 21/P22/Q2/b

- (b) A software company is working on a project to develop a website for a school.

The school principal has some ideas about the appearance of the website but is unclear about all the details of the solution. The principal would like to see an initial version of the website.

- (i) Identify a life cycle method that would be appropriate in this case.

Give a reason for your choice.

Life cycle method

Reason

[2]

- (ii) The website project has progressed to the design stage.

State **three** activities that will take place during the design stage of the program development life cycle.

1

2

3

[3]

76. O/N 21/P22/Q6/c(i)

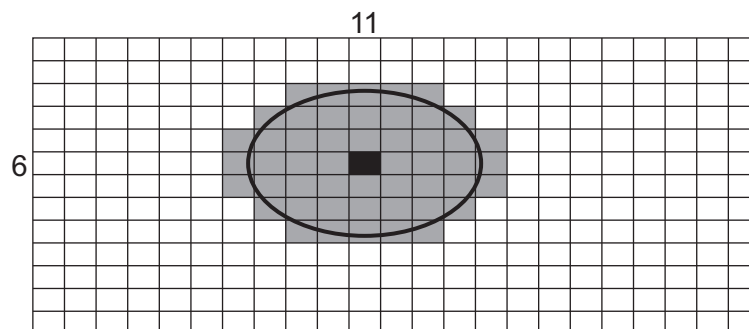
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will be set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	• Searches for the first row that has an array element set to 1 • Returns the index of that row (1 is the first row) • Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	• Searches for the last row that has an array element set to 1 • Returns the index of that row • Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	• Searches for the first column that has an array element set to 1 • Returns the index of that column (1 is the first column) • Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	• Searches for the last column that has an array element set to 1 • Returns the index of that column • Returns -1 if there are no elements set to 1

(c) The programmer decides to produce a **single** search module `FindSet()`, which will be able to perform each of the individual searches performed by the first four modules in the table.

- (i) Outline the changes needed to convert one of the existing modules into this single module.

[2]

77. O/N 21/P23/Q1/a

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(a) (i) State what her colleague means by “does not contain any syntax errors”.

[1]

(ii) Identify **and** describe **one** other type of error that the program may contain.

Type of error

Description

[2]

78. O/N 21/P23/Q4/a,b

4 A program controls the heating system in a sports hall.

Part of the program involves reading a value from a sensor. The sensor produces a numeric value that represents the temperature. The value is an integer, which should be in the range 0 to 40 inclusive.

A program function has been written to validate the values from the sensor.

(a) A test plan is needed to test the function.

Complete the table. The first line has been completed for you.

You can assume that the sensor will generate only integer data values.

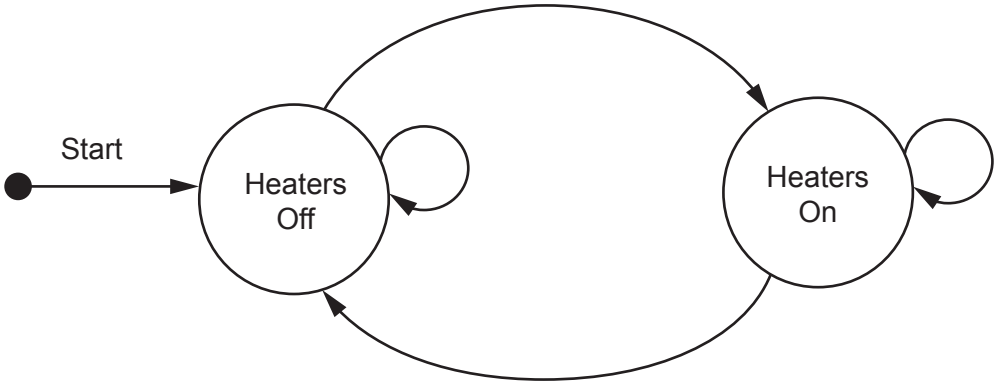
Test	Test data value	Explanation	Expected outcome
1	23	Normal data	Data is accepted
2			
3			
4			
5			

[4]

(b) A program module controls the heaters. This module operates as follows:

- If the temperature is below 10, switch the heaters on.
- If the temperature is above 20, switch the heaters off.

Complete the following state-transition diagram for the heating system:



[3]

79. O/N 21/P23/Q5/b

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (b) The choice of data structure was made during one stage of the program development life cycle.

Identify this stage.

[1]

80. M/J 21/P21/Q2/a

- 2 (a)** Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
UpdateLoan()	• Calls either <code>LoanExtend()</code> or <code>LoanReturn()</code>
LoanExtend()	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Calls <code>CheckReserve()</code> to see whether the book has been reserved for another library user • Returns <code>TRUE</code> if the loan has been extended, otherwise returns <code>FALSE</code>
CheckReserve()	• Called with <code>BookID</code> • Returns <code>TRUE</code> if the book has been reserved, otherwise returns <code>FALSE</code>
LoanReturn()	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Returns a <code>REAL</code> (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

81. M/J 21/P21/Q4/a,c

4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert (Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH (Name)
24     ThisChar ← MID (Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE (ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(a) Complete the trace table below by dry running the function when it is called as follows:

Result ← Convert ("VinVaVV Cup")

Note: The symbol '∇' has been used to represent a space character.
Use this symbol for any space characters in the trace table.

Write the modified line.

[1]

- (ii) Change 2 may be implemented by moving one line of the pseudocode.

Write the number of the line to be moved and state its new position.

Line number

New position

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Characters that are not upper case alphabetic are returned unchanged

Example: LCASE('W') returns 'w'

UCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Characters that are not lower case alphabetic are returned unchanged

Example: UCASE('a') returns 'A'

TO_UPPER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case

Example: TO_UPPER("Error 803") returns "ERROR 803"

TO_LOWER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to lower case

Example: TO_LOWER("JIM 803") returns "jim 803"

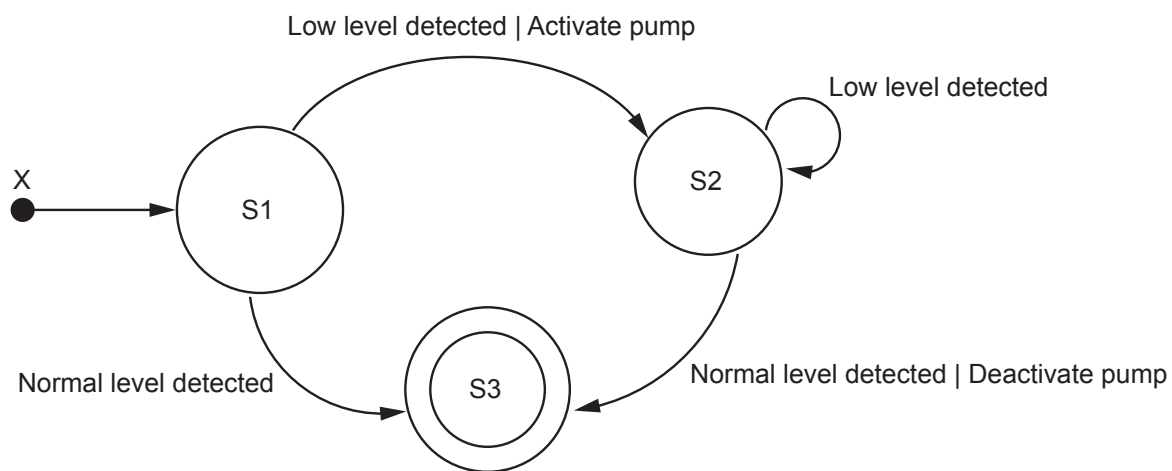
NUM_TO_STR(x : <data type>) RETURNS STRING
returns a string representation of a numeric value

Note: <data type> may be REAL or INTEGER

Example: NUM_TO_STR(87.5) returns "87.5"

82. M/J 21/P22/Q2/a

2 (a) Examine the following state-transition diagram.



(i) Complete the table with reference to the diagram.

Answer

The number of transitions that result in a different state	
The number of transitions with associated outputs	
The label that should replace 'X'	
The final or halting state	

[4]

(ii) The current state is S1. The following inputs occur.

1. Low level detected
2. Low level detected
3. Low level detected
4. Low level detected

Give the number of outputs and the current state.

Number of outputs

Current state

[2]

83. M/J 21/P22/Q7/a,b

7 A procedure, `FormatName()`:

- is called with a string containing words and spaces as its parameter. In this context, a word is any sequence of characters that does not contain a space character.
- creates a new formatted string from this string with the following requirements:
 1. Any leading spaces removed (spaces before the first word).
 2. Any trailing spaces removed (spaces after the last word).
 3. Any multiple spaces between words converted to a single space.
 4. All characters converted to lower case.

The `FormatName()` procedure has been written in a programming language and is to be tested using the black-box method.

- (a) Give a test string that could be used to show that all **four** formatting requirements have been applied correctly.

Use the symbol '∇' to represent a space character.

[3]

- (b) The `FormatName()` procedure should assign a value to the global variable `FString`.

There is a fault in the program, which means that the assignment does not always take place.

Explain **two** ways of exposing the fault.

[2]

84. M/J 21/P23/Q2/a

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
<code>UpdateLoan()</code>	• Calls either <code>LoanExtend()</code> or <code>LoanReturn()</code>
<code>LoanExtend()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Calls <code>CheckReserve()</code> to see whether the book has been reserved for another library user • Returns <code>TRUE</code> if the loan has been extended, otherwise returns <code>FALSE</code>
<code>CheckReserve()</code>	• Called with <code>BookID</code> • Returns <code>TRUE</code> if the book has been reserved, otherwise returns <code>FALSE</code>
<code>LoanReturn()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Returns a <code>REAL</code> (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

85. M/J 21/P23/Q4/a,c

4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert (Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH (Name)
24     ThisChar ← MID (Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE (ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(a) Complete the trace table below by dry running the function when it is called as follows:

Result ← Convert ("∇in∇a∇∇Cup")

Note: The symbol '∇' has been used to represent a space character.
Use this symbol for any space characters in the trace table.

Write the modified line.

[1]

- (ii) Change 2 may be implemented by moving one line of the pseudocode.

Write the number of the line to be moved and state its new position.

Line number

New position

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Characters that are not upper case alphabetic are returned unchanged

Example: LCASE('W') returns 'w'

UCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Characters that are not lower case alphabetic are returned unchanged

Example: UCASE('a') returns 'A'

TO_UPPER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case

Example: TO_UPPER("Error 803") returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>)` RETURNS STRING
returns a string representation of a numeric value

Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

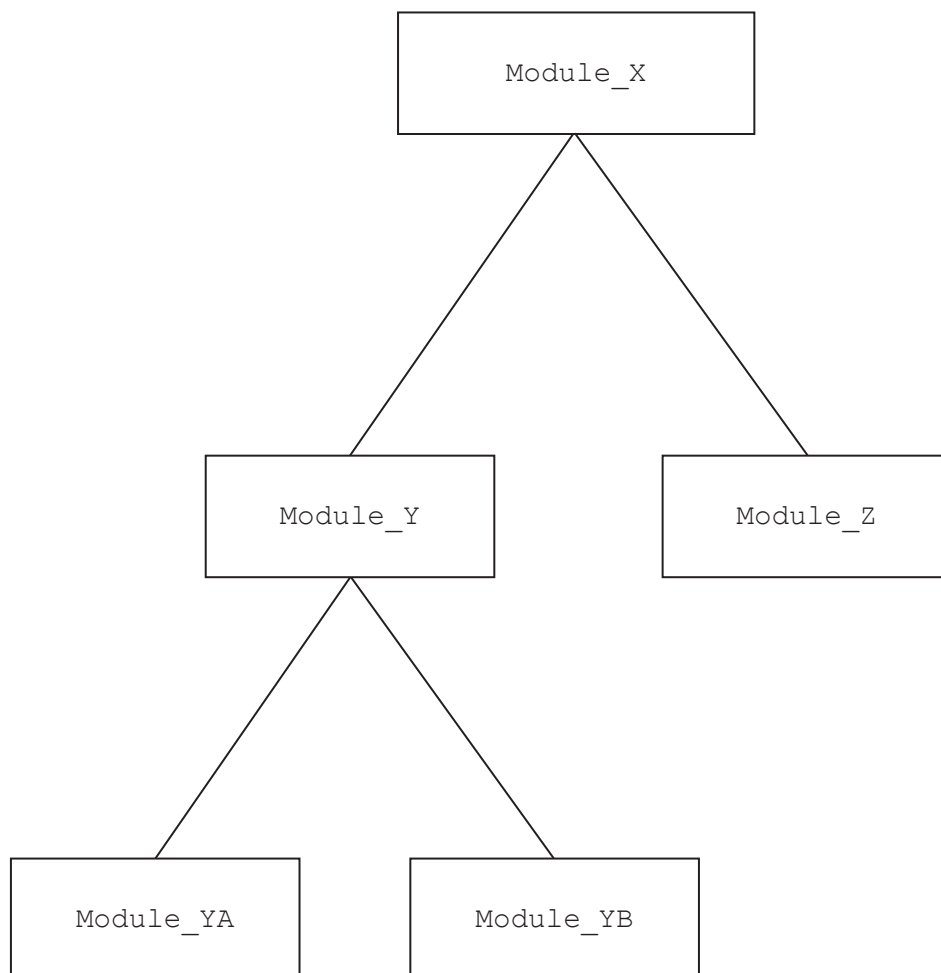
86. 9608 M/J 21/P21/Q3/a,b

also Unit 11 (2 marks); this unit 8 marks

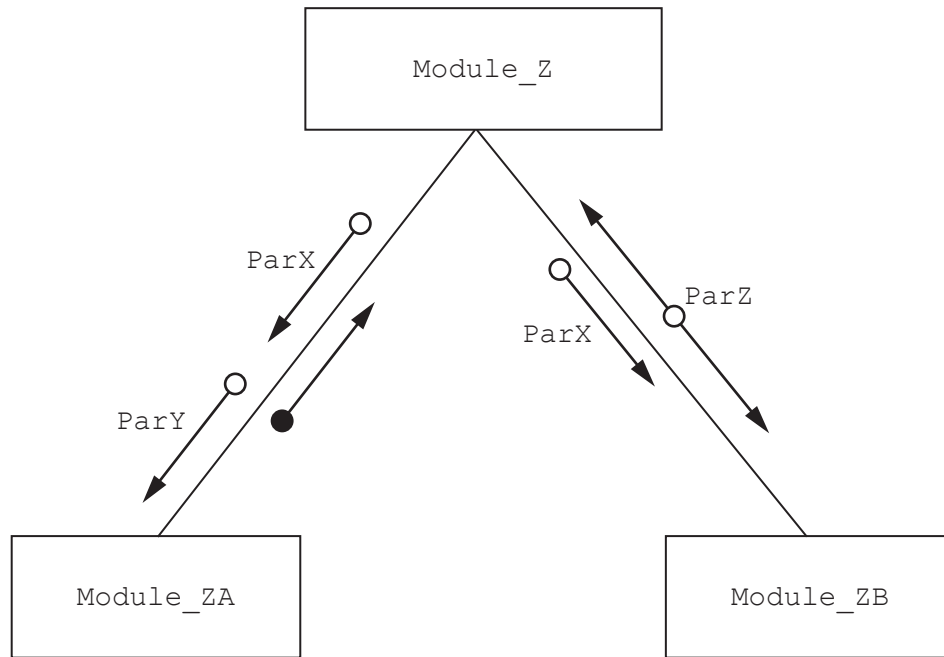
- 3 (a)** The following table contains information about five modules in a program. It describes the calls made and the parameters passed.

Module	Description
Module_X	- repeatedly calls Module_Y then Module_Z - passes a parameter of type REAL to Module_Y - passes two parameters of type INTEGER to Module_Z
Module_Y	calls either Module_YA or Module_YB
Module_Z	called with two parameters of type INTEGER
Module_YA	- called with a parameter of type REAL - parameter is passed by reference
Module_YB	- called with a parameter of type INTEGER - returns a BOOLEAN value

Complete the structure chart to include the information given about the five modules.



(b) Two more modules are added to the chart below `Module_Z` as shown:



Parameter data types are:

```

ParX  : REAL
ParY  : INTEGER
ParZ  : STRING
  
```

(i) State whether `Module_ZA()` is a function or a procedure **and** justify your choice.

[2]

(ii) Write the **pseudocode** header for `Module_ZB()`.

[3]

87. 9608 M/J 21/P21/Q4/c

4 The following is part of a program written in pseudocode:

```

DECLARE ThisArray : ARRAY[1:1000] OF STRING
DECLARE ArrayResult : INTEGER
  
```

~

```

PROCEDURE ScanArray(SearchString : STRING)

  DECLARE Index, Total : INTEGER
  DECLARE Error : BOOLEAN

  Index ← 1
  Total ← 0
  Error ← FALSE

  WHILE Index <= 1000 AND Error <> TRUE
    IF LENGTH(ThisArray[Index]) > 5
      THEN
        IF ThisArray[Index] = SearchString
          THEN
            Total ← Total + LENGTH(ThisArray[Index])
          ENDIF
        Index ← Index + 1
      ELSE
        Error ← TRUE
      ENDIF
    ENDWHILE

  ArrayResult ← INT(Total / (Index - 1))

ENDPROCEDURE

```

The procedure `ScanArray()` is amended as follows:

- `SearchString` is compared with just the first **four** characters of each array element.
- The total ignores the first **five** characters of each array element.
- When calculating `ArrayResult`, prevent any possible division by zero.

Refer to the **Appendix** on pages 22–23 for the list of built-in pseudocode functions and operators.

(c) (i) Identify the first stage in the program development cycle.

State the tasks that are completed during this stage.

First stage

Tasks

[3]

(ii) A program will be translated using a compiler.

Identify the stage of the program development cycle where a syntax error may occur.

[1]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
If `ThisChar` is not an upper case alphabetic character, it is returned unchanged.

Example: `LCASE('W')` returns 'w'

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`RAND(x : INTEGER)` RETURNS REAL
returns a real number in the range 0 to `x` (not inclusive of `x`).

Example: `RAND(87)` could return 35.43

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

88. 9608 M/J 21/P21/Q5/b(i,ii)

- 5 Alice is developing an algorithm for a game. The game will count the number of attempts by a player to guess the secret number generated by the built-in random number function.

She will use pseudocode to develop a procedure, `GuessNum()`, that will:

1. generate a secret number that is a random integer in the range 1 to 20 inclusive
2. prompt and input the player's guess
3. output a message if the guess is not the same as the secret number and repeat from step 2
4. count the number of attempts it takes to guess the secret number
5. output a message when the secret number has been guessed correctly, giving the number of attempts.

- (b) Alice is converting her pseudocode into a high-level language for use in a larger modular program.

She wants to start testing the program before all the subroutines (procedures and functions) have been implemented.

- (i) Identify this type of testing.

[1]

- (ii) Her program contains a function `Status()` that she has not yet written, but will be called from several places within the program.

Explain what Alice needs to do to allow the program to be tested.

[2]

89. 9608 M/J 21/P22/Q1/a

- 1 (a) Maintenance of programs may be needed for a number of different reasons.

State **two** types of maintenance **and** give a reason why each may be needed.

Type

Reason

Type

Reason

[4]

90. 9608 M/J 21/P22/Q3/a,b

- 3 (a) (i) Module names and parameters are features that may be represented on a structure chart.

State **two other** features than can be represented on a structure chart.

Feature 1

Feature 2

[2]

- (ii) The headers for three modules in a program are defined in pseudocode as follows:

Pseudocode module header
PROCEDURE Create(S2 : INTEGER, P3 : STRING)
PROCEDURE Modify(S2 : INTEGER, BYREF P4 : STRING)
FUNCTION Delete(P4 : INTEGER, M4 : STRING) RETURNS INTEGER

A fourth module, `Membership()`, may call any one of the three modules.

Draw a structure chart to represent the information given about the **four** modules.

[5]

- (b) Draw a diagram to show the stages of the program development cycle. Use arrows to indicate how the stages are linked.



[2]

91. 9608 M/J 21/P23/Q2/a,c

- 2 (a) After using a program for some time, a user notices a fault in the program.

Describe the term **program fault**.

[2]

- (c) A programmer chooses data to test each path through her program.

Identify the type of testing that the programmer has decided to perform.

[1]

92. 9608 M/J 21/P23/Q4/a,b,c

- 4 Study the following pseudocode for a string handling function `Check()`.

Refer to the **Appendix** on pages 22 and 23 for the list of built-in pseudocode functions and operators.

```
FUNCTION Check(InString : STRING) RETURNS INTEGER

DECLARE Index, Result, Count : INTEGER
DECLARE NextChar : CHAR

Result ← 0
Count ← 1

FOR Index ← 1 TO LENGTH(InString)

    NextChar ← MID(InString, Index, 1)
    IF (NextChar >= '0' AND NextChar <= '9') OR NextChar = '.'
    THEN
        Result ← Result + 1
    ELSE
        IF NextChar = ','
        THEN
            Count ← Count + 1
        ELSE
            Result ← -1
        ENDIF
    ENDIF
ENDFOR

IF Count < 3
    THEN
        RETURN -1
    ELSE
        RETURN Result
    ENDIF

ENDFUNCTION
```

- (a) (i) Complete the trace table by performing a dry run of the function when it is called as follows:

Answer ← Check("74.0,4.6,3x2")

- a count of the number groups in the parameter passed

or

- -1 if there are less than three number groups in the string **or** if any non-numeric characters occur in the string (other than decimal point and comma).

There is an error in the algorithm causing an incorrect value to be returned by the function.

- (i) Explain why this error can occur.

[2]

- (ii) Describe how the algorithm could be amended to correct the error.

[1]

- (c) A dry run of a pseudocode algorithm may help to locate logic errors.

Give **another** type of program error and describe how it can occur.

Type of error

Description

[2]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.
Note: This function will also work if `x` is of type CHAR

Example: `STRING_TO_NUM("23.45")` returns 23.45

`IS_NUM(ThisString : STRING)` RETURNS BOOLEAN
returns the value TRUE if `ThisString` contains only numeric characters ('0' to '9').

Example: `IS_NUM("123a")` returns FALSE

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

93. 9608 M/J 21/P23/Q5/b

- 5 A program stores a contact list of telephone numbers. Each telephone number is stored as a string of six or more numeric characters.

Before they are displayed, number strings are formatted to make them easier to read. This involves forming the characters into groups, separated by the space character.

The maximum length of a number group is five characters.

Different numbers may have different groupings. A template string is used to define the grouping.

For example:

Number string	Template string	Formatted string
"01223553998"	"53"	"01223 553 998"
"509700101"	"222"	"50 97 00 101"
"4044496128"	"33"	"404 449 6128"

For the first row, template "53" results in a formatted string comprising:

- the first five characters in the first group
- a space character
- the next three characters in the second group
- a space character
- the remaining characters from the number string.

- (a) Write **pseudocode** for a function `GroupNum()`, which takes a telephone number and a template as parameter strings and returns a formatted string.

You may assume that the template and telephone number are valid.

Refer to the **Appendix** on pages 22 and 23 for the list of built-in pseudocode functions and operators.

[8]

- (b) The function `GroupNum()` is to be extended to include parameter checking.

State **one** check that could be applied to **each** parameter.

Give an example of test data that could be used to demonstrate that each check identifies invalid data.

The type of check must be different for each parameter.

Telephone number check

Test data

Template check

Test data

[4]

94. M/J 20/P21/Q2/a,b

also Unit 11 (4 marks); this unit 9 marks

- 2 (a)** Three modules form part of a program for a car rental company. A description of the relationship between the modules is summarised as follows:

Module name	Description
RentCar()	A customer will pay for each car rental either by bank card or by using their account with the rental company.
PayByCard()	Called with parameter <code>HireCost</code> , representing the cost of the rental. Returns a <code>BOOLEAN</code> value to indicate whether or not the card payment was successful.
PayByAccount()	Called with parameters <code>HireCost</code> , <code>AccountNumber</code> , <code>CurrentBalance</code> and <code>AccountLimit</code> . • Checks whether <code>HireCost</code> plus the <code>CurrentBalance</code> would exceed the <code>AccountLimit</code>. If so, then the rental is not authorised. • If the rental is authorised, then the <code>CurrentBalance</code> is updated. • Returns a <code>BOOLEAN</code> value to indicate whether or not the rental was authorised.

Draw a structure chart to show the relationship between the **three** modules and the parameters passed between them.

[5]

- (b)** The following pseudocode algorithm has been developed to check whether a string contains a valid password.

To be a valid password, a string must:

- be longer than 6 characters
- contain at least one lower case letter
- contain at least one upper case letter
- contain at least one non-alphabetic character.

```
10 FUNCTION Check(InString : STRING) RETURNS BOOLEAN
11
12   DECLARE Index : INTEGER
13   DECLARE StrLen : INTEGER
14   DECLARE NumUpper, NumLower : INTEGER
15   DECLARE NumNonAlpha : INTEGER
16   DECLARE NextChar : CHAR
17
18   NumUpper ← 0
19   NumLower ← 0
20   NumNonAlpha ← 0
21
22   StrLen ← LENGTH(InString)
23   IF StrLen < 7
24     THEN
25       RETURN FALSE
26     ELSE
27       FOR Index ← 1 TO StrLen
28         NextChar ← MID(InString, Index, 1)
29         IF NextChar >= 'a' AND NextChar <= 'z'
30           THEN
31             NumLower ← NumLower + 1
32           ELSE
33             IF NextChar > 'A' AND NextChar <= 'Z'
34               THEN
35                 NumUpper ← NumUpper + 1
36               ELSE
37                 NumNonAlpha ← NumNonAlpha + 1
38             ENDIF
39           ENDIF
40       ENDFOR
41     ENDIF
42
43   IF (NumUpper >= 1) AND (NumLower >= 1) AND (NumNonAlpha >= 1)
44     THEN
45       RETURN TRUE
46     ELSE
47       RETURN FALSE
48   ENDIF
49
50 ENDFUNCTION
```

Refer to the **Appendix** on page 21 for a list of built-in pseudocode functions and operators.

The pseudocode does not work under all circumstances.

A dry run performed on the function `Check()`, with the string "crAsh99", produced the following trace table.

The string is a valid password, but the pseudocode would return the value `FALSE`.

Trace table row	StrLen	Index	NextChar	NumUpper	NumLower	NumNonAlpha
1	7			0	0	0
2		1	'c'			
3					1	
4		2	'r'			
5					2	
6		3	'A'			
7						1
8		4	's'			
9					3	
10		5	'h'			
11					4	
12		6	'9'			
13						2
14		7	'9'			
15						3

- (i) Describe how the completed trace table may be used to identify the error in the pseudocode. In your answer, refer to the trace table row number(s).

[2]

- (ii) State the **pseudocode** line number that has to be changed to correct the error **and** write the correct pseudocode for the complete line.

Line number

Correct pseudocode

[2]

- (iii) Rewrite lines 29 to 39 of the original pseudocode using a `CASE` structure.

[4]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

95. M/J 20/P21/Q4/a,b,c

- 4 (a) An inexperienced user buys a games program. A program fault occurs while the user is playing the game.

Explain what is meant by a **program fault**.

[2]

- (b) Give **three** ways to minimise the risk of faults when writing programs.

1

2

3

[3]

- (c) Three types of program error are syntax, logic and run-time.

Define these **three** types.

Syntax error

Logic error

Run-time error

[3]

96. M/J 20/P21/Q6/c(ii)

- 6 A company hires out rowing boats on a lake. The company has ten boats, numbered from 1 to 10.

The company is developing a program to help manage and record the hiring out process.

Hire information is stored in three global 1D arrays when a boat is hired out. Each array contains 10 elements representing each of the ten boats.

The three 1D arrays are summarised as follows:

Array	Data type	Description	Example data value
HireTime	STRING	The time the boat was hired out	"10:15"
Duration	INTEGER	The number of minutes of the hire	30
Cost	REAL	The cost of the hire	5.75

If an individual boat is not currently on hire, the corresponding element of the `HireTime` array will be set to "Available".

The programmer has started to define program modules as follows:

Module	Description
<code>AddTime()</code>	- Called with two parameters: - a <code>STRING</code> value representing a time - an <code>INTEGER</code> value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a <code>STRING</code>
<code>ListAvailable()</code>	- Called with a <code>STRING</code> value representing the time the hire will start - Outputs the boat numbers that will be available for hire at the given start time. A boat will be available for hire if it is either: - currently not on hire, or - due back before the given hire start time - Outputs the number of each boat available - Outputs the total number of boats available or a suitable message if there are none
<code>RecordHire()</code>	- Called with four parameters: - an <code>INTEGER</code> value representing the boat number - a <code>STRING</code> value representing the hire start time - an <code>INTEGER</code> value representing the hire duration in minutes - a <code>REAL</code> value representing the cost of hire - Updates the appropriate element in each array - Adds the cost of hire to the global variable <code>DailyTakings</code> - Converts the four input parameters to strings, concatenated using commas as separators, and writes the resulting string to the end of the existing text file <code>HireLog.txt</code>

(c) The module description of `AddTime()` is repeated here for reference.

Module	Description
<code>AddTime()</code>	- Called with two parameters: - a <code>STRING</code> value representing a time - an <code>INTEGER</code> value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a <code>STRING</code>

(ii) The function `AddTime()` is to be tested using black-box testing.

Complete the following **two** tests that can be performed to check the operation of the function. Note that test 1 and test 2 are different.

Test 1 – Boat is returned during the same hour as rental starts

Start time value

Duration value

Expected new time value

Test 2 – Boat is returned during the hour after the rental starts

Start time value

Duration value

Expected new time value

[2]

97. M/J 20/P22/Q2/a,b(ii)

- 2 (a) A structure chart is often produced as part of a modular program design. The chart shows the hierarchy of modules and the sequence of execution.

Give **two other** features the structure chart can show.

Feature 1

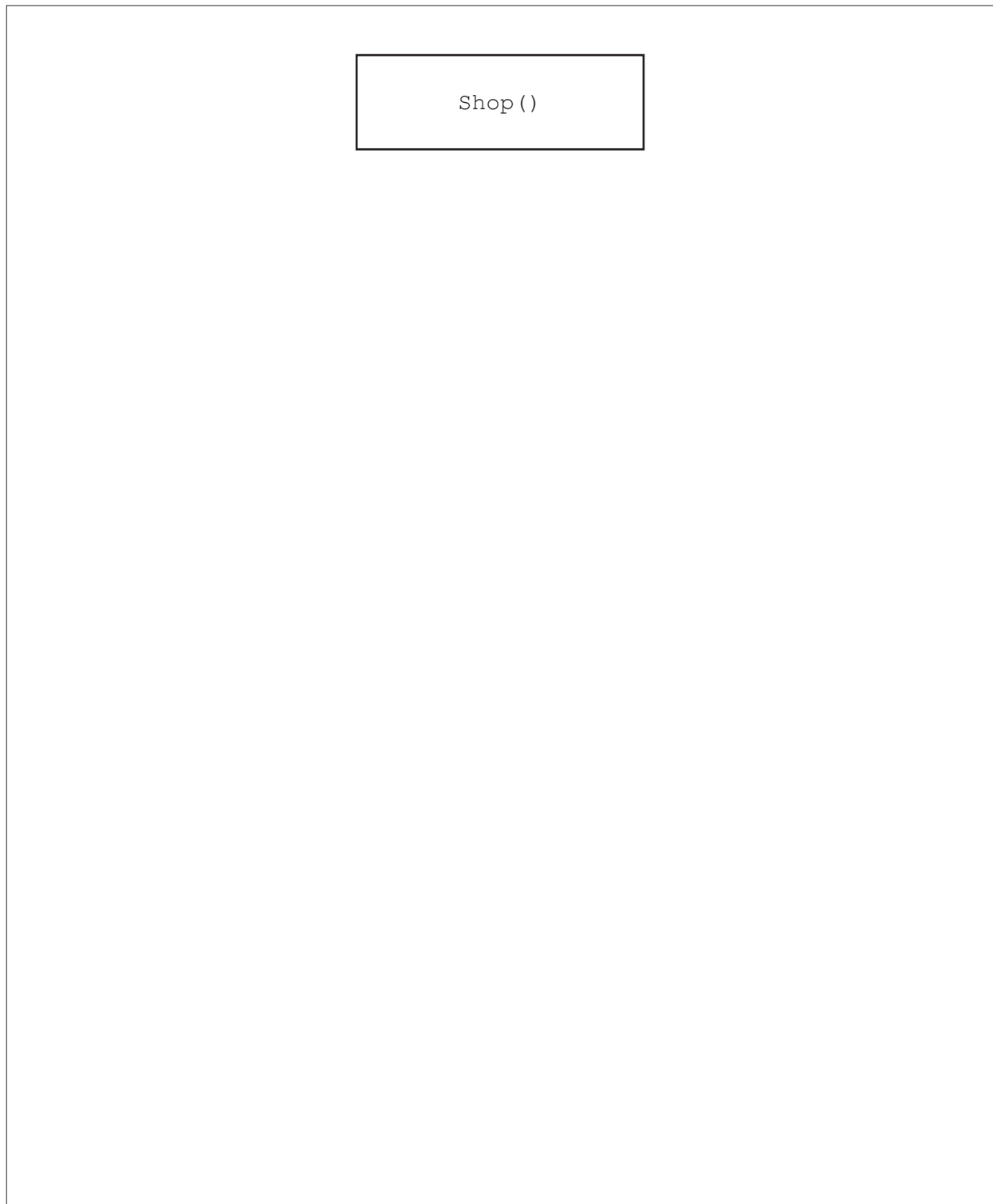
Feature 2

[2]

- (b) Six program modules implement part of an online shopping program. The following table gives the modules and a brief description of each module:

Module	Description
Shop()	Allows the user to choose a delivery slot, select items to be added to the basket and finally check out
ChooseSlot()	Allows the user to select a delivery time. Returns a delivery slot number
FillBasket()	Allows the user to select items and add them to the basket
Checkout()	Completes the order by allowing the user to pay for the items. Returns a Boolean value to indicate whether or not payment was successful
Search()	Allows the user to search for a specific item. Returns an item reference
Add()	Adds an item to the basket. Takes an item reference and a quantity as parameters

(ii) Complete the structure chart for the six modules described in **part (b)**.



[6]

98. M/J 20/P22/Q4/b

- 4 (a) Mustafa has developed the following pseudocode to generate ten random integers in the range 1 to 100.

```
DECLARE Random : ARRAY [1:10] OF INTEGER
DECLARE Count, RNum : INTEGER

FOR Count ← 1 TO 10
    Rnum ← INT(RAND(100)) + 1
    Random[Count] ← RNum
ENDFOR
```

Refer to the **Appendix** on page 19 for a list of built-in pseudocode functions and operators.

Rewrite the pseudocode so that there are no duplicated numbers in the list of random numbers.

[6]

- (b) The changes made to the pseudocode in **part (a)** were as a result of changes to the program requirement.

Give the term used to describe changes made for this reason.

[1]

99. M/J 20/P22/Q6/a(ii,iii),b

6 A company hires out rowing boats on a lake. The company has 17 boats numbered from 1 to 17.

Boats may be hired between 9:00 and 18:00, with a maximum hire duration of 90 minutes.

The company is developing a program to help manage and record the boat hire process.

The programmer has decided to store all values relating to hire time as strings. The program will use a 24-hour clock format. For example:

Time (in words)	String value
Nine o'clock in the morning	"09:00"
Five minutes past ten o'clock in the morning	"10:05"
Ten minutes before three o'clock in the afternoon	"14:50"

The programmer has defined the first module as follows:

Module	Description
AddTime()	- Takes two parameters: - StartTime: a <code>STRING</code> value representing a time as described - Duration: an <code>INTEGER</code> value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a <code>STRING</code>

(ii) AddTime() will be tested using white-box testing.

State the reason for using white-box testing.

[1]

(iii) A run-time error is one type of error that black-box testing can reveal.

Describe **one other** type of error that black-box testing can reveal.

[2]

(b) The user will input the desired start time of a hire. A new module will be written to validate the input string as a valid time in 24-hour clock format.

The string is already confirmed as being in the format "`NN:NN`", where `N` is a numeric character.

Give an example of suitable test data that is in this format but which is **invalid**. Explain your answer.

Test data

Explanation

[2]

100. M/J 20/P23/Q2/a,b

- 2 (a) A structure chart is often produced as part of a modular program design. The chart shows the relationship between modules and the parameters that are passed between them.

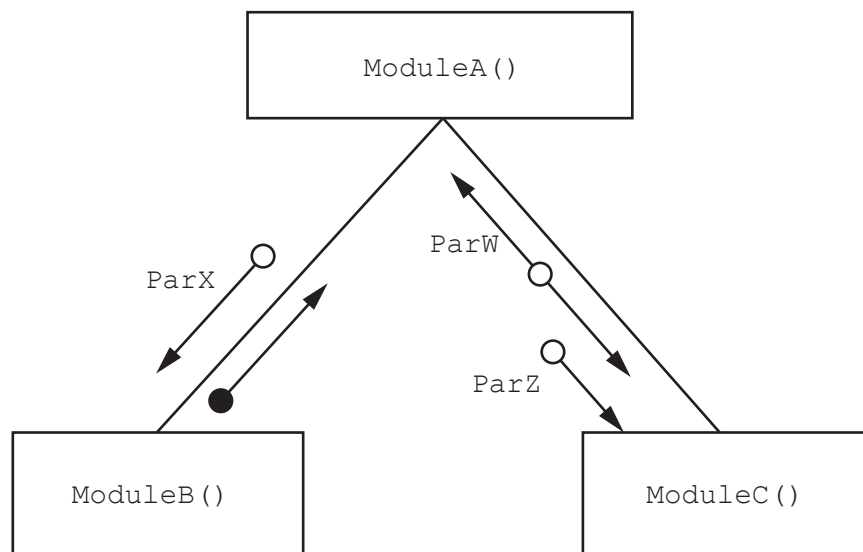
Give **two other** features the structure chart can show.

Feature 1

Feature 2

[2]

- (b) The following structure chart shows the relationship between three modules.



Parameter data types are:

```

ParW : REAL
ParX : INTEGER
ParZ : STRING
  
```

- (i) Write the **pseudocode** header for module `ModuleB()`.

[3]

- (ii) Write the **pseudocode** header for module `ModuleC()`.

[3]

101. M/J 20/P23/Q3/b,c

- (b) The following pseudocode algorithm has been developed to check whether a string contains a valid password.

To be a valid password, a string must:

- be longer than five characters
- contain at least one numeric digit
- contain at least one upper case letter
- contain at least one other character (not a numeric digit or an upper case letter).

```
10 FUNCTION Check(InString : STRING) RETURNS BOOLEAN
11
12   DECLARE Index : INTEGER
13   DECLARE StrLen : INTEGER
14   DECLARE NumUpper, NumDigit : INTEGER
15   DECLARE NextChar : CHAR
16   DECLARE NumOther : INTEGER
17
18   NumUpper ← 0
19   NumDigit ← 0
20
21   StrLen ← LENGTH(InString)
22   IF StrLen < 6
23     THEN
24       RETURN FALSE
25     ELSE
26       FOR Index ← 1 TO StrLen - 1
27         // loop for each character
28         NextChar ← MID(InString, Index, 1)
29         IF NextChar >= '0' AND NextChar <= '9'
30           THEN
31             NumDigit ← NumDigit + 1 // count digits
32           ELSE
33             IF NextChar >= 'A' AND NextChar <= 'Z'
34               THEN
35                 NumUpper ← NumUpper + 1 // count upper case
36             ENDIF
37           ENDIF
38       ENDFOR
39     ENDIF
40
41   NumOther ← StrLen - (NumDigit - NumUpper)
42   IF NumDigit >= 1 AND NumUpper >= 1 AND NumOther >= 1
43     THEN
44       RETURN TRUE
45     ELSE
46       RETURN FALSE
47   ENDIF
48
49 ENDFUNCTION
```

The pseudocode does not work under all circumstances.

The function was dry run with the string "1234AP" and the following trace table was produced. The string is an invalid password, but the pseudocode returned the value TRUE.

Trace table row	StrLen	Index	NextChar	NumUpper	NumDigit	NumOther
1	6			0	0	
2		1	'1'			
3					1	
4		2	'2'			
5					2	
6		3	'3'			
7					3	
8		4	'4'			
9					4	
10		5	'A'			
11				1		
12						3

- (i) The pseudocode algorithm contains two errors.

State how the given trace table indicates the existence of each error.

Error 1

Error 2

[2]

- (ii) Give the line number of each error in the pseudocode algorithm **and** write the modified pseudocode to correct each error.

Line number for error 1

Correct pseudocode

Line number for error 2

Correct pseudocode

[2]

- (c) The term **adaptive maintenance** refers to amendments that are made in response to changes to the program specification. These changes usually affect the program algorithm.

Name **one other** part of the design that can change as a result of adaptive maintenance.

[1]

102. M/J 20/P23/Q5/b(ii),d(i,ii)

- 5 A company hires out rowing boats on a lake. The company has 20 boats, numbered from 1 to 20.

For safety reasons, the boats have to be serviced (checked and any damage repaired) regularly.

The company is developing a program to help manage the servicing of the boats.

Every time a boat is serviced, details are added at the end of the text file, `ServiceLog.txt`, as a single line of information. Each boat is serviced before it is hired out for the first time.

The format of each line is as follows:

`<BoatNumber><Date>`

`BoatNumber` and `Date` are as follows:

- `BoatNumber` is a two-digit numeric string in the range "01" to "20"
- `Date` is an 8-digit numeric string in the format `YYYYMMDD`

The programmer has defined the first module as follows:

Module	Description
<code>GetLastService()</code>	• Called with a string parameter representing the <code>BoatNumber</code> • Searches through the file <code>ServiceLog.txt</code> • Returns the date of the last service in the form "YYYYMMDD"

- (ii) An additional module, `Validate()`, has been written to check that a given string corresponds to a valid `BoatNumber`. A valid `BoatNumber` is a two-digit numeric string in the range "01" to "20".

Give **three** test strings that are **invalid** for different reasons. Explain your choice in each case.

String 1

Reason

String 2

Reason

String 3

Reason

[6]

- (d) (i) A team of programmers will work on the program. Before they begin, the team meet to discuss ways in which the risk of program faults may be reduced during the design and coding stages.

State **two** ways to minimise program faults during the design and coding stages.

1

2

[2]

- (ii) During development, the team test the program using a process known as stub testing.

Explain this process.

[2]

103. O/N 19/P21/Q2/a,d

- 2 (a) A structure chart is often used in modular program design. One feature shown is the sequence of module execution.

State **four** other features that may be shown.

Feature 1

Feature 2

Feature 3

Feature 4

[4]

- (d) Explain the term **adaptive maintenance**.

[2]

104. O/N 19/P21/Q5/a,b

- 5** The following pseudocode checks whether a string is a valid password.

```
FUNCTION CheckPassword(InString : STRING) RETURNS BOOLEAN

    DECLARE Index, Upper, Lower, Digit, Other : INTEGER
    DECLARE NextChar : CHAR

    Upper ← 0
    Lower ← 0
    Digit ← 0
    Other ← 0

    FOR Index ← 1 TO LENGTH(InString)

        NextChar ← MID(InString, Index, 1)
        IF NextChar >= 'A' AND NextChar <= 'Z'
            THEN
                Upper ← Upper + 1
            ELSE
                IF NextChar >= 'a' AND NextChar <= 'z'
                    THEN
                        Lower ← Lower + 1
                    ELSE
                        IF NextChar >= '0' AND NextChar <= '9'
                            THEN
                                Digit ← Digit + 1
                            ELSE
                                Other ← Other + 1
                            ENDIF
                        ENDIF
                    ENDIF
                ENDIF
            ENDIF

    ENDFOR

    IF Upper > 1 AND Lower >= 5 AND (Digit - Other) > 0
        THEN
            RETURN TRUE
        ELSE
            RETURN FALSE
        ENDIF

ENDFUNCTION
```

- (a)** Describe the validation rules that are implemented by this pseudocode. Refer **only** to the contents of the string and **not** to features of the pseudocode.

[3]

(b) (i) Complete the trace table by dry running the function when it is called as follows:

`Result ← CheckPassword("Jim+Smith*99")`

Index	NextChar	Upper	Lower	Digit	Other

[5]

(ii) State the value returned when the function is called using the expression shown. Justify your answer.

Value

Justification

[2]

105. O/N 19/P22/Q2/a

2 (a) Describe the program development cycle with reference to the following:

- source code
- object code
- corrective maintenance.

[3]

106. O/N 19/P22/Q4/b,c

- 4 A program is being written to control the operation of a portable music player. One part of the program controls the output volume.

The player has two buttons, one to increase the volume and one to decrease it. Whenever a button is pressed, a procedure `Button()` is called with a parameter value representing the button as follows:

Button	Parameter value
Volume increase	10
Volume decrease	20

For example, pressing the volume increase button three times followed by pressing the volume decrease button once would result in the calls:

```
CALL Button(10) // VolLevel increased by 1
CALL Button(10) // VolLevel increased by 1
CALL Button(10) // VolLevel increased by 1
CALL Button(20) // VolLevel decreased by 1
```

The program makes use of two global variables of type `INTEGER` as follows:

Variable	Description
<code>VolLevel</code>	The current volume setting. This must be in the range 0 to 49.
<code>MaxVol</code>	A value that can be set to limit the maximum value of <code>VolLevel</code> , in order to protect the user's hearing. A value in the range 1 to 49 indicates the volume limit. A value of zero indicates that no volume limit has been set.

The procedure `Button()` will modify the value of `VolLevel` depending on which button has been pressed and whether a maximum value has been set.

- (b) The procedure `Button()` is to be tested using black-box testing.

Fill in the gaps below to define **three** tests that could be carried out.

TEST 1 – `VolLevel` is changed

Parameter value: 10

`MaxVol`:

`VolLevel` value before call to `Button()`: 48

`VolLevel` expected value after call to `Button()`:

TEST 2 – `VolLevel` is **not** changed

Parameter value: 10

`MaxVol`: 34

VolLevel value before call to Button():

VolLevel expected value after call to Button():

TEST 3 – VolLevel is not changed

Parameter value:

MaxVol: 40

VolLevel value before call to Button(): 0

VolLevel expected value after call to Button():

[6]

(c) The testing stage is part of the program development cycle.

- (i) The program for the music player has been completed. The program does not contain any syntax errors, but testing could reveal further errors.

Identify **and** describe **one different** type of error that testing could reveal.

Type

Description

[2]

- (ii) Stub testing is a technique often used in the development of modular programs.

Describe the technique.

[3]

107. O/N 19/P22/Q5

- 5 The module headers for three modules in a program are defined in pseudocode as follows:

Pseudocode module header
PROCEDURE Lookup(P4 : INTEGER, BYREF M4 : STRING)
FUNCTION Update(T4 : INTEGER) RETURNS INTEGER
FUNCTION Validate(S2 : INTEGER, P3 : STRING) RETURNS BOOLEAN

A fourth module, `Renew()`, calls the three modules in the following sequence.

```
Validate()  
Lookup()  
Update()
```

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.



108. O/N 19/P23/Q1/e

- (e) A trace table is often used during program testing.

Identify the type of testing that includes the use of a trace table.

[1]

109. O/N 19/P23/Q3

- 3 Three program modules process updating of passwords in a file. A description of the relationship between the modules is summarised as follows:

Module name	Description
GetPassword()	• Takes two parameters: AccountID and OldPassword • Returns a string containing the new password
UpdateFile()	• Takes two parameters: AccountID and NewPassword • Returns a Boolean value to indicate whether or not the update was successful
ChangePassword()	• Calls GetPassword() to obtain the new password then calls UpdateFile() to write the new password to the file

Draw a structure chart to show the relationship between the three modules and the parameters passed between them.

[5]

110. O/N 19/P23/Q4/a,b,c

- 4** The following pseudocode algorithm checks whether a string is a valid email address.

```
FUNCTION Check(InString : STRING) RETURNS BOOLEAN

    DECLARE Index : INTEGER
    DECLARE NumDots : INTEGER
    DECLARE NumAsts : INTEGER
    DECLARE NextChar : CHAR
    DECLARE NumOthers : INTEGER

    NumDots ← 0
    NumAsts ← 0
    NumOthers ← 0

    FOR Index ← 1 TO LENGTH(InString)

        NextChar ← MID(InString, Index, 1)
        CASE OF NextChar
            '.': NumDots ← NumDots + 1
            '@': NumAsts ← NumAsts + 1
            OTHERWISE NumOthers ← NumOthers + 1
        ENDCASE

    ENDFOR

    IF (NumDots >= 1 AND NumAsts = 1 AND NumOthers > 5)
        THEN
            RETURN TRUE
        ELSE
            RETURN FALSE
    ENDIF

ENDFUNCTION
```

- (a)** Describe the validation rules that are implemented by this pseudocode. Refer **only** to the contents of the string and **not** to features of the pseudocode.

[3]

Justification

Value

Justification

[4]

111. M/J 19/P21/Q1/c

- (c) White-box and black-box are two types of testing. In white-box testing, data are chosen to test every possible path through the program.

Explain how data are chosen in black-box testing.

[2]

112. M/J 19/P21/Q4/a,c

- 4 The following pseudocode is a string handling function.

For the built-in functions list, refer to the **Appendix** on page 16.

```

FUNCTION Clean(InString : STRING) RETURNS STRING

    DECLARE NewString : STRING
    DECLARE Index : INTEGER
    DECLARE AfterSpace : BOOLEAN
    DECLARE NextChar : CHAR
    CONSTANT Space = ' '

    AfterSpace ← FALSE
    NewString ← ""

    FOR Index ← 1 TO LENGTH(InString)
        NextChar ← MID(InString, Index, 1)
        IF AfterSpace = TRUE
            THEN
                IF NextChar <> Space
                    THEN
                        NewString ← NewString & NextChar
                        AfterSpace ← FALSE
                    ENDIF
            ELSE
                NewString ← NewString & NextChar
                IF NextChar = Space
                    THEN
                        AfterSpace ← TRUE
                    ENDIF
            ENDIF
        ENDIF
    ENDFOR

    RETURN NewString

ENDFUNCTION

```



```
Result ← Clean("VVXVVVYVandVVZ")
```

[2]

- (c) State the term used for changes that are made to a program in response to a specification change.

[1]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

RAND(x : INTEGER) RETURNS REAL
returns a real number in the range 0 to x (x not inclusive).

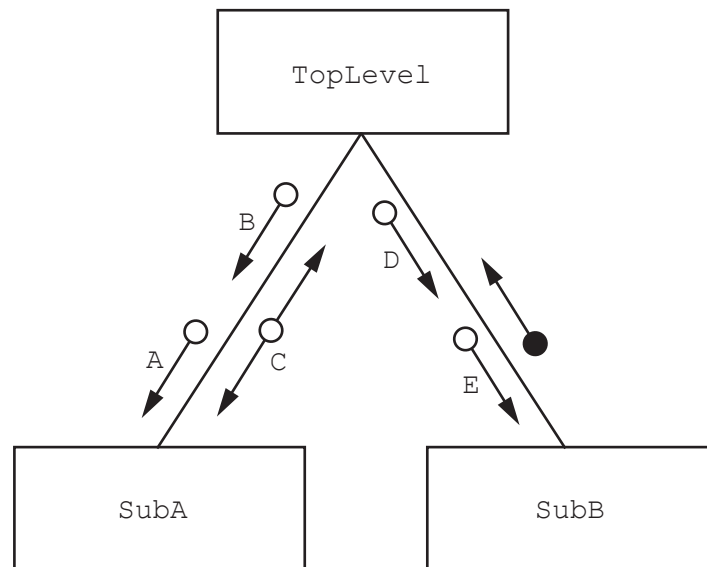
Example: RAND(87) could return 35.43

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

113. M/J 19/P22/Q3/a,b

- 3 The following structure chart shows the relationship between three modules.



Parameters A to E have the following data types:

A, D : STRING
 C : CHAR
 B, E : INTEGER

- (a) (i) Write the **pseudocode** header for module `SubA()`.

[3]

- (ii) Write the **pseudocode** header for module `SubB()`.

[3]

- (b) Module hierarchy and parameters are two features that may be represented on a structure chart.

State **two other** features than can be represented.

Feature 1

Feature 2

[2]

114. M/J 19/P22/Q4/a,b

- 4 The following is pseudocode for a string handling function.

For the built-in functions list, refer to the **Appendix** on page 16.

```
FUNCTION Search(InString : STRING) RETURNS INTEGER
```

```
    DECLARE NewString : STRING
    DECLARE Index : INTEGER
    DECLARE NextChar : CHAR
    DECLARE Selected : INTEGER
    DECLARE NewValue : INTEGER
```

```
    NewString ← '0'
    Selected ← 0
```

```
    FOR Index ← 1 TO LENGTH(InString)
```

```
        NextChar ← MID(InString, Index, 1)
        IF NextChar < '0' OR NextChar > '9'
            THEN
                NewValue ← STRING_TO_NUM(NewString)
                IF NewValue > Selected
                    THEN
                        Selected ← NewValue
                    ENDIF
                NewString ← '0'
            ELSE
                NewString ← NewString & NextChar
            ENDIF
    ENDIF
```

```
    ENDFOR
```

```
    RETURN Selected
```

```
ENDFUNCTION
```

(a) (i) The following assignment calls the `Search()` function:

```
Result ← Search("12∇34∇5∇∇39")
```

Complete the following trace table by performing a dry run of this function call.

The symbol '∇' represents a space character. Use this symbol to represent a space character in the trace table.

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Note: This function will also work if `x` is of type INTEGER

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.

Example: `STRING_TO_NUM("23.45")` returns 23.45

Note: This function will also work if `x` is of type CHAR

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

115. M/J 19/P23/Q3/a

- 3** The following pseudocode represents three separate modules from an algorithm design. The module contents are not shown.

```
FUNCTION Search(AA : INTEGER, BB : STRING) RETURNS INTEGER
```

```
    {  
ENDFUNCTION
```

```
FUNCTION Allocate() RETURNS BOOLEAN
```

```
    {  
ENDFUNCTION
```

```
PROCEDURE Enable(CC : INTEGER, BYREF DD : INTEGER)
```

```
    {  
ENDPROCEDURE
```

A fourth module, `Setup()`, refers to the previous three modules as follows:

```
PROCEDURE Setup()
```

```
    {  
        WHILE Authorised = TRUE  
            {  
                ThisValue ← Search(27, "Thursday")  
                {  
                    Authorised ← Allocate()  
                    {  
                        CALL Enable(ThisValue, 4)  
                    }  
                }  
            }  
        ENDWHILE  
    }  
ENDPROCEDURE
```

- (a)** Draw a structure chart to show the four modules and the parameters that these pass between them.

116. M/J 19/P23/Q5/a,b

- 5 Nigel is learning about string handling. He wants to write code to count the number of words in a given string. A word is defined as a sequence of alphabetic characters that is separated by one or more space characters.

His first attempt at writing an algorithm in pseudocode is as follows:

```
PROCEDURE CountWords (Message : STRING)

    DECLARE NumWords : INTEGER
    DECLARE Index : INTEGER
    CONSTANT Space = ' '

    NumWords ← 0

    FOR Index ← 1 TO LENGTH (Message)
        IF MID (Message, Index, 1) = Space
            THEN
                NumWords ← NumWords + 1
            ENDIF
        ENDFOR

    OUTPUT "Number of words : " , NumWords

ENDPROCEDURE
```

For the built-in functions list, refer to the **Appendix** on page 18.

His first attempt is incorrect. He will use white-box testing to help him to identify the problem.

- (a) (i) State the purpose of white-box testing.

[1]

- (ii) Dry running the code is often used in white-box testing. In this method, the programmer records the values of variables as they change.

Identify what the programmer would normally use to record the changes.

[1]

- (b) (i) Write a test string containing two words that gives the output:

```
Number of words : 2
```

Use the symbol '▽' to represent each space character in your test string.

Explain why the algorithm gives the output shown above.

String

Explanation

[3]

- (ii) Nigel tested the procedure with the strings:

String 1: "Red∇and∇Yellow"

String 2: "Green∇∇and∇∇Pink∇"

Give the output that is produced for each of the strings.

Describe the changes that would need to be made to the algorithm to give the correct output in each case.

Do **not** write pseudocode **or** program code.

String 1

Description

String 2

Description

[6]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.

Example: `NUM_TO_STRING(x)` returns "87.5" if `x` has the value 87.5

Note: This function will also work if `x` is of type `INTEGER`

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.

Example: `STRING_TO_NUM(x)` returns 23.45 if `x` has the value "23.45"

Note: This function will also work if `x` is of type `CHAR`

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
<code>AND</code>	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

117. O/N 18/P22/Q2/b

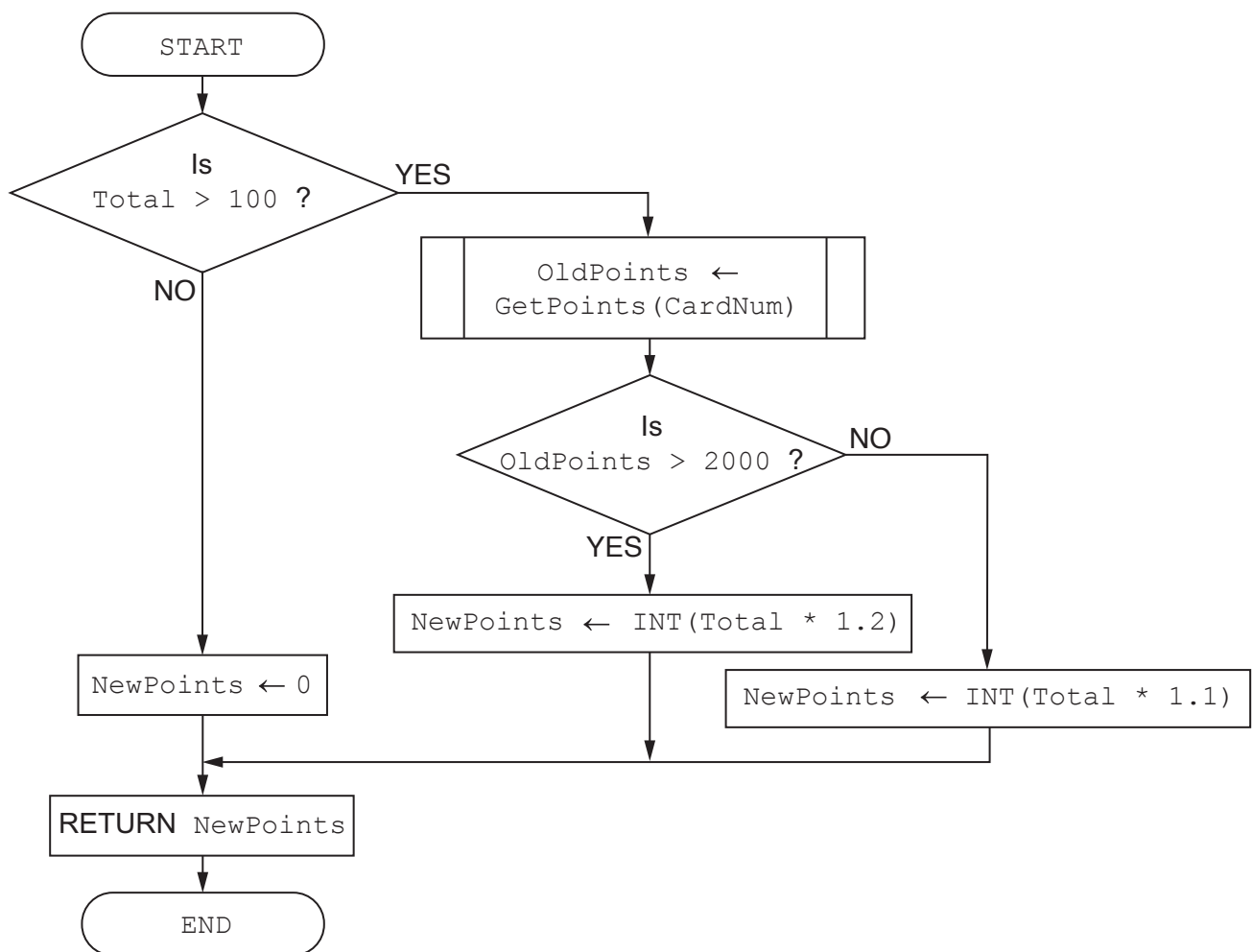
- 2 Shop customers have a discount card with a unique card number. Customers collect points each time they buy items. The number of points they collect depends on:

- the total amount they spend
- the number of points already collected.

The function `CalcPoints()` takes the card number and the total amount spent as parameters. It returns the number of new points collected. A flowchart for the function is shown.

The function uses the following variables and functions.

Identifier	Data type	Description
<code>CardNum</code>	<code>STRING</code>	A numeric string representing the unique card number
<code>OldPoints</code>	<code>INTEGER</code>	The number of points already collected
<code>NewPoints</code>	<code>INTEGER</code>	The number of new points collected
<code>Total</code>	<code>REAL</code>	The amount spent
<code>GetPoints()</code>	<code>FUNCTION</code>	Takes the card number as a parameter and returns the number of points already collected
<code>INT()</code>	<code>FUNCTION</code>	Refer to the Appendix on page 16



(b) The function `CalcPoints()` is written in a high-level language. It has been checked and it does not contain any syntax or logic errors.

- (i) Name **and** describe **one** other type of error that the high-level language code could contain.

Name

Description

[2]

- (ii) The function `CalcPoints()` is tested using white-box testing.

State **two** different values of `Total` that could be used to test different paths through the algorithm. Justify your choices.

Value

Justification

Value

Justification

[4]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns string "FGH"

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.

Example: If x has the value 87.5 then NUM_TO_STRING(x) will return "87.5"

Note: This function will also work if x is of type integer.

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

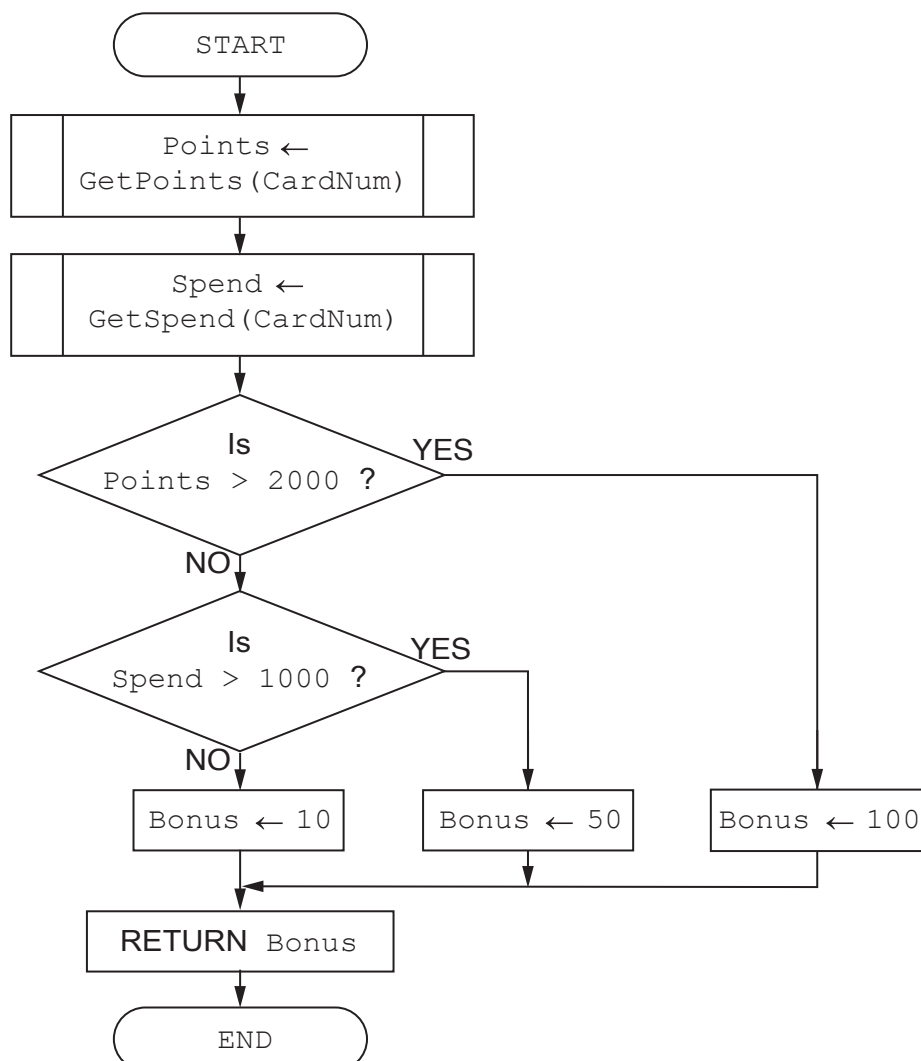
118. O/N 18/P23/Q2/b,c

- 2 Shop customers have a discount card with a unique card number. Customers collect points when they buy items. At the end of each year, customers are given bonus (extra) points related to the total amount they have spent during the year, and the number of points they have on their card.

The function `CalcBonus()` takes the card number as a parameter. It returns the bonus points for the customer. A flowchart for the function is shown.

The function uses the following variables and functions.

Identifier	Data type	Description
CardNum	STRING	A numeric string representing the unique card number
Points	INTEGER	The number of points collected
Spend	REAL	The total amount that customer has spent during the year
Bonus	INTEGER	The number of bonus points
GetPoints()	FUNCTION	Takes the card number as a parameter and returns the number of points already collected
GetSpend()	FUNCTION	Takes the card number as a parameter and returns the total amount that customer has spent during the year



(b) The function `CalcBonus()` is written in a high-level language.

(i) The function is tested using black-box testing and does not contain any syntax errors.

Name **and** describe **one** other type of error that black-box testing could find.

Name

Description

[2]

(ii) The function `CalcBonus()` is tested using white-box testing.

State **two** different pairs of values for `Spend` and `Points` that can be used to test different paths through the function. Justify your choices.

`Spend`

`Points`

Justification

`Spend`

`Points`

Justification

[4]

(c) Name **two** types of program maintenance **and** state the reason why each is needed.

Name

Reason

Name

Reason

[4]

119. M/J 18/P21/Q4/a,b

- 4 (a) A structure chart is used in modular program design.

Iteration and selection are two features of an algorithm that may be shown on a structure chart.

Give **three** other features.

Feature 1

Feature 2

Feature 3

[3]

- (b) Pseudocode for a function is shown.

```
FUNCTION ItemProcess (AddItem, InString : STRING) RETURNS BOOLEAN
  DECLARE RetFlag : BOOLEAN
  RetFlag ← FALSE

  IF AddItem = "Yes"
    THEN
      RetFlag ← AddToList(InString)
    ELSE
      CALL RemoveFromList(InString)
    ENDIF

  RETURN RetFlag

ENDFUNCTION
```

Draw a structure chart on the next page to represent this pseudocode.

[6]

120. M/J 18/P22/Q2/c(i)

2 The following is a function design in pseudocode.

Line numbers are given for reference only.

```
01 FUNCTION CountDigits(InString : STRING) RETURNS CHAR
02
03 DECLARE nc : CHAR
04 DECLARE c : INTEGER
05 DECLARE n : INTEGER
06
07
08 c ← 0
09 n ← LENGTH(InString) // get number of characters for loop
10
11 WHILE n > 0 // repeat until no more characters left
12
13 nc ← LEFT(InString,1)
14 n ← n - 1
15 InString ← RIGHT(InString,n) // remove first character
16
17 IF (nc < '0') OR (nc > '9')
18 THEN
19 // do nothing
20 ELSE
21 c ← c + 1
22 ENDIF
23
24 ENDWHILE
25
26 RETURN c
27
28 ENDFUNCTION
```

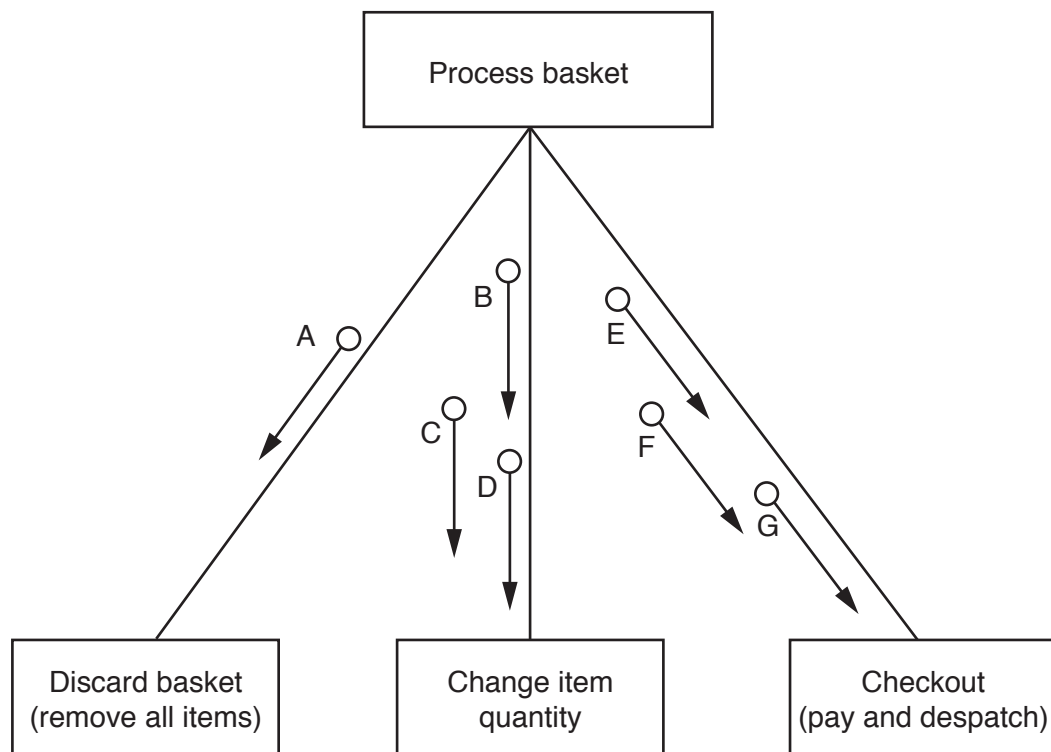
- (c) (i)** There is a mistake in the pseudocode that would produce a data type mismatch error if a programmer were to write similar program code.

Describe this mistake and how it may be corrected.

[2]

121. M/J 18/P22/Q4/a

- 4 The structure chart shows part of the design of a program for an online shopping system.



- (a) (i) Draw on the chart to show the following facts.

- Each of the modules at the lower level returns a Boolean parameter, X.
- Process basket will call only one of the modules shown at the lower level.

[2]

- (ii) The parameters A to G shown on the chart will be used to pass the following information.

PaymentDetails
Quantity
BasketID
DeliveryAddress
ItemID

Complete the following table to show the parameter and the information it represents.

Parameter	Information
A	
B	
C	
D	
E	
F	
G	

[3]

122. M/J 18/P23/Q2/c(i)

2 The following is a function design in pseudocode.

Line numbers are given for reference only.

```
01  FUNCTION GradeCalc(Mark : ARRAY[] OF INTEGER) RETURNS INTEGER
02
03  DECLARE DGradeCount : INTEGER
04  DECLARE n : INTEGER
05  DECLARE Grade : STRING
06  DECLARE ThisMark : INTEGER
07
08  DGradeCount ← 0      // initialise variables
09  n ← 1
10  ThisMark ← 1
11  REPEAT
12      ThisMark ← Mark[n]      // get next mark from Mark array
13
14      IF ThisMark > 74
15          THEN
16              Grade ← "Distinction"
17              DGradeCount ← DGradeCount + 1 // one more Distinction
18          ELSE
19              IF ThisMark > 59
20                  THEN
21                      Grade ← "Merit"
22                  ELSE
23                      IF ThisMark > 39
24                          THEN
25                              Grade ← "Pass"
26                          ELSE
27                              Grade ← "Fail"
28                      ENDIF
29                  ENDIF
30          ENDIF
31
32      OUTPUT "Grade for Student " & n & " is " & Grade
33
34      n ← n + 1
35  UNTIL n = 101 // no more elements in the array
36
37  RETURN Grade // return the number of Distinction grades
38
39  ENDFUNCTION
40
41  DECLARE Mark : ARRAY[1:100] OF INTEGER
42  DECLARE DistinctionGrades : INTEGER
43
44  CALL GetMarks(Mark[]) // fill the array with student marks
45  DistinctionGrades ← GradeCalc(Mark[])
```

(c) (i) There is a mistake in the pseudocode that would produce a data type mismatch error if a programmer were to write similar program code.

Describe this mistake and how it may be corrected.

[2]

123. M/J 18/P23/Q3/b

3 A modular program design consists of four modules:

Module1 has three sub-tasks. Each sub-task is implemented by a single subroutine (either a function or a procedure).

The subroutine headings are defined as follows:

```
FUNCTION  Module2 (Weight : REAL) RETURNS BOOLEAN  
PROCEDURE Module3 (Weight : REAL, Customer : STRING, Purchased : DATE)  
FUNCTION  Module4 (Purchased : DATE, Account : INTEGER) RETURNS INTEGER
```

(b) Draw a structure chart to represent the program design.

Use the letters in the table to label the values passed between modules.

Value	Label
Boolean return value	A
Integer return value	B
Account	C
Customer	D
Purchased	E
Weight	F

Structure chart

[4]

124. O/N 17/P21/Q2/a,b

2 The following pseudocode represents a simple algorithm.

```

DECLARE NumberFound, Remainder, Number : INTEGER
DECLARE StartNumber, EndNumber, Divisor : INTEGER

INPUT StartNumber
INPUT EndNumber
INPUT Divisor
NumberFound ← 0

FOR Number ← StartNumber TO EndNumber
    Remainder ← MODULUS(Number, Divisor)
    IF Remainder = 0
        THEN
            OUTPUT Number
            NumberFound ← NumberFound + 1
        ENDIF
    ENDFOR
OUTPUT "Count: " & NumberFound

```

For the built-in functions list, refer to the **Appendix** on page 14.

(a) Complete the following trace table.

StartNumber	EndNumber	Divisor	NumberFound	Number	Remainder	Output
11	13	2	0			

[3]

(b) Describe the purpose of this algorithm.

[3]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

MODULUS (x : INTEGER, y : INTEGER) RETURNS INTEGER

returns the remainder when x is divided by y using integer arithmetic.

Example: MODULUS (5, 2) returns 1


```
INT(x : REAL) RETURNS INTEGER
```

returns the integer part of x .

Example: `INT(27.5415)` returns 27

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```

returns the integer value representing the length of string `ThisString`.

Example: `LENGTH("Happy Days")` returns 10

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns leftmost x characters from `ThisString`.

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

```
RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
```

returns rightmost x characters from `ThisString`.

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

125. O/N 17/P21/Q5/b

- 5 A sports club maintains a record of the email address of each of its members. The details are stored in a text file, `EmailDetails.txt`. The format of each line of the text file is as follows:

`<MembershipNumber><EmailAddress>`

- `MembershipNumber` is a four-character string of numerals.
- `EmailAddress` is a variable-length string.

Membership of the club has increased and a four-character membership number is no longer adequate.

A procedure, `MakeNewFile`, is required to perform the following actions:

1. Create a new file, `NewEmailDetails.txt`
2. Read a line from file `EmailDetails.txt`
3. Extend `MembershipNumber` by adding two leading zero digits (for example, "1234" becomes "001234")
4. Write the new line to file `NewEmailDetails.txt`
5. Repeat steps 2 to 4 for all lines in the original file.

- (b) A function, `IsValidEmail`, is to be written to test for a valid email address format.

An email address has a valid format if it obeys the following three rules:

1. It contains a single '@' symbol.
2. The '@' symbol must be preceded by at least one character.
3. The '@' symbol must be followed by at least three characters.

Choose **three** different invalid strings to test distinct aspects of the rules.

Explain your choice in each case.

1

Explanation

2

Explanation

3

Explanation

[6]

126. O/N 17/P22/Q3

- 3 A 1D array, `Product`, of type `STRING` is used to store information about a range of products in a shop. There are 100 elements in the array. Each element stores one data item.

The format of each data item is as follows:

`<ProductID><ProductName>`

- `ProductID` is a four-character string of numerals
- `ProductName` is a variable-length string

The following pseudocode is an initial attempt at defining a procedure, `ArraySort`, which will perform a bubble sort on `Product`. The array is to be sorted in ascending order of `ProductID`. Line numbers have been added for identification purposes only.

```
01  PROCEDURE SortArray
02      DECLARE Temp : CHAR
03      DECLARE FirstID, SecondID : INTEGER
04      FOR I ← 1 TO 100
05          FOR J ← 2 TO 99
06              FirstID ← MODULUS(LEFT(Product[J], 6))
07              SecondID ← MODULUS(LEFT(Product[J + 1], 6))
08              IF FirstID > SecondID
09                  THEN
10                      Temp ← Product[I]
11                      Product[I] ← Product[J + 1]
12                      Product[J + 1] ← Temp
13              ENDFOR
14          ENDIF
15      ENDFOR
16  ENDPROCEDURE
```

The pseudocode on page 8 contains a number of errors. Complete the following table to show:

- the line number of the error
- the error itself
- the correction that is required.

Note:

- If the same error occurs on more than one line, you should only refer to it ONCE.
- Lack of optimisation should not be regarded as an error.

Line number	Error	Correction
01	Wrong procedure name – “SortArray”	PROCEDURE ArraySort

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

MODULUS (x : INTEGER, y : INTEGER) RETURNS INTEGER

returns the remainder when x is divided by y using integer arithmetic.

Example: MODULUS (5, 2) will return 1

INT (x : REAL) RETURNS INTEGER

returns the integer part of x.

Example: INT (27.5415) returns 27

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`.

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns leftmost `x` characters from `ThisString`.

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns rightmost `x` characters from `ThisString`.

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

`TONUM(ThisString : STRING) RETURNS INTEGER`

returns a numeric value equivalent to `ThisString`.

Example: `TONUM("1201")` returns integer value 1201

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values. Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values. Example: TRUE OR FALSE produces TRUE

127. O/N 17/P22/Q4/c

- 4 Programming languages provide built-in functions to generate random numbers. To be truly random, the frequency of each number generated should be the same.

You are required to write program code to test the random number generator of your chosen language.

The test should:

- generate a given number of random numbers between 1 and 10 inclusive
- keep a count of the number of times each number is generated
- calculate the expected frequency of each number 1 to 10
- output the actual frequency of each number 1 to 10
- output the difference between the actual frequency and the expected frequency.

The program code should be written as a procedure. In pseudocode, the procedure heading will be:

```
PROCEDURE TestRandom(Repetitions AS INTEGER)
```

The parameter, `Repetitions`, contains a value representing the total number of random numbers that should be generated.

The following example shows the expected output for the procedure call, `TestRandom(200)`.

The expected frequency is 20.

Number	Frequency	Difference
1	17	-3
2	21	1
3	12	-8
4	28	8
5	20	0
6	19	-1
7	21	1
8	16	-4
9	24	4
10	22	2

- (c) The procedure is developed and run using the call `TestRandom(200)`. No system errors are produced.

To ensure that the procedure works correctly, you need to check the output.

Describe **two** checks you should make to suggest the program works correctly.

1

2

128. O/N 17/P23/Q2/a,b

2 The following pseudocode represents a simple algorithm.

```

DECLARE NumberFound, Remainder, Number : INTEGER
DECLARE StartNumber, EndNumber, Divisor : INTEGER

INPUT StartNumber
INPUT EndNumber
INPUT Divisor
NumberFound ← 0

FOR Number ← StartNumber TO EndNumber
    Remainder ← MODULUS(Number, Divisor)
    IF Remainder = 0
        THEN
            OUTPUT Number
            NumberFound ← NumberFound + 1
        ENDIF
    ENDFOR
OUTPUT "Count: " & NumberFound

```

For the built-in functions list, refer to the **Appendix** on page 14.

(a) Complete the following trace table.

StartNumber	EndNumber	Divisor	NumberFound	Number	Remainder	Output
11	13	2	0			

[3]

(b) Describe the purpose of this algorithm.

[3]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

MODULUS (x : INTEGER, y : INTEGER) RETURNS INTEGER

returns the remainder when x is divided by y using integer arithmetic.

Example: MODULUS (5, 2) returns 1

```
INT(x : REAL) RETURNS INTEGER
```

returns the integer part of x .

Example: `INT(27.5415)` returns 27

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```

returns the integer value representing the length of string `ThisString`.

Example: `LENGTH("Happy Days")` returns 10

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns leftmost x characters from `ThisString`.

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

```
RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
```

returns rightmost x characters from `ThisString`.

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

129. O/N 17/P23/Q5/b

- 5 A sports club maintains a record of the email address of each of its members. The details are stored in a text file, `EmailDetails.txt`. The format of each line of the text file is as follows:

`<MembershipNumber><EmailAddress>`

- `MembershipNumber` is a four-character string of numerals.
- `EmailAddress` is a variable-length string.

Membership of the club has increased and a four-character membership number is no longer adequate.

A procedure, `MakeNewFile`, is required to perform the following actions:

1. Create a new file, `NewEmailDetails.txt`
2. Read a line from file `EmailDetails.txt`
3. Extend `MembershipNumber` by adding two leading zero digits (for example, "1234" becomes "001234")
4. Write the new line to file `NewEmailDetails.txt`
5. Repeat steps 2 to 4 for all lines in the original file.

- (b) A function, `IsValidEmail`, is to be written to test for a valid email address format.

An email address has a valid format if it obeys the following three rules:

1. It contains a single '@' symbol.
2. The '@' symbol must be preceded by at least one character.
3. The '@' symbol must be followed by at least three characters.

Choose **three** different invalid strings to test distinct aspects of the rules.

Explain your choice in each case.

1

Explanation

2

Explanation

3

Explanation

130. M/J 17/P21/Q4/a,b

- 4 (a) A structure chart is a tool used in modular program design.

State **three** pieces of information that a structure chart can convey about a program design.

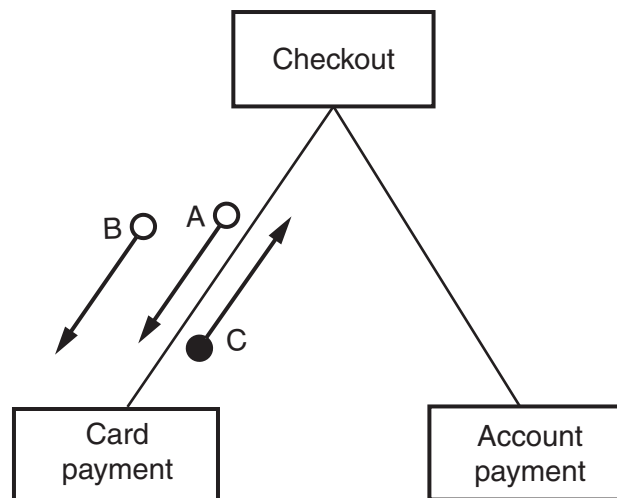
1

2

3

[3]

- (b) The following diagram shows part of a structure chart.



Examples of the data items that correspond to the arrows are given in this table:

Arrow	Data item
A	234.56
B	"Mr Robert Zimmerman"
C	True

Use **pseudocode** to write the function header for the **Card payment** module.

[3]

131. M/J 17/P21/Q6/b

- 6 A multi-user computer system makes use of passwords.

To be valid, a password must comply with the following rules:

- at least two lower-case alphabetic characters
- at least two upper-case alphabetic characters
- at least three numeric characters
- alpha-numeric characters only

A function, `ValidatePassword`, is needed to check that a given password follows these rules. This function takes a string, `Pass`, as a parameter and returns a Boolean value:

- `TRUE` if `Pass` contains a valid password
- `FALSE` otherwise.

(b) (i) The function will be tested.

Give a valid string to check that the function returns `TRUE` under the correct conditions.

String1:

Modify the valid string given for String1 to test each rule separately.

Explain your choice in each case.

String2:

Explanation:

String3:

Explanation:

String4:

Explanation:

String5:

Explanation:

[5]

(ii) When testing a module, it is necessary to test all possible paths through the code.

State the name given to this type of testing.

[1]

(iii) A program consisting of several modules may be tested using a process known as stub testing.

Explain this process.

[2]

132. M/J 17/P22/Q6/b

- 6** A computerised vehicle licensing system stores details about vehicles and their registration marks (number plates or license plates).

To be valid, a vehicle registration must comply with the following rules:

- It must be between six and nine characters long.
- Characters 1 to 3 are upper case alphabetic characters.
- Characters 4 to 5 are numeric characters.
- Remaining characters are upper case alphabetic.

A function, `ValidateRegistration` is needed to check that a given registration mark follows these rules. This function takes a string, `Registration` as a parameter and returns a Boolean value:

- `TRUE` if it is a valid registration
- `FALSE` otherwise.

(b) The function is to be tested.

Give a valid string that could be used to check that the function returns `TRUE` under the correct conditions.

String1:

Modify your valid String1 to test each rule separately.

Explain your choice in each case.

String2:

Explanation:

String3:

Explanation:

String4:

Explanation:

String5:

Explanation:

133. M/J 17/P23/Q4/a,b

- 4 (a) A structure chart is a tool used in modular program design.

State **three** pieces of information that a structure chart can convey about a program design.

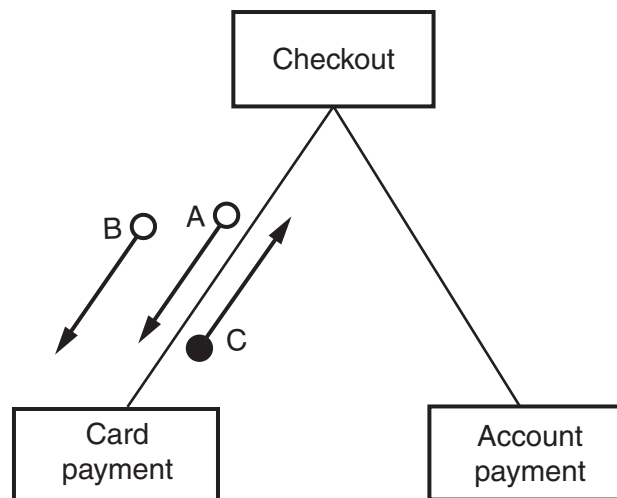
1

2

3

[3]

- (b) The following diagram shows part of a structure chart.



Examples of the data items that correspond to the arrows are given in this table:

Arrow	Data item
A	234.56
B	"Mr Robert Zimmerman"
C	True

Use **pseudocode** to write the function header for the **Card payment** module.

[3]

134. M/J 17/P23/Q6/b

- 6 A multi-user computer system makes use of passwords.

To be valid, a password must comply with the following rules:

- at least two lower-case alphabetic characters
- at least two upper-case alphabetic characters
- at least three numeric characters
- alpha-numeric characters only

A function, `ValidatePassword`, is needed to check that a given password follows these rules. This function takes a string, `Pass`, as a parameter and returns a Boolean value:

- `TRUE` if `Pass` contains a valid password
- `FALSE` otherwise.

(b) (i) The function will be tested.

Give a valid string to check that the function returns `TRUE` under the correct conditions.

String1:

Modify the valid string given for String1 to test each rule separately.

Explain your choice in each case.

String2:

Explanation:

String3:

Explanation:

String4:

Explanation:

String5:

Explanation:

[5]

(ii) When testing a module, it is necessary to test all possible paths through the code.

State the name given to this type of testing.

[1]

(iii) A program consisting of several modules may be tested using a process known as stub testing.

Explain this process.

[2]

135. O/N 16/P21/Q2/a

2 A sensing device sends bit values to a computer along data channels.

- Channel 1 transmits a sequence of binary values from a sensor
- Channel 2 transmits at regular intervals to indicate whether the sensor is switched on or off:
 - 0 indicates switched off
 - 1 indicates switched on

A program tests the bits received from the sensing device.

A program reads the signal from Channel 2 after every six values from Channel 1.

A built-in function `READ(<ChannelNumber>)` reads a value from the specified channel.

Pseudocode for the program is as follows:

```

01      BitCount ← 0
02      Status2 ← READ(2)
03      WHILE Status2 = 1
04
05          FOR ReadingCount ← 1 TO 6
06              ThisBit ← READ(1)
07              IF ThisBit = 1
08                  THEN
09                      BitCount ← BitCount + 1
10              ENDIF
11              IF BitCount = 5
12                  THEN
13                      OUTPUT "Error - Investigate"
14                      BitCount ← 0
15              ENDIF
16          ENDFOR
17
18          Status2 ← READ(2)
19      ENDWHILE
  
```

(a) Trace the execution of the program for the following sequence of bits.

Channel 1		1	0	1	1	1	0		1	1	0	0	1	1	
Channel 2	1							1							0

Status2	ReadingCount	ThisBit	BitCount	OUTPUT
			0	
1	1	1	1	
	2			

[7]

136. O/N 16/P22/Q4/d

- 4** A social club runs a weekly prize draw. A member can buy a ticket for \$2. Each week, the club sells up to 150 tickets, with consecutive numbers starting from 1. There is a main cash prize of \$120 and a further three prizes of \$10. The computer program will generate the winning numbers.

Throughout this question, you will need to refer to the list of pseudocode functions in the **Appendix**.

- (b)** Write **program code** to generate and output four winning numbers.
Ignore the issue that duplicate numbers may be generated.

Visual Basic and Pascal: You should include declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

[4]

- (d)** The organisers do not want the same number to be drawn more than once in a week. To deal with the issue of duplicate numbers being generated, the program in **part (b)** will require adaptive maintenance.

- (i)** Describe what is meant by adaptive maintenance.

[2]

- (ii)** The club does not want the program to output the same number more than once in a week.

For each winning number, outline the extra steps that the program must have. Include any data structure that the program needs.
Do not write pseudocode or program code.

[3]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR

returns the single character at position `Position` (counting from the start of the string with value 1) from the string `ThisString`.

For example: `ONECHAR("New York", 5)` returns 'Y'

CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER

returns the number of characters in `ThisString`.

For example: `CHARACTERCOUNT("New York")` returns 8

SUBSTR(ThisString : STRING, Value1 : INTEGER, Value2 : INTEGER) RETURNS STRING

returns a sub-string from within `ThisString`.

`Value1` is the start index position (counting from the left, starting with 1).

`Value2` is the final index position.

For example: `SUBSTR("art nouveau", 5, 11)` returns "nouveau"

TONUM(ThisString : STRING) RETURNS INTEGER or REAL

returns the integer or real equivalent of the string `ThisString`.

For example: `TONUM("502")` returns the integer 502

`TONUM("56.36")` returns the real number 56.36

ASC(ThisCharacter : CHAR) RETURNS INTEGER

returns an integer which is the ASCII character code for the character `ThisCharacter`.

For example: `ASC('A')` returns integer 65

CHR(Value : INTEGER) RETURNS CHAR

returns the character that ASCII code number `Value` represents.

For example: `CHR(65)` returns 'A'

`RND()` RETURNS REAL

returns a random number in the range 0 to 0.99999

For example: `RND()` returns 0.67351

`INT(ThisNumber : REAL)` RETURNS INTEGER

returns the integer part of `ThisNumber`.

For example: `INT(12.79)` returns 12

Errors

For any function, if the program calls the function incorrectly, the function returns an error.

Concatenation operator

`&` operator – Concatenates two expressions of `STRING` or `CHAR` data type.

For example: `"South" & " " & "Pole"` produces `"South Pole"`
`'B' & "000654"` produces `"B000654"`

137. O/N 16/P22/Q5/c(iii)

- 5 A team keeps a record of the scores made by each of their eight players in a number of games.

The data in the two tables below shows:

- the scores of the eight players after twenty games
- the eight player names.

	1	2	3		8
1	12	17	67		31
2	35	82	44		29
3	61	39	80		17
4	81	103	21		11
5	56	0	98		4
...					
19	45	6	81		77
20	12	11	3		6

1	Vorma
2	Ravi
3	Chada
4	Nigam
5	Bahri
6	Smith
7	Goyal
8	Lata

The team wants a computer program to input and record the player data.

- (a) A programmer designs the following pseudocode for the input of a player's score from one game.

```
01 INPUT GameNumber
02 INPUT PlayerNumber
03 INPUT PlayerGameScore
04 PlayerScore[GameNumber, PlayerNumber] ← PlayerGameScore
```

Describe the data structure the programmer has used for the storage of all player scores.

[2]

- (c) The team wants the program to produce a report, with the following specification.

The program outputs the total number of player scores that are:

- 50 and over but less than 100
- 100 or higher.

You can assume that before the section runs, the program has assigned all eight player scores to the `PlayerScore` data structure.

A first attempt at the pseudocode is shown below:

```

01 Total50 ← 0
02 Total100 ← 0
03 FOR PlayerIndex ← 1 TO 8
04   FOR GameIndex ← 1 TO 20
05     IF PlayerScore[GameIndex, PlayerIndex] > 100
06       THEN
07         Total100 ← Total100 + 1
08       ELSE
09         IF PlayerScore[GameIndex, PlayerIndex] > 50
10           THEN
11             Total50 ← Total50 + GameIndex
12           ENDIF
13         ENDIF
14       ENDFOR
15     ENDFOR
16 OUTPUT Total50
17 OUTPUT Total100

```

(iii) The programmer has made logic errors in the design.

State a line number at which an error occurs.

Explain the error or write the corrected pseudocode statement.

Line number

Explanation

[1]

138. O/N 16/P23/Q2/a

2 A sensing device sends bit values to a computer along data channels.

- Channel 1 transmits a sequence of binary values from a sensor
- Channel 2 transmits at regular intervals to indicate whether the sensor is switched on or off:
 - 0 indicates switched off
 - 1 indicates switched on

A program tests the bits received from the sensing device.

A program reads the signal from Channel 2 after every six values from Channel 1.

A built-in function `READ(<ChannelNumber>)` reads a value from the specified channel.

Pseudocode for the program is as follows:

```

01   BitCount ← 0
02   Status2 ← READ(2)
03   WHILE Status2 = 1

```

```

04
05     FOR ReadingCount ← 1 TO 6
06         ThisBit ← READ(1)
07         IF ThisBit = 1
08             THEN
09                 BitCount ← BitCount + 1
10         ENDIF
11         IF BitCount = 5
12             THEN
13                 OUTPUT "Error - Investigate"
14                 BitCount ← 0
15         ENDIF
16     ENDFOR
17
18     Status2 ← READ(2)
19     ENDWHILE

```

(a) Trace the execution of the program for the following sequence of bits.

Channel 1		1	0	1	1	1	0		1	1	0	0	1	1	
Channel 2	1							1							0

Status2	ReadingCount	ThisBit	BitCount	OUTPUT
			0	
1	1	1	1	
	2			

139. M/J 16/P21/Q4/b

- (b)** A team needs to write a program to implement an online shopping system. Customers will access the program via a website.

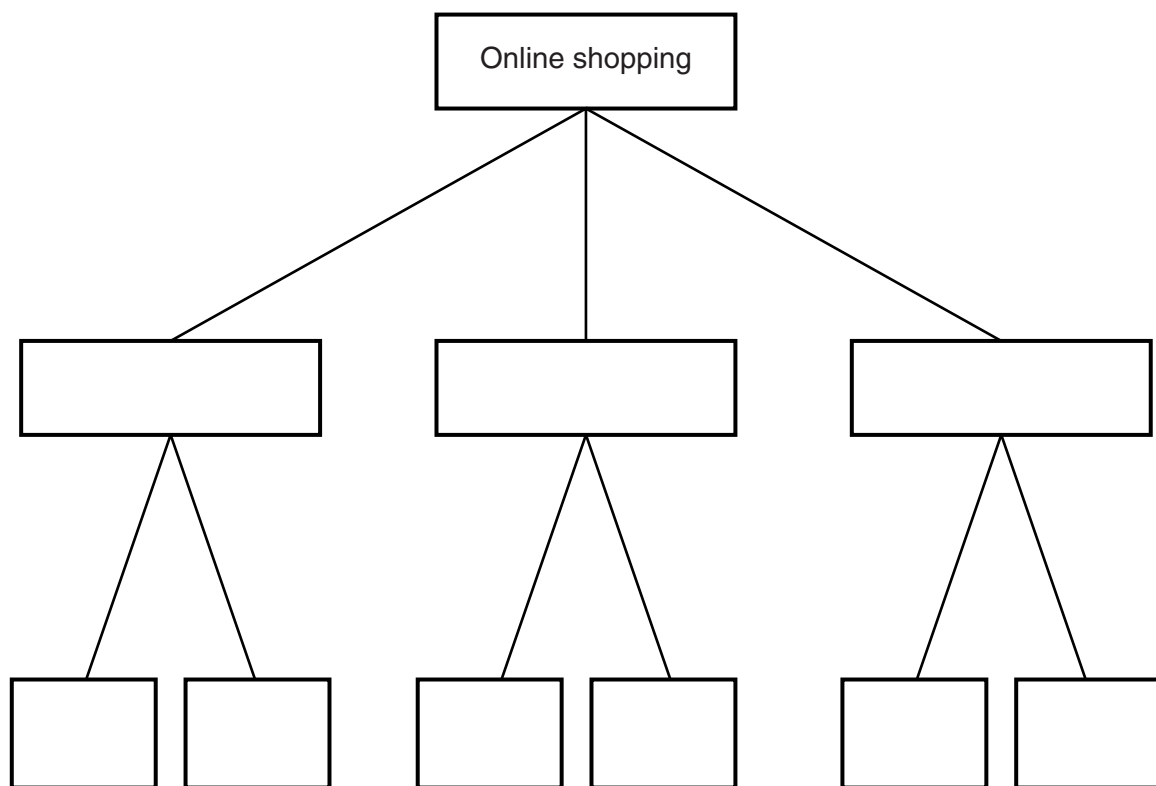
Customers can search for items before adding them to a virtual shopping basket. When they have finished shopping, they pay for the items. The program provides output for the dispatch of the items.

Some of the key features of the system are as follows:

- a customer can add many items to the shopping basket
- payment may be either by credit or debit card, or by adding to a customer account
- the shop may dispatch the items in one or more packages

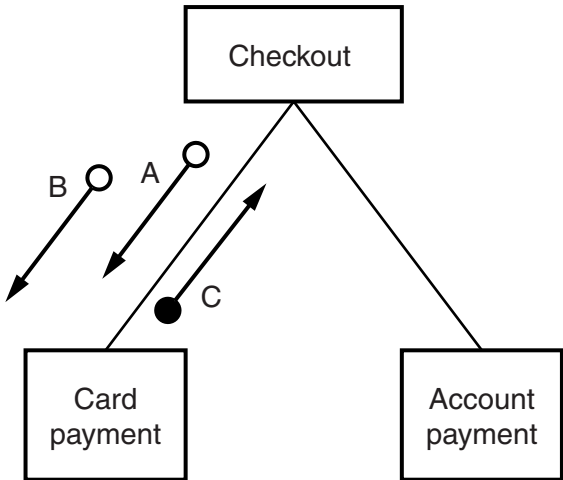
The structure chart below shows the program modules only.

- (i)** Draw on the chart, the symbols that represent the key features listed in **part (b)** above.



[3]

(ii) A section of the chart in **part (b)(i)** is shown below. It is to show the parameters passed between the Checkout and Card payment modules.



Name the three data items corresponding to the arrows.

Arrow	Data item
A	
B	
C	

[3]

140. M/J 16/P21/Q6/a,b

6 A string-handling function has been developed. The pseudocode for this function is shown below.

For the built-in functions list, refer to the **Appendix** on page 18.

```

FUNCTION SSM(String1, String2 : STRING) RETURNS INTEGER
  DECLARE n, f, x, y : INTEGER

  n ← 0
  f ← 0

  REPEAT
    n ← n + 1
    x ← n
    y ← 1
    WHILE MID(String1, x, 1) = MID(String2, y, 1)
      IF y = LENGTH(String2)
        THEN
          f ← n
        ELSE
          x ← x + 1
          y ← y + 1
        ENDIF
      ENDWHILE
    UNTIL (n = LENGTH(String1)) OR (f <> 0)
  RETURN f
ENDFUNCTION

```

- (a) Complete the trace table below by performing a dry run of the function when it is called as follows:

SSM("RETRACE", "RAC")

n	f	x	y	MID(String1, x, 1)	MID(String2, y, 1)
0	0				

[6]

- (b) (i) Describe the purpose of function SSM.

[2]

- (ii) One of the possible return values from function SSM has a special meaning.

State the value and its meaning.

Value

Meaning

[2]

- (iii) There is a problem with the logic of the pseudocode. This could generate a run-time error.

Describe the problem.

[2]

Appendix of this paper:

Appendix

Built-in functions

In each function below, if the function call is not properly formed, the function returns an error.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`

returns the string of length `y` starting at position `x` from `ThisString`

Example: `MID ("ABCDEFGH", 2, 3)` will return string `"BCD"`

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns the leftmost `x` characters from `ThisString`

Example: `LEFT ("ABCDEFGH", 3)` will return string `"ABC"`

`RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING`

returns the rightmost `x` characters from `ThisString`

Example: `RIGHT ("ABCDEFGH", 3)` will return string `"FGH"`

`ASC(ThisChar : CHAR) RETURNS INTEGER`

returns the ASCII value of character `ThisChar`

Example: `ASC ('w')` will return 87

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`

Example: `LENGTH("Happy Days")` will return 10

String operator

`&` operator

concatenates (joins) two strings

Example: `"Summer" & " " & "Pudding"` produces `"Summer Pudding"`

141. M/J 16/P22/Q4/b

- (b)** A team needs to write a program to implement an online shopping system. Customers will access the program via a website.

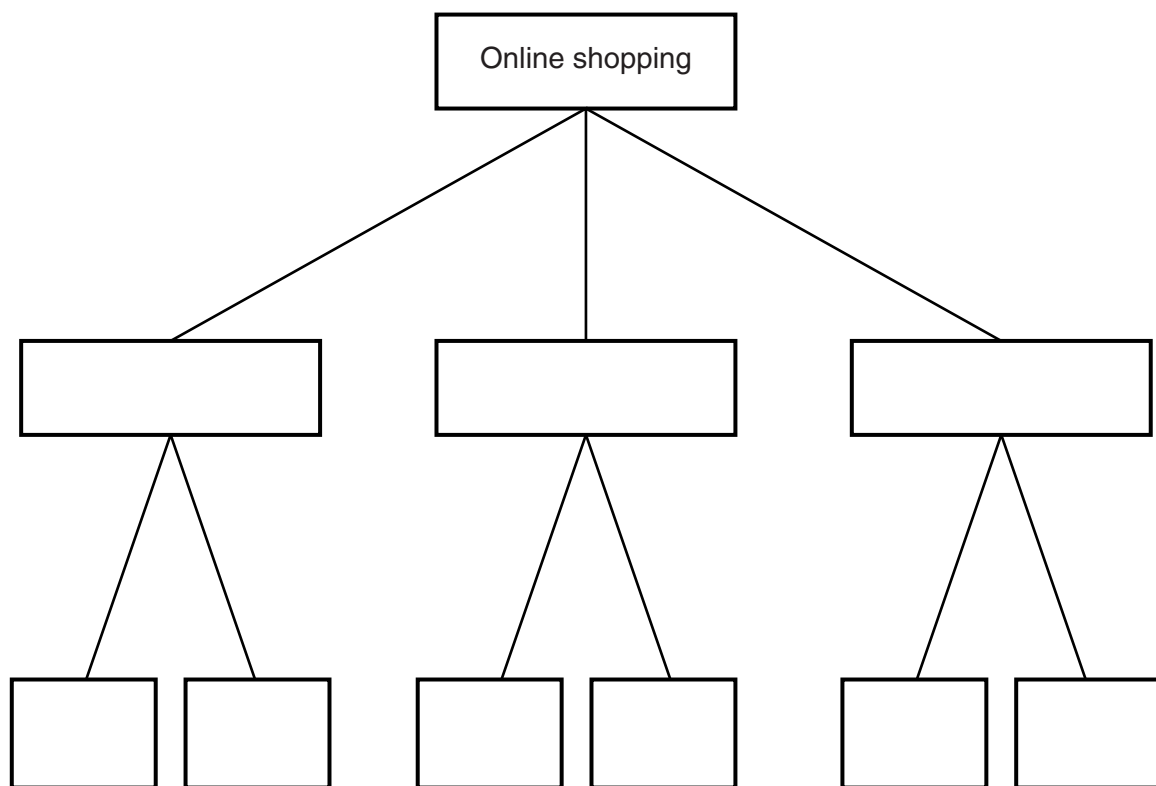
Customers can search for items before adding them to a virtual shopping basket. When they have finished shopping, they pay for the items. The program provides output for the dispatch of the items.

Some of the key features of the system are as follows:

- a customer can add many items to the shopping basket
- payment may be either by credit or debit card, or by adding to a customer account
- the shop may dispatch the items in one or more packages

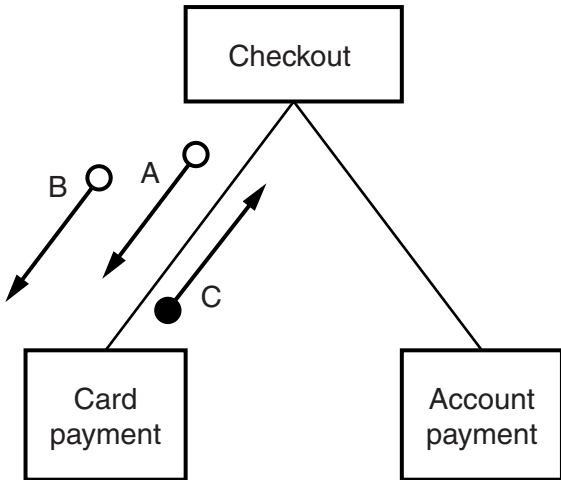
The structure chart below shows the program modules only.

- (i)** Draw on the chart, the symbols that represent the key features listed in **part (b)** above.



[3]

(ii) A section of the chart in **part (b)(i)** is shown below. It is to show the parameters passed between the Checkout and Card payment modules.



Name the three data items corresponding to the arrows.

Arrow	Data item
A	
B	
C	

[3]

142. M/J 16/P22/Q6/a,b

6 A string-handling function has been developed. The pseudocode for this function is shown below.

For the built-in functions list, refer to the **Appendix** on page 18.

```

FUNCTION SSM(String1, String2 : STRING) RETURNS INTEGER
  DECLARE n, f, x, y : INTEGER

  n ← 0
  f ← 0

  REPEAT
    n ← n + 1
    x ← n
    y ← 1
    WHILE MID(String1, x, 1) = MID(String2, y, 1)
      IF y = LENGTH(String2)
        THEN
          f ← n
        ELSE
          x ← x + 1
          y ← y + 1
        ENDIF
      ENDWHILE
    UNTIL (n = LENGTH(String1)) OR (f <> 0)
  RETURN f
ENDFUNCTION

```

- (a) Complete the trace table below by performing a dry run of the function when it is called as follows:

SSM("RETRACE", "RAC")

n	f	x	y	MID(String1, x, 1)	MID(String2, y, 1)
0	0				

[6]

- (b) (i) Describe the purpose of function SSM.

[2]

- (ii) One of the possible return values from function SSM has a special meaning.

State the value and its meaning.

Value

Meaning

[2]

- (iii) There is a problem with the logic of the pseudocode. This could generate a run-time error.

Describe the problem.

[2]

Appendix of this paper:

Appendix

Built-in functions

In each function below, if the function call is not properly formed, the function returns an error.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`

returns the string of length `y` starting at position `x` from `ThisString`

Example: `MID ("ABCDEFGH", 2, 3)` will return string `"BCD"`

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns the leftmost `x` characters from `ThisString`

Example: `LEFT ("ABCDEFGH", 3)` will return string `"ABC"`

`RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING`

returns the rightmost `x` characters from `ThisString`

Example: `RIGHT ("ABCDEFGH", 3)` will return string `"FGH"`

`ASC(ThisChar : CHAR) RETURNS INTEGER`

returns the ASCII value of character `ThisChar`

Example: `ASC ('w')` will return 87

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`

Example: `LENGTH("Happy Days")` will return 10

String operator

`&` operator

concatenates (joins) two strings

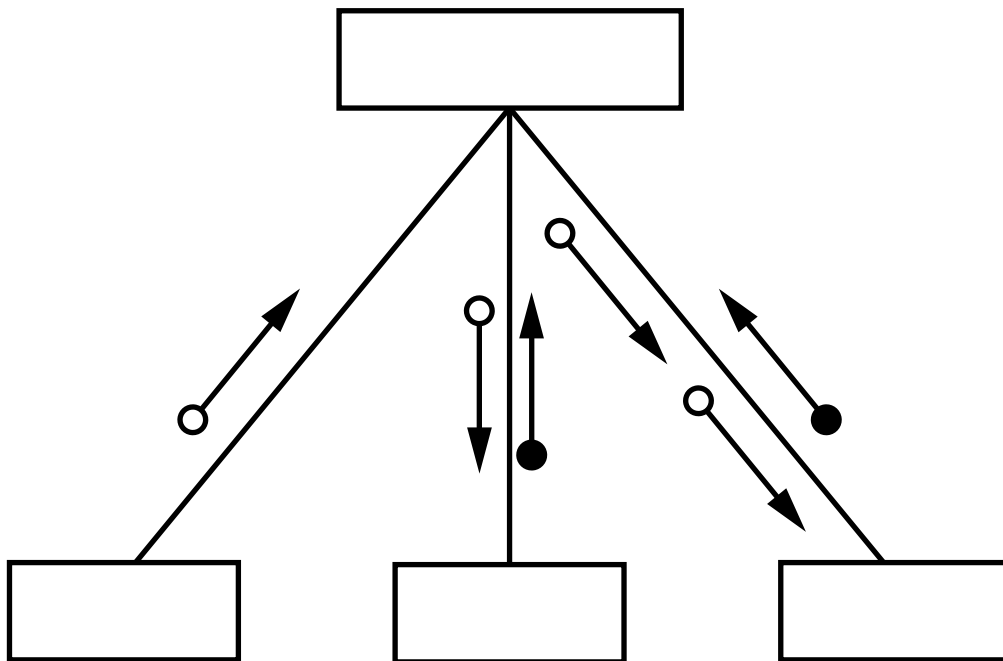
Example: `"Summer" & " " & "Pudding"` produces `"Summer Pudding"`

143. M/J 16/P23/Q4/b

- (b) (i) The structure chart shows part of the design of a program for an online shopping system.

The user has already added a number of products to their virtual basket.

Draw on the chart, the symbol to show that the process of modifying the basket contents may be iterated (repeated).



[1]

- (ii) Each arrow in the structure chart above represents a parameter.

The table below shows the three data items that the six parameters pass between modules.

Tick (✓) to match each parameter to the correct data item.

Data item	Parameter					
	A	B	C	D	E	F
Product ID						
Quantity						
Flag Value – indicating operation success or fail						

[4]

144. M/J 16/P23/Q6/a,b

- 6 A string-handling function has been developed.

For the built-in functions list, refer to the **Appendix** on the last page.

The pseudocode for this function is shown below.

```

FUNCTION SF(ThisString : STRING) RETURNS STRING
  DECLARE x          : CHAR
  DECLARE NewString  : STRING
  DECLARE Flag       : BOOLEAN
  DECLARE m, n       : INTEGER

  Flag ← TRUE
  NewString ← ""
  m ← LENGTH(ThisString)

  FOR n ← 1 TO m
    IF Flag = TRUE
      THEN
        x ← UCASE(MID(ThisString, n, 1))
        Flag ← FALSE
      ELSE
        x ← LCASE(MID(ThisString, n, 1))
      ENDIF
    NewString ← NewString & x

    IF x = " "
      THEN
        Flag ← TRUE
      ENDIF
  ENDFOR

  RETURN NewString
ENDFUNCTION

```

- (a) (i) Complete the trace table below by performing a dry run of the function when it is called as follows:

SF("big BEN")

n	x	Flag	m	NewString

[4]

- (ii) Describe the purpose of function SF.

[2]

- (b) Test data must be designed for the function SF.

- (i) State what happens when the function is called with an empty string.

[1]

(ii) The function should be thoroughly tested.

Give **three** examples of non-empty strings that may be used.

In each case explain why the test string has been chosen.

String

Explanation

String

Explanation

String

Explanation

[3]

Appendix of this paper:

Appendix

Built-in functions (Pseudocode)

In each function below, if the function call is not properly formed, the function returns an error.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns the string of length y starting at position x from ThisString

Example: **MID**("ABCDEFGH", 2, 3) will return string "BCD"

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns the leftmost x characters from ThisString

Example: **LEFT**("ABCDEFGH", 3) will return string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING

returns the rightmost x characters from ThisString

Example: **RIGHT**("ABCDEFGH", 3) will return string "FGH"

CHR(x : INTEGER) RETURNS CHAR

returns the character whose ASCII value is x

Example: **CHR**(87) will return 'W'

ASC(x : CHAR) RETURNS INTEGER

returns the ASCII value of character x

Example: **ASC**('W') will return 87

LCASE(x : CHAR) RETURNS CHAR

returns the lower case equivalent character of x

Example: **LCASE**('W') will return 'w'

UCASE(x : CHAR) RETURNS CHAR

returns the upper case equivalent character of x

Example: **UCASE**('h') will return 'H'

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: **INT(27.5415)** will return **27**

145. O/N 15/P21/Q3/a

3 A program is to simulate the operation of a particular type of logic gate.

- The gate has two inputs (TRUE or FALSE) which are entered by the user.
- The program will display the output (TRUE or FALSE) from the gate.

The program uses the following identifiers in the pseudocode below:

Identifier	Data type	Description
InA	BOOLEAN	Input signal
InB	BOOLEAN	Input signal
OutZ	BOOLEAN	Output signal

```

01  INPUT InA
02  INPUT InB
03  IF (InA = FALSE AND InB = FALSE) OR (InA = FALSE AND InB = TRUE)
                                OR (InA = TRUE AND InB = FALSE)
04      THEN
05          OutZ ← TRUE
06      ELSE
07          OutZ ← FALSE
08  ENDIF
09  OUTPUT OutZ

```

(a) The programmer chooses the following four test cases.

Show the output (OutZ) expected for each test case.

Test case	Input		Output OutZ
	InA	InB	
1	TRUE	TRUE	
2	TRUE	FALSE	
3	FALSE	TRUE	
4	FALSE	FALSE	

Show the values assigned to each variable.

FractionString ← "3/7+2/9"	
N3Char ← ONECHAR(FractionString, 5)	(i) N3Char [1]
Op ← ONECHAR(FractionString, 4)	(ii) Op [1]

(iii) Complete the function call to isolate the character '9' from FractionString.

FractionString ← "3/7+2/9"

ONECHAR(FractionString,) [1]

The following additional variables are to be used by the program:

Identifier	Data type	Description
N1	INTEGER	The number value of N1Char
N2	INTEGER	The number value of N2Char
N3	INTEGER	The number value of N3Char
N4	INTEGER	The number value of N4Char
TopAnswer	INTEGER	The numerator of the fraction answer
BottomAnswer	INTEGER	The denominator of the fraction answer

(c) The following pseudocode uses these additional built-in functions:

TONUM(ThisDigit : CHAR) RETURNS INTEGER
returns the integer value of character ThisDigit
For example: TONUM('8') returns digit 8.

TOSTR(ThisNumber : INTEGER) RETURNS STRING
returns the string value of integer ThisNumber
For example: TOSTR(27) returns "27".

Study the pseudocode.

Complete the **three** dry runs for the three given values of FractionString.

```
OUTPUT "Enter the expression"
INPUT FractionString

// isolate each number digit and assign its number value
N1Char ← ONECHAR(FractionString, 1)
N1 ← TONUM(N1Char)
N2Char ← ONECHAR(FractionString, 3)
N2 ← TONUM(N2Char)
N3Char ← ONECHAR(FractionString, 5)
N3 ← TONUM(N3Char)
N4Char ← ONECHAR(FractionString, 7)
N4 ← TONUM(N4Char)

BottomAnswer ← N2 * N4

Op ← ONECHAR(FractionString, 4)
IF Op = '+'
    THEN
        // add fractions
        TopAnswer ← (BottomAnswer/N2) * N1 + (BottomAnswer/N4) * N3
    ELSE
        // subtract fractions
        TopAnswer ← (BottomAnswer/N2) * N1 - (BottomAnswer/N4) * N3
ENDIF

IF TopAnswer = BottomAnswer
    THEN
        OUTPUT '1'
    ELSE
        IF TopAnswer > BottomAnswer
            THEN
                TopAnswer ← TopAnswer MOD BottomAnswer
                // the & operator joins strings or character values
                OUTPUT "1 " & TOSTR(TopAnswer) & "/" & TOSTR(BottomAnswer)
            ELSE
                OUTPUT TOSTR(TopAnswer) & "/" & TOSTR(BottomAnswer)
            ENDIF
        ENDIF
    ENDIF
```

(i) FractionString $\leftarrow$ "2/5-3/8"

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT

[2]

(ii) FractionString $\leftarrow$ "3/4+1/4"

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT

[2]

(iii) FractionString $\leftarrow$ "7/9+2/3"

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT

[3]

147. O/N 15/P21/Q8/d

8 In this question you will need to use the given pseudocode built-in function:

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR
returns the single character at position Position (counting from the start of the string with value 1)
from the string ThisString.
For example: ONECHAR("Barcelona", 3) returns 'r'.

- (d) The programmer writes code from the given pseudocode design. The program works, but the design is limited.

The programmer is to make amendments to the design following suggested specification changes.

- (i) State the name for this type of maintenance.

[1]

- (ii) Describe **three** specification changes which will make the program more useful.

1

2

3

[3]

148. O/N 15/P22/Q2/a

2 A program is to simulate the operation of a particular type of logic gate.

- The gate has two inputs (0 or 1) which are entered by the user.
- The program will display the output (0 or 1) from the gate.

The program uses the following identifiers in the pseudocode below:

Identifier	Data type	Description
P	INTEGER	Input signal
Q	INTEGER	Input signal
X	INTEGER	Output signal

```

01 INPUT P
02 INPUT Q
03 IF (P = 1 AND Q = 0) OR (P = 0 AND Q = 1) OR (P = 0 AND Q = 0)
04     THEN
05         X ← 0
06     ELSE
07         X ← 1
08 ENDIF
09 OUTPUT X

```

(a) The programmer chooses the following four test cases.

Show the output (X) for each test case.

Test case	Input		Output X
	P	Q	
1	1	1	
2	1	0	
3	0	1	
4	0	0	

[4]

149. O/N 15/P22/Q5/a(i)

5 A program is to process a set of integers.

The integers are stored in an array, Num. The first N elements are to be processed.

150. O/N 15/P23/Q3/a

3 A program is to simulate the operation of a particular type of logic gate.

- The gate has two inputs (TRUE or FALSE) which are entered by the user.
- The program will display the output (TRUE or FALSE) from the gate.

The program uses the following identifiers in the pseudocode below:

Identifier	Data type	Description
InA	BOOLEAN	Input signal
InB	BOOLEAN	Input signal
OutZ	BOOLEAN	Output signal

```

01  INPUT InA
02  INPUT InB
03  IF (InA = FALSE AND InB = FALSE) OR (InA = FALSE AND InB = TRUE)
                                OR (InA = TRUE AND InB = FALSE)
04      THEN
05          OutZ ← TRUE
06      ELSE
07          OutZ ← FALSE
08  ENDIF
09  OUTPUT OutZ

```

(a) The programmer chooses the following four test cases.

Show the output (OutZ) expected for each test case.

Test case	Input		Output OutZ
	InA	InB	
1	TRUE	TRUE	
2	TRUE	FALSE	
3	FALSE	TRUE	
4	FALSE	FALSE	

[4]

151. O/N 15/P23/Q8/a,b,c

also Unit 11 (4 marks); this unit 7 marks

8 In this question you will need to use the given pseudocode built-in function:

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR
 returns the single character at position Position (counting from the start of the string with value 1)
 from the string ThisString.
 For example: ONECHAR("Barcelona", 3) returns 'r'.

(a) Give the value assigned to variable *y* by the following statement:

y ← ONECHAR("San Francisco", 6)

y

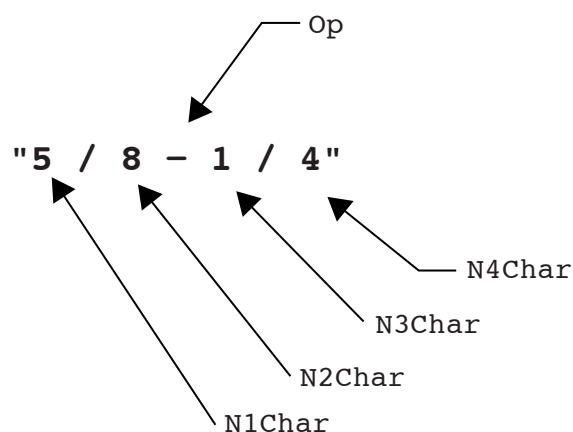
[1]

A program reads a string entered by the user. The string represents the addition or subtraction of two fractions. Each part of the fraction within the string is always a single digit only and the top digit is always less than the bottom digit.

Example strings are: "3/8+3/5" and "5/8-1/4"

The program steps are:

- the user enters the string
- the program isolates each digit and the operator
- the program computes the answer as either:
 - a fraction
 - a whole number followed by a fraction
 - a whole number
- the program displays the answer to the user



The identifier table shows the variables to be used to store the characters in the string as shown in the diagram.

Identifier	Data type	Description
FractionString	STRING	String input by user. For example: "5/8-1/4"
N1Char	CHAR	See diagram
N2Char	CHAR	See diagram
N3Char	CHAR	See diagram
N4Char	CHAR	See diagram
Op	CHAR	See diagram

(b) Study the sequence of pseudocode statements.

Show the values assigned to each variable.

FractionString ← "3/7+2/9"	
N3Char ← ONECHAR(FractionString, 5)	(i) N3Char [1]
Op ← ONECHAR(FractionString, 4)	(ii) Op [1]

(iii) Complete the function call to isolate the character '9' from FractionString.

FractionString ← "3/7+2/9"

ONECHAR(FractionString,)

[1]

The following additional variables are to be used by the program:

Identifier	Data type	Description
N1	INTEGER	The number value of N1Char
N2	INTEGER	The number value of N2Char
N3	INTEGER	The number value of N3Char
N4	INTEGER	The number value of N4Char
TopAnswer	INTEGER	The numerator of the fraction answer
BottomAnswer	INTEGER	The denominator of the fraction answer

(c) The following pseudocode uses these additional built-in functions:

TONUM(ThisDigit : CHAR) RETURNS INTEGER
returns the integer value of character ThisDigit
For example: TONUM('8') returns digit 8.

TOSTR(ThisNumber : INTEGER) RETURNS STRING
returns the string value of integer ThisNumber
For example: TOSTR(27) returns "27".

Study the pseudocode.

Complete the **three** dry runs for the three given values of FractionString.

```
OUTPUT "Enter the expression"
INPUT FractionString

// isolate each number digit and assign its number value
N1Char ← ONECHAR(FractionString, 1)
N1 ← TONUM(N1Char)
N2Char ← ONECHAR(FractionString, 3)
N2 ← TONUM(N2Char)
N3Char ← ONECHAR(FractionString, 5)
N3 ← TONUM(N3Char)
N4Char ← ONECHAR(FractionString, 7)
N4 ← TONUM(N4Char)

BottomAnswer ← N2 * N4

Op ← ONECHAR(FractionString, 4)
IF Op = '+'
    THEN
        // add fractions
        TopAnswer ← (BottomAnswer/N2) * N1 + (BottomAnswer/N4) * N3
    ELSE
        // subtract fractions
        TopAnswer ← (BottomAnswer/N2) * N1 - (BottomAnswer/N4) * N3
ENDIF

IF TopAnswer = BottomAnswer
    THEN
        OUTPUT '1'
    ELSE
        IF TopAnswer > BottomAnswer
            THEN
                TopAnswer ← TopAnswer MOD BottomAnswer
                // the & operator joins strings or character values
                OUTPUT "1 " & TOSTR(TopAnswer) & "/" & TOSTR(BottomAnswer)
            ELSE
                OUTPUT TOSTR(TopAnswer) & "/" & TOSTR(BottomAnswer)
            ENDIF
        ENDIF
    ENDIF
```

(i) FractionString $\leftarrow$ "2/5-3/8"

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT

[2]

(ii) FractionString $\leftarrow$ "3/4+1/4"

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT

[2]

(iii) FractionString $\leftarrow$ "7/9+2/3"

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT

[3]

152. O/N 15/P23/Q8/d

8 In this question you will need to use the given pseudocode built-in function:

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR
returns the single character at position Position (counting from the start of the string with value 1)
from the string ThisString.
For example: ONECHAR("Barcelona", 3) returns 'r'.

- (d) The programmer writes code from the given pseudocode design. The program works, but the design is limited.

The programmer is to make amendments to the design following suggested specification changes.

- (i) State the name for this type of maintenance.

[1]

- (ii) Describe **three** specification changes which will make the program more useful.

1

2

3

[3]

153. M/J 15/P21/Q1/c

- 1** A marathon runner records their time for a race in hours, minutes and seconds.

An algorithm is shown below in structured English.

INPUT race time as hours, minutes and seconds

CALCULATE race time in seconds

STORE race time in seconds

OUTPUT race time in seconds

- (c)** The program code will be tested using white-box testing.

- (i)** Explain what is meant by white-box testing.

[2]

- (ii)** Complete the table heading.

Complete Test Number 1.

Add the data for Test Number 2 and Test Number 3.

Input values					Output	
Test number	Race hours	Race minutes	Race seconds		Total time (seconds)	Message
1	3	4	13	11053	11053	
2				11053		
3				11053		

[6]

154. M/J 15/P21/Q3/a,b

- 3** When the guarantee on a computer runs out, the owner can take out insurance to cover breakdown and repairs.

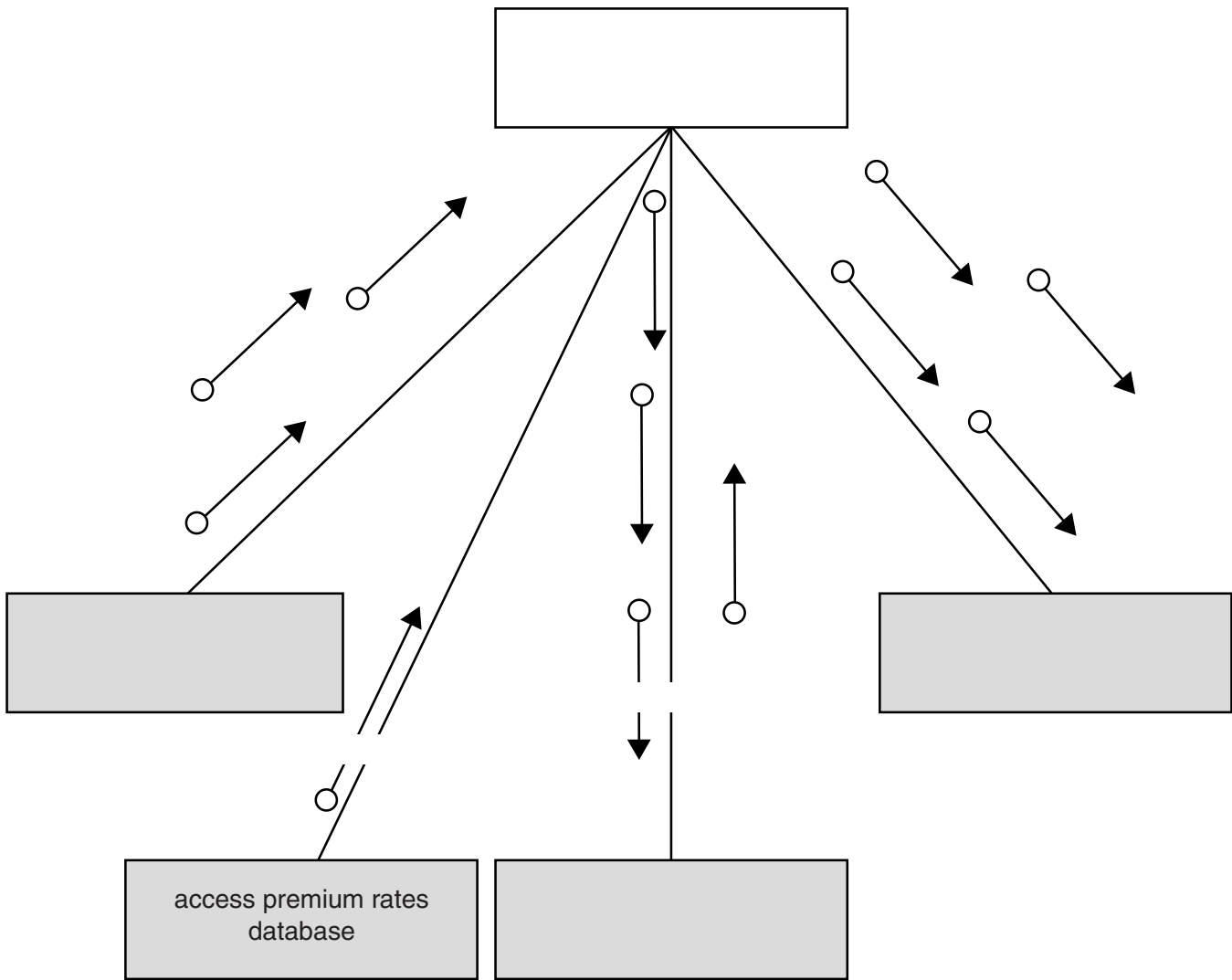
The price of the insurance is calculated from:

- the model of the computer
- the age of the computer
- the current insurance rates

Following an enquiry to the insurance company, the customer receives a quotation letter with the price of the insurance.

A program is to be produced.

The structure chart below shows the modular design for this process:



(a) Using the letters **A** to **D**, add the labelling to the chart boxes on the opposite page.

Modules	
A	Send quotation letter
B	Calculate price
C	Produce insurance quotation
D	Input computer details

[2]

(b) Using the letters **E** to **J**, complete the labelling on the chart opposite.

Some of these letters will be used more than once.

Data items	
E	CustomerName
F	CustomerEmail
G	Model
H	Age
I	PolicyCharge
J	PolicyNumber

[4]

155. M/J 15/P21/Q4/(ii)

4 A game is played between two players:

- they take turns at rolling a six-sided die (numbered 1 to 6) and record their throw
- a player scores 1 point if their throw is higher than their opponent
- they each roll the die 20 times
- if the player's throw is the same as their opponent, the total points is unchanged
- the winner is the player with the larger number of points after 20 throws

The pseudocode will use variable `NoOfThrows` as shown.

Identifier	Data type	Description
<code>NoOfThrows</code>	INTEGER	Loop control variable

(ii) Identify the game result which will produce incorrect output.

[1]

156. M/J 15/P21/Q5/b

5 A company creates two new websites, Site X and Site Y, for selling bicycles.

Various programs are to be written to process the sales data.

These programs will use data about daily sales made from Site X (using variable `salesX`) and Site Y (using variable `SalesY`).

Data for the first 28 days is shown below.

	SalesDate	SalesX	SalesY
1	03/06/2015	0	1
2	04/06/2015	1	2
3	05/06/2015	3	8
4	06/06/2015	0	0
5	07/06/2015	4	6
6	08/06/2015	4	4
7	09/06/2015	5	9
8	10/06/2015	11	9
9	11/06/2015	4	1
...			
28	01/07/2015	14	8

(b) The programmer writes a program from the following pseudocode design.

```

x ← 0
FOR DayNumber ← 1 TO 7
    IF SalesX[DayNumber] + SalesY[DayNumber] >= 10
        THEN
            x ← x + 1
            OUTPUT SalesDate[DayNumber]
        ENDIF
    ENDFOR
OUTPUT x

```

(i) Trace the execution of this pseudocode by completing the trace table below.

x	DayNumber	OUTPUT
0		

[4]

(ii) Describe, in detail, what this algorithm does.

[3]

157. M/J 15/P22/Q1/c

- 1** A marathon runner records their time for a race in hours, minutes and seconds.

An algorithm is shown below in structured English.

INPUT race time as hours, minutes and seconds

CALCULATE race time in seconds

STORE race time in seconds

OUTPUT race time in seconds

- (c)** The program code will be tested using white-box testing.

- (i)** Explain what is meant by white-box testing.

[2]

- (ii)** Complete the table heading.

Complete Test Number 1.

Add the data for Test Number 2 and Test Number 3.

Input values					Output	
Test number	Race hours	Race minutes	Race seconds		Total time (seconds)	Message
1	3	4	13	11053	11053	
2				11053		
3				11053		

[6]

158. M/J 15/P22/Q3/a,b

- 3** When the guarantee on a computer runs out, the owner can take out insurance to cover breakdown and repairs.

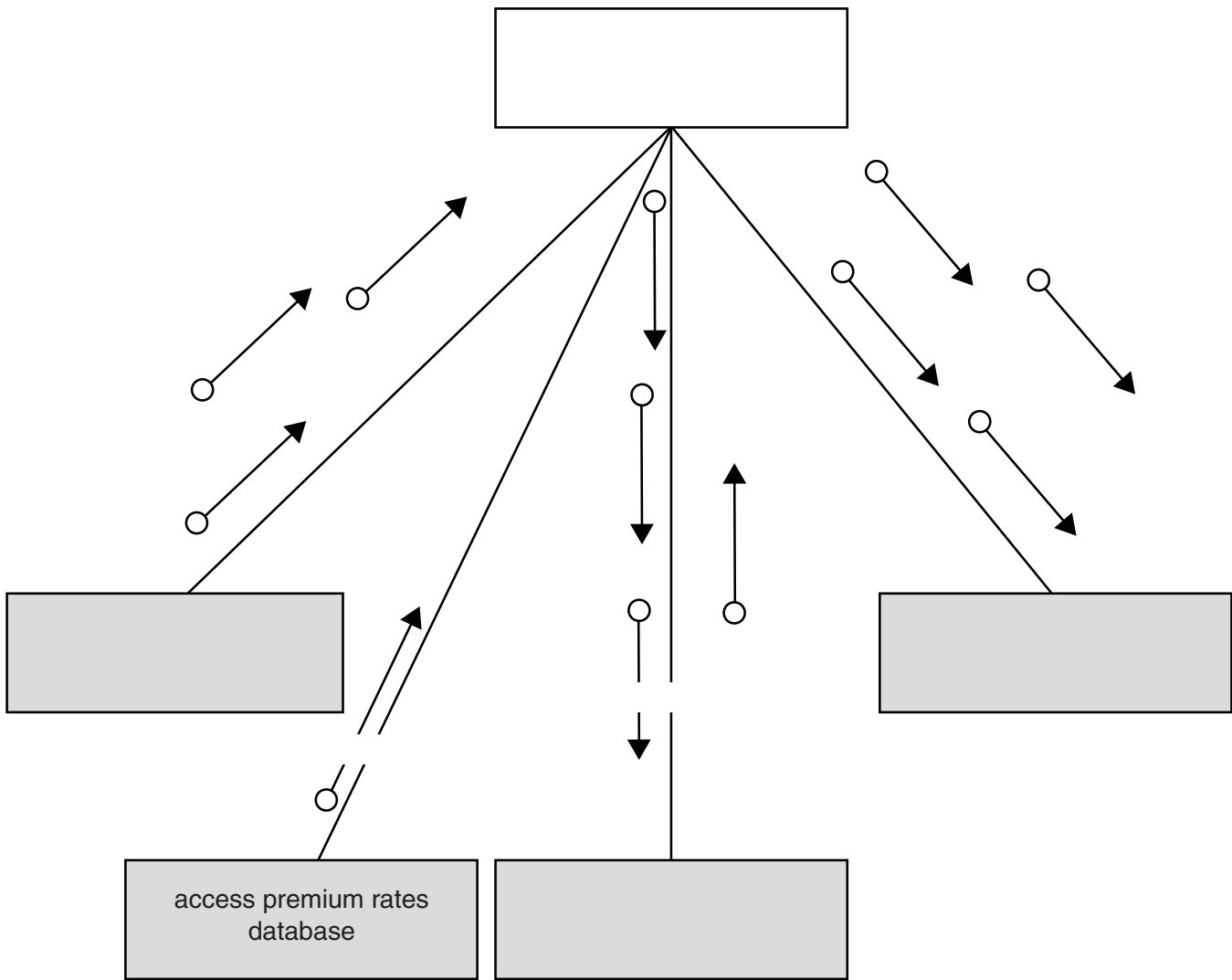
The price of the insurance is calculated from:

- the model of the computer
- the age of the computer
- the current insurance rates

Following an enquiry to the insurance company, the customer receives a quotation letter with the price of the insurance.

A program is to be produced.

The structure chart below shows the modular design for this process:



(a) Using the letters **A** to **D**, add the labelling to the chart boxes on the opposite page.

Modules	
A	Send quotation letter
B	Calculate price
C	Produce insurance quotation
D	Input computer details

[2]

(b) Using the letters **E** to **J**, complete the labelling on the chart opposite.

Some of these letters will be used more than once.

Data items	
E	CustomerName
F	CustomerEmail
G	Model
H	Age
I	PolicyCharge
J	PolicyNumber

[4]

159. M/J 15/P22/Q4/(ii)

4 A game is played between two players:

- they take turns at rolling a six-sided die (numbered 1 to 6) and record their throw
- a player scores 1 point if their throw is higher than their opponent
- they each roll the die 20 times
- if the player's throw is the same as their opponent, the total points is unchanged
- the winner is the player with the larger number of points after 20 throws

The pseudocode will use variable `NoOfThrows` as shown.

Identifier	Data type	Description
<code>NoOfThrows</code>	INTEGER	Loop control variable

(ii) Identify the game result which will produce incorrect output.

[1]

160. M/J 15/P22/Q5/b

5 A company creates two new websites, Site X and Site Y, for selling bicycles.

Various programs are to be written to process the sales data.

These programs will use data about daily sales made from Site X (using variable `salesX`) and Site Y (using variable `SalesY`).

Data for the first 28 days is shown below.

	SalesDate	SalesX	SalesY
1	03/06/2015	0	1
2	04/06/2015	1	2
3	05/06/2015	3	8
4	06/06/2015	0	0
5	07/06/2015	4	6
6	08/06/2015	4	4
7	09/06/2015	5	9
8	10/06/2015	11	9
9	11/06/2015	4	1
...			
28	01/07/2015	14	8

(b) The programmer writes a program from the following pseudocode design.

```

x ← 0
FOR DayNumber ← 1 TO 7
    IF SalesX[DayNumber] + SalesY[DayNumber] >= 10
        THEN
            x ← x + 1
            OUTPUT SalesDate[DayNumber]
        ENDIF
    ENDFOR
OUTPUT x

```

(i) Trace the execution of this pseudocode by completing the trace table below.

x	DayNumber	OUTPUT
0		

[4]

(ii) Describe, in detail, what this algorithm does.

[3]

161. M/J 15/P23/Q2/b(i)

(b) Bank customers withdraw money from their account at a cash dispenser machine using their bank card. The machine operates as follows:

- it can dispense the following notes:
 - \$50
 - \$20
 - \$10
- the maximum amount for a single withdrawal is \$500

When a customer withdraws money, they enter the amount to withdraw. (This must be a multiple of \$10).

The machine will always dispense the least possible number of notes.

A program is designed for the machine to process a withdrawal.

The following variables are used:

Identifier	Data type	Description
Amount	INTEGER	Amount to withdraw entered by the user
FiftyDollar	INTEGER	Number of \$50 notes to dispense
TwentyDollar	INTEGER	Number of \$20 notes to dispense
TenDollar	INTEGER	Number of \$10 notes to dispense
Temp	INTEGER	Used in the calculation of the number of each note required

(i) The following four tests have been designed.

Complete the test data table showing the expected results with comments.

Input value	Output			Comment
	FiftyDollar	TwentyDollar	TenDollar	
70	1	1	0	Least possible number of notes
85				
130				
600				

[3]

162. M/J 15/P23/Q3/(i)

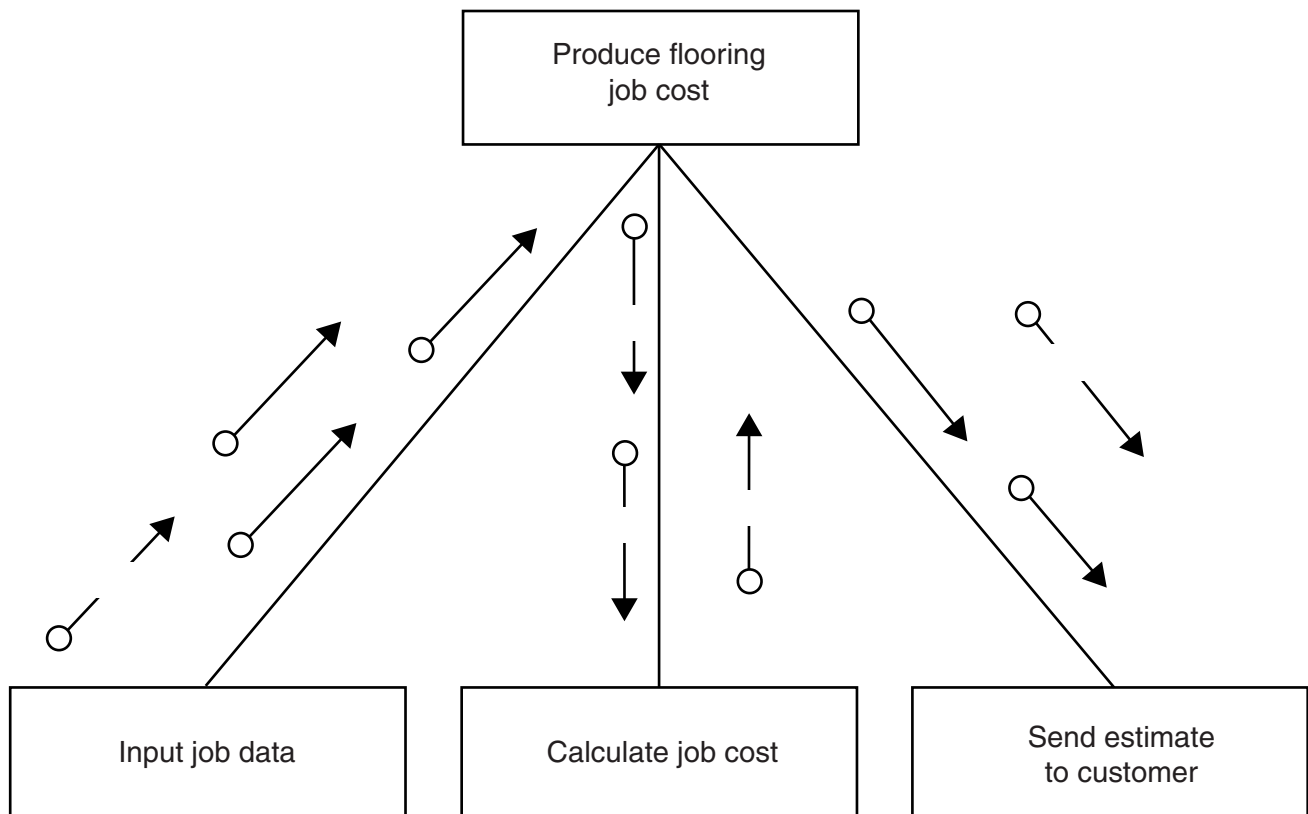
- 3** A flooring company provides for each customer an estimated price for a new job. Each job is given a Job ID.

The job cost is calculated from the length (nearest metre) and width (nearest metre) of the room.

The process for calculating the price is as follows:

- the floor area is calculated with 18% added to allow for wastage
- the job cost is calculated at \$50 per square metre

The structure chart shows the modular design for a program to produce a new job cost.



- (i)** Give the data items corresponding to the labels A to E in the structure chart.

A
B
C
D
E

[5]

163. M/J 15/P23/Q5/b

- 5** A firm employs workers who assemble amplifiers. Each member of staff works an agreed number of hours each day.

The firm records the number of completed amplifiers made by each employee each day.

Management monitor the performance of all its workers.

Production data was collected for 3 workers over 4 days.

Daily hours worked	
Worker 1	5
Worker 2	10
Worker 3	10

Production data			
	Worker 1	Worker 2	Worker 3
Day 1	10	20	9
Day 2	11	16	11
Day 3	10	24	13
Day 4	14	20	17

A program is to be written to process the production data.

- (a) The production data is to be stored in a 2-dimensional array `ProductionData`, declared as follows:

```
DECLARE ProductionData ARRAY[1:4, 1:3] : INTEGER
```


Answers Section

1. M/J 26/P21/Q3/a,b

3(a)	MP1 Investigate the current system /any example // investigate any other similar systems MP2 Discussion/interviews with managers MP3 Discussion/interviews with staff who operate the system MP4 Carry out a survey of employees/customers // carry out a focus group of employees/customers MP5 Observe the current system in use MP6 Produce the <u>requirements specification</u> MP7 Decide on the (program development) life cycle to be used Max 3	3
3(b)	Before any programming is started, the DESIGN stage for the proposed new system must be completed. For this large project, the tasks will be broken down into several MODULES // SUB-TASKS each will be coded by a different programming team. Once each team has completed its work, INTEGRATION // UNIT testing must be carried out. Before the new system is delivered to the business, ALPHA testing is carried out in-house. It is decided to install the software initially in only one of the call centres where BETA testing is carried out. The new software is finally rolled out to all five call centres. In the first week of its use, staff report that the software still requires extra features which were not originally agreed. The software is withdrawn until ADAPTIVE maintenance is completed. 1 mark per gap	6

2. M/J 26/P21/Q8/a,b,c

8(a)	<table><tr><th>LeftPointer</th><th>RightPointer</th><th>MidPointer</th></tr><tr><td>1</td><td>15</td><td>8</td></tr><tr><td></td><td>7</td><td>4</td></tr><tr><td>5</td><td></td><td>6</td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	LeftPointer	RightPointer	MidPointer	1	15	8		7	4	5		6																4
	LeftPointer	RightPointer	MidPointer																										
	1	15	8																										
		7	4																										
	5		6																										
8(b)	Found 6	1																											
8(c)	6	1																											

3. M/J 26/P22/Q1/b,c,d,e,f

1(b)(i)	One mark per state Example answers: 1 Sensor activated (mode) // Intruder/movement detected (mode) // Alert Mode 2 Alarm sounding // Light flashing (mode) // Alarm activated 3 Error mode // Power loss detected (mode) // Battery too low mode 4 Tamper detected (mode) // Lockdown (mode) 5 Disarm Mode // Maintenance Mode // Password <u>Change</u> Mode // Service Mode Max 2	2	
1(b)(ii)	One mark per event Example answers: 1 Movement / pressure (detected) 2 Window opened / broken (detected) 3 Door open (detected) 4 Entering wrong password (repeatedly) 5 Entering password to enter / exit maintenance mode / silence alarm 6 Tampering / damaging alarm (system) 7 Time delay // Time passed after alarm triggered / before alarm activated 8 Battery charge too low 9 Pressing panic button	3	Do not accept repeating examples in stem. Reject Turning system on. Reject reference to creating password.

1(c)	Example Solution <table><thead><tr><th>rule being tested</th><th>example of test data</th><th>reason for invalid test data</th></tr></thead><tbody><tr><td>rule 1</td><td>1A</td><td>password is too short</td></tr><tr><td>rule 1</td><td>PHRASE345678</td><td>Password too long</td></tr><tr><td>rule 2</td><td>ABCDEF</td><td>Does not contain a digit</td></tr><tr><td>rule 2</td><td>1234567</td><td>Does not contain a letter</td></tr><tr><td>rule 3</td><td>PASSWRD2</td><td>letters are repeated</td></tr><tr><td>rule 3</td><td>ABCDEF11</td><td>Digits / Numbers are repeated</td></tr><tr><td>rule 3</td><td>AAA111</td><td>Both digits and letters are repeated Accept Characters repeated</td></tr></tbody></table> Mark as follows: Rule 1 test: 1 mark for one example of test data for rule 1 and appropriate reason <table><tr><td>Test data</td><td>A string of more than 8 characters that contains both letters and digits. The letters must be upper case Each character only occurs once</td></tr><tr><td>Reason</td><td>Password is too long</td></tr></table>	rule being tested	example of test data	reason for invalid test data	rule 1	1A	password is too short	rule 1	PHRASE345678	Password too long	rule 2	ABCDEF	Does not contain a digit	rule 2	1234567	Does not contain a letter	rule 3	PASSWRD2	letters are repeated	rule 3	ABCDEF11	Digits / Numbers are repeated	rule 3	AAA111	Both digits and letters are repeated Accept Characters repeated	Test data	A string of more than 8 characters that contains both letters and digits. The letters must be upper case Each character only occurs once	Reason	Password is too long	3 Do not accept once if lower-case letters or characters that are not letters or digits used. Reject if test data breaks Rule 2 or/and Rule 3
rule being tested	example of test data	reason for invalid test data																												
rule 1	1A	password is too short																												
rule 1	PHRASE345678	Password too long																												
rule 2	ABCDEF	Does not contain a digit																												
rule 2	1234567	Does not contain a letter																												
rule 3	PASSWRD2	letters are repeated																												
rule 3	ABCDEF11	Digits / Numbers are repeated																												
rule 3	AAA111	Both digits and letters are repeated Accept Characters repeated																												
Test data	A string of more than 8 characters that contains both letters and digits. The letters must be upper case Each character only occurs once																													
Reason	Password is too long																													

1(c)	Rule 2 test (1 mark): 1 mark for two examples of test data that only breaks rule 2 and appropriate reasons. <table><tr><td>Test data</td><td>A string between 6 and 8 characters long that contains no digits</td></tr><tr><td>Reason</td><td>Does not contain a digit</td></tr><tr><td>Test data</td><td>A string between 6 and 8 characters long that contains no letters</td></tr><tr><td>Reason</td><td>Does not contain a letter</td></tr></table> Rule 3: 1 mark for two examples of test data for rule 3 and appropriate reasons <table><tr><td>Test data</td><td>A string between 6 and 8 characters long where only digits are repeated</td></tr><tr><td>Reason</td><td>Digits are repeated</td></tr><tr><td>Test data</td><td>A string between 6 and 8 characters long where both characters and digits are repeated</td></tr><tr><td>Reason</td><td>Both digits and letters are repeated</td></tr></table>	Test data	A string between 6 and 8 characters long that contains no digits	Reason	Does not contain a digit	Test data	A string between 6 and 8 characters long that contains no letters	Reason	Does not contain a letter	Test data	A string between 6 and 8 characters long where only digits are repeated	Reason	Digits are repeated	Test data	A string between 6 and 8 characters long where both characters and digits are repeated	Reason	Both digits and letters are repeated		Reject if letters repeated Reject if digits repeated Reject characters repeated Reject if letters repeated Reject if no letters
Test data	A string between 6 and 8 characters long that contains no digits																		
Reason	Does not contain a digit																		
Test data	A string between 6 and 8 characters long that contains no letters																		
Reason	Does not contain a letter																		
Test data	A string between 6 and 8 characters long where only digits are repeated																		
Reason	Digits are repeated																		
Test data	A string between 6 and 8 characters long where both characters and digits are repeated																		
Reason	Both digits and letters are repeated																		
1(d)	One mark per example Example answers: 1 Structure chart // Problem / Program broken into modules // Modular Diagram 2 Flowchart 3 Data Flow Diagram 4 Identifier / Variable table // Data structure / Record structure (designs) 5 File structure // Database Design 6 E-R Diagram 7 HCI Design // Report Design 8 Class Diagram // UML Diagram 9 Programming language to use Max 3	3	Reject 'Decomposition' by itself Reject Module Overview by itself																
1(e)	One mark per stage 1 Analysis 2 Coding // Implementation 3 Maintenance Max 2 marks	2	NE Development / Programming																
1(f)(i)	Integration	1																	

1(f)(ii)	One mark for the name, one for the description Method: Stub (testing) Description: 1 A simple / dummy module is written to replace the module (that is not working) 2 The simple / dummy module will return an expected value / will output a message (to show the module has been called)	3	Do not mark description if method incorrect
1(a)(i)	STRING	1	
1(a)(ii)	IF LENGTH(Pass) < 6 OR LENGTH(Pass) > 8 / >= 9 THEN Mark as follows: 1 Mark: Use of LENGTH function twice 1 Mark: Number 8 used in second condition	2	
1(a)(iii)	Mark as follows: 1 DECLARE LetterFlag, DigitFlag: <u>BOOLEAN</u> 1 mark for <u>BOOLEAN</u> 2 LetterFlag ← <u>FALSE</u> 1 mark for <u>FALSE</u> 3 FOR Index ← 1 TO <u>LENGTH</u>(Pass) 1 mark for <u>LENGTH</u> 4 Character ← <u>MID</u>(Pass, <u>Index</u>, <u>1</u>) 1 mark for <u>MID</u>, <u>Index</u> and 1 5 IF <u>IS_NUM</u>(Character) = TRUE THEN 1 mark for <u>IS_NUM</u> 6 DigitFlag ← <u>TRUE</u> ELSE LetterFlag ← <u>TRUE</u> 1 mark for both variables being set to TRUE 7 IF <u>NOT</u>(LetterFlag AND DigitFlag) THEN 1 mark for <u>NOT</u>	7	

4. M/J 26/P23/Q3/a,b,c

3(a)	The initial ANALYSIS stage is carried out by studying in detail the current system. A design for the proposed new system is produced. Following the design and coding stages, it is decided to install the completed new software initially in only two of the shops. BETA testing is carried out. The final stage of the project uses ACCEPTANCE testing to test all programs on the company's hardware. After successful testing, the company approves the new system. The company then installs the new software in all shops. In the first week of its use, unexpected bugs are reported. CORRECTIVE maintenance has to be carried out.	4	
3(b)	Team A MP1 Rapid Application Development /RAD // Iterative MP2 Large project can be broken down into smaller steps MP3 All stakeholders/users are involved (at all stages) MP4 There is constant discussion between the client (and Team A) MP5 Prioritises the client needs and end-user experience MP6 A <u>prototype</u> is available at an early stage MP7 More readily allows for requirement changes to be made MP8 Minimal planning is required MP9 Programs/modules can be developed in parallel Max 2 Team B MP10 Waterfall (method/model) MP11 Next stage does not start until the current stage is successfully completed MP12 More predictable for budget MP13 More predictable for timeline/completion MP14 All requirements are identified before coding MP15 Late changes cannot be accommodated Max 2	4	Reject 'early version'.
3(c)	MP1 Production of flowcharts // structure <u>charts</u> // STDs // Structured English // E-R diagrams MP2 Consider data storage (hardware / database / files) MP3 Design inputs screens // Design screen outputs // reports to be produced MP4 Select (Programming) language to be used MP5 Produce <u>pseudocode algorithms</u> / data structures / procedures/functions/modules are produced / Identifier table MP6 Produce a testing strategy statement // (detailed) test plan MP7 Consider – is there relevant code available from program libraries? Max 3	3	Reject 'program structure'.

6. O/N 25/P21/Q1/b

1(b)	Test description	Test method	3
	carried out as soon as a program module has been coded	alpha	
	carried out as program modules are being combined	integration	
	completed by the developers without referring to the code	black-box	
	completed by the customer	acceptance / beta	
One mark per row (2 to 4)			

7. O/N 25/P21/Q7/a,b

7(a)	One mark for each: 1 Label A1 X1 added on line from S1 to state labelled S2 2 States S3, S4 and S5 labelled 3 Line from S2 to S5 with event label A4 X4 4 Lines from S3 with correct event labels including 'loop' label 5 Lines from S4 with correct event labels including 'loop'	5
------	--	----------

7(b)	One mark per point: 1 All boxes correctly labelled 2 Iteration arrow 3 Parameter to Restart 4 Return value from Modify and parameter to Final 5 BYREF parameter to Update	5
------	---	---

8. O/N 25/P21/Q8/c

8(c)	Award marks for reference to following design features: Change to existing Record: 1 Store date due back as a new data item / in the loan record 2 Store date of loan and the loan length category as new data items / in the loan record Max 1 New Record 3 New Records with Category (and Loan length field) Change to Program: 4 (Use the loan length category of a book to) calculate the date due date 5 Compare due date with today's date (to check if overdue / to calculate fine) 6 Map loan length category to length of loan / date due back Max 1 Max 2 marks	2
------	--	---

9. O/N 25/P22/Q5/a,b

5(a)	Activity	Name of life cycle stage	4
	a structure chart is produced	Design	
	a program is modified to allow it to run on new hardware	Maintenance	
	the programmer identifies the customer's requirements	Analysis	
	an Integrated Development Environment (IDE) provides context-sensitive help such as 'auto-complete'	Coding	
One mark per row			
5(b)	Acceptance testing		1

10. O/N 25/P22/Q7/a,b

7(a)	Example explanation: The arrow means that Convert repeatedly calls Analyse then Update and then Sort (in that order)¹⁰ MP1 Reference to 'repetition'... MP2 Naming all four modules correctly and the sequence is stated or implied	2
7(b)	MP1 <u>PROCEDURE Analyse (Count : INTEGER)</u> <u>FUNCTION Update (Key : STRING) RETURNS INTEGER</u> MP2 MP3 MP4 <u>PROCEDURE Sort (BYREF P2 : BOOLEAN)</u> Mark as follows: One mark per underlined part	4

11. O/N 25/P22/Q8/d

8(d)	Additional date(s) solution MP1 Store the return date of each loan // the start date <u>and</u> the loan period MP2 Algorithm will do a calculation involving the MP1 data item(s) OR The student email address solution MP3 Store the student <u>email address</u> MP4 In order to send the email // the student is alerted to return the book Max 2	2
------	---	----------

12. O/N 25/P23/Q5/a

5(a)

5

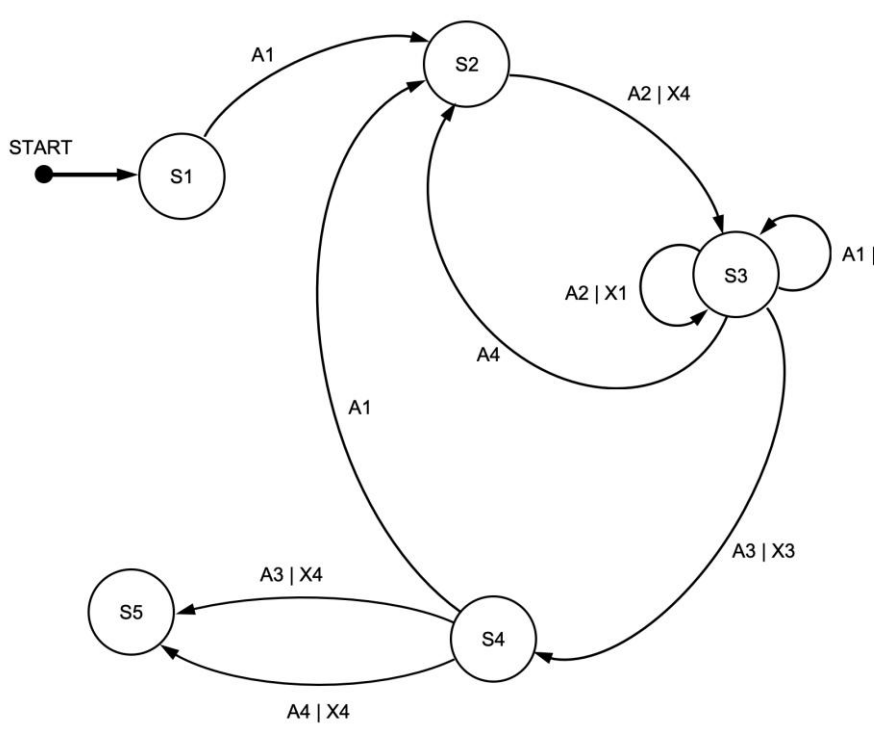
Index	CaseVar	Count	Num [1]	Num [2]	Num [3]	Num [4]
1		0	1	2	5	3
	1					
2		1	2			Zone 1
3						
	5					
		20				
4						
	3					
7		19				6
4						
	6					
		20				
1						
	2					
3		22	3			
4						
Zone 2	Zone 3	Zone 4	Zone 5			

Zones 1 to 4 : One mark for each set of values as shown
Zone 5: Num[4] set to 6 when Index is 7
and
Num[1] set to 3 when index is 3 for **second** time

13. O/N 25/P23/Q6/a

6(a)	One mark per point:	2
	1 Create a list of numbers that should generate all of the possible check digit values	
	2 and check that each / the / a generated value is as expected	

14. O/N 25/P23/Q7/a,b

7(a)	 One mark for each: 1 S2 to S5 labelled 2 Label A1 added to event from S1 to S2 3 Event lines from S2 to S3 and S3 to S2 correctly labelled 4 Events lines from S3 to S3 and event line from S3 to S4 all labelled correctly 5 Event lines from S4 to S5 correctly and event line from S4 to S2 labelled correctly Max 4 for any additional events/lines	5
------	---	----------

7(b)	<pre> graph TD Setup{Setup} --> Restart[Restart] Setup --> Confirm[Confirm] Restart --> Update[Update] Restart --> Modify[Modify] Update --> Restart Modify --> Restart </pre> One mark for each: 1 All boxes correctly labelled and connected in correct hierarchy 2 BYREF parameter to Restart, parameter to Confirm and return 3 Diamond and loop symbol 4 Parameter to Update and return from Modify	4
------	---	---

15. M/J 25/P21/Q2/a,b,c

2(a)(i)	MP1 The two decision boxes are in the wrong order // The first decision symbol is wrong MP2 The first decision should check for values greater than or equal to 2000 Max 1	1
2(a)(ii)	Explaining how to correct the flowchart. MP1 Swap around the two decision boxes MP2 The first decision box should check for values greater than or equal to 2000 and the second decision box should check for values above 4000 Max 1	1
2(b)(i)	2000 / 4000 / 10 / 100	1
2(b)(ii)	MP1 The value cannot be changed // avoids being different in two places MP2 Makes the program easier to understand MP3 Avoids repeatedly writing the same value throughout the program MP4 A change to the value requires a change in only one statement Max 2	2

2(b)(iii)	MP1 (The code is) tried and tested so error free MP2 Provides functionality / code that the programmer may find difficult to code themselves Max 1	1
2(c)	MP1 Syntax (error) MP2 Rules of programming language // the language grammar was not followed MP3 Run-time (error) MP4 The program performs an illegal operation Mark as follows: One mark for naming each type of error and one mark for the description	4

16. M/J 25/P21/Q4/b

4(b)

6

I	Key	J	Chars [J]	Chars			
				[1]	[2]	[3]	[4]
2				'D'	'T'	'H'	'R'
	'T'	1	'D'				
3							
	'H'	2	'T'			'T'	
		1	'D'				
					'H'		
4							
	'R'	3	'T'				'T'
		2	'H'			('R')	
						'R'	
5							

Chars array final values

'D'	'H'	'R'	'T'
-----	-----	-----	-----

Mark as follows:

MP1, MP2, MP3, MP4, MP5

1 mark per enclosure

17. M/J 25/P21/Q6/a

6(a)

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted
boundary/ extreme/ normal	0 / 1	data item is accepted
boundary/ extreme/ normal	59 / 60	data item is accepted
abnormal	>= 61	data item is rejected
normal	15	data item is accepted

Mark as follows:
1 mark for each row with (Type and Value and Outcome)

4

18. M/J 25/P21/Q7/b(ii)

7(b)(ii)	MP1 Check if the text file <code>loyalty.txt</code> exists / is empty MP2 If file does not exist it must be created (using write mode) MP3 If the file has to be created / is empty then set the first <code>CustomerID</code> to 100001 MP4 Write "100001,0" to <code>loyalty.txt</code> (using write mode) MP5 For all but the first customer (to be added to the empty file) use the existing code / module Max 4	4
----------	---	----------

19. M/J 25/P22/Q1/a,b

1(a)(i)	One mark for Choice of program development cycle depends on the approach/method required to produce the program Or One mark for a factor that would influence the choice of program development cycle, e.g. • Rapid development of program/prototypes required • Need for prototypes (at an early stage) • Complexity of problem • Skills / experience of development team / programmer(s) • Budget / Time / Resources available • Size of development team • Developer / programmer can return to earlier stages / any stage • Avoidance of repeating previous stages in development of program • How much involvement clients will have • Allows the (program) requirements to be changed during program development • (Program) requirements agreed at start of development of program Max 1	1
1(a)(ii)	Which programming language would best be suited to control the production line // Which programming language would best suit the problem being solved	1
1(b)(i)	Examples include: • Change to (production line) requirements • New technology / hardware available (to control production line) • Changes made to library modules used • Change in relevant legislation Max 3	3
1(b)(ii)	Perfective // Corrective	1

20. M/J 25/P22/Q2/b

2(b)(i)	One mark for each of set test values Hours worked between 1 and 40 inclusive Sales value ≤ 2000 Bonus Pay: 0 Hours worked above 40 Sales value ≤ 2000 Bonus Pay: 10 Hours worked between 1 and 40 inclusive Sales value > 2000 Bonus Pay: 50 Hours worked above 40 Sales value above 2000 Bonus Pay: 100 max 2	2
2(b)(ii)	One mark per point - Simple modules are written to replace each of the unfinished modules. - Each simple module will return an expected value / will output a message to show it has been called.	2
2(b)(iii)	One mark for naming type of error and one mark for corresponding description Type of error: Logic (error) Description: Where the program does not behave as expected / Does not give expected result / An error in the logic of the algorithm Or Type of error: Run-time (error) Description: The program performs an illegal operation Max 2	2

21. M/J 25/P22/Q5/a

5(a)

Start

standby mode

detect mode

active mode

sequence complete

(turn) on

(turn) off

(turn) off

movement detected

20 second elapsed

time saved

Mark as follows:

1 Mark: The three new states drawn and labelled.

1 Mark: Two of the events drawn with labels connecting correct states and correct line direction

1 Mark: Four of the events drawn with labels connecting correct states and correct line direction.

1 Mark: All and only six events drawn with labels connecting correct states and correct line direction.

4

22. M/J 25/P22/Q6/a

6(a)(i)

Value	Start	Unused	New	Last	Current
1043	2	8	8	-1	2
				2	3
				3	1
				1	7
				7	4

Region 1

Region 2

Region 3

Region 4

Award 1 mark per region

4

6(a)(ii)

	Data		Pointer
1	1018	1	7
2	1007	2	3
3	1010	3	1
4	1056	4	6
5	1092	5	-1
6	1062	6	5
7	1034	7	8
8	1043	8	4
9	0	9	10
10	0	10	-1

3

Mark as follows:

1 mark for global array `Data` row 8 containing the value 1043

1 mark for global array `Pointer` row 7 containing the value 8 and row 8 containing the value 4

1 mark for all other rows in both arrays

23. M/J 25/P23/Q4/b

4(b)	MP1 Going through the program a line / statement at a time // doing a dry run // single-stepping (at run-time with an IDE) // Peer testing/review is carried out MP2 Creating a trace table MP3 Draw up test data // draw up list of inputs with expected output // boundary, normal, abnormal data is used MP4 Errors will be indicated when a variable / output gives an unexpected value MP5 Faults in the logic of the program can be detected // Errors may be indicated by an unexpected path through the program Max 3	3
------	---	---

24. M/J 25/P23/Q6/b,c

6(b)	Sub_Part1A: PROCEDURE Sub Part1A(R: INTEGER) 1 Mark Sub_Part2A: FUNCTION Sub Part2A(T : REAL) RETURNS BOOLEAN 2 Marks Sub_Part3A: PROCEDURE Sub Part3A(V: INTEGER, W: BOOLEAN, BYREF U : STRING) 2 Marks	5						
6(c)	<table><tr><th>Symbol</th><th>Explanation</th></tr><tr><td></td><td>The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)</td></tr><tr><td></td><td>The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A</td></tr></table> MP1 Shape1: reference to <u>selection</u> MP2 Shape2: reference to <u>repetition</u> // iteration MP3 Description (x2) including correct use of all module names in both parts	Symbol	Explanation		The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)		The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A	3
Symbol	Explanation							
	The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)							
	The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A							

25. O/N 24/P21/Q1/a

1(a)	One mark per point: Error: The calculation of TaxPayable is incorrect Correction: TaxPayable ← (ItemCost * TaxRate) / 100	2
------	---	----------

26. O/N 24/P21/Q3/c

3(c)	Two loop solution One mark per point: 1 Loop until given line number (-1) / <parameter> (-1) (lines have been read) 2 ... if end of file is reached then return FALSE 3 Loop for three lines // Read three lines // Repeat of 4 and 5 for three lines 4 Read a line and output it 5 ... if end of file is reached then return FALSE 6 After outputting the (required) lines return TRUE 7 Ordering of lines no longer needed Max 4 marks Alternative solution One loop solution mark scheme that reads to given line number + 2 One mark per point 1 Loop until given line number + 2 / <parameter> + 2 (lines have been read) 2 OR end of file is reached 3 Return FALSE if EOF reached 4 Read a line from the file in the loop 5 Continue to order the last three lines read in the loop // Steps 4 to 6 stay the same 6 (If FALSE has not been returned) output the required three lines (in the correct order) 7 ... and Return TRUE Max 4 marks	4
3(a)	One mark per emboldened part: 1 Open the file (Result.txt) in read mode 2 Loop until EOF(Result.txt) // EOF // end of file 3 Read a / the / next line (from the file) and store in ThisLine 4 Assign LineY to LineX 5 Assign LineZ to LineY 6 Assign ThisLine to LineZ 7 After the loop, close the file (Result.txt) 8 Output LineX, LineY, LineZ	4

27. O/N 24/P21/Q4/a

4(a)	One mark per point: Type: Run-time error Cause: A divide by zero operation is attempted // Attempt to convert a non-numeric string to a number	2
------	---	----------

28. O/N 24/P21/Q5/a,b

5(a)	Activity		Name of life cycle stage	5
	The walkthrough method is used.		Testing	
	An algorithm is implemented in a programming language.		Coding	
	The client is interviewed about problems with the current system.		Analysis	
	The program is modified to run on new hardware.		Maintenance	
	Records and file structures are defined.		Design	
	One mark per row			

5(b)(i)	One mark per row			4
	Type of test data	Test data value	Expected result	
	Abnormal	12 (< 23)	FALSE	
	Abnormal / Boundary / Extreme	23	FALSE	
	Boundary / Extreme	24	TRUE	
	Boundary	25	TRUE	
	Boundary	36	TRUE	
	Boundary /Extreme	37	TRUE	
	Abnormal / Boundary / Extreme	38	FALSE	
	Abnormal	99 (> 38)	FALSE	
Max 4 marks				

5(b)(ii)	Integration (testing)	1
----------	-----------------------	---

29. O/N 24/P21/Q7/a,b

7(a)	One mark per point: 1 All module identified in correct hierarchy 2 Parameters to both Sync and Activate and return values 3 Parameters to both Reset and Enable 4 Parameters to Initialise 5 Decision diamond	5
7(b)	Explanation: Means that Sync repeatedly calls Reset and (then) Enable One mark for each point: MP1: reference to iteration MP2: naming all three modules correctly including correct sequence of calls	2

30. O/N 24/P22/Q1/a

1(a)	(Corrective) Maintenance	1
------	--------------------------	----------

31. O/N 24/P22/Q5/a,b

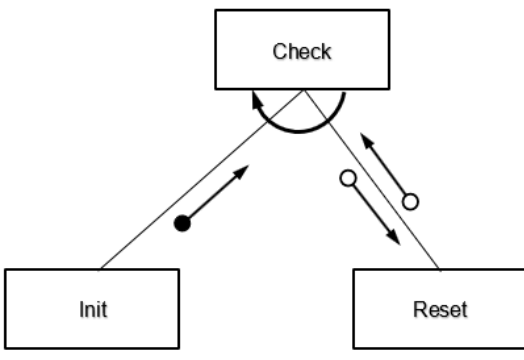
5(a)	MP1 <code>Count ← INT(100 / Number)</code> Number could be zero (giving a divide by zero) MP2 <code>Index ← Data[Number]</code> Potential error: Value <code>Number</code> could be outside the range of array indices MP3 <code>ReturnValue ← TO_UPPER(RIGHT(Label, Count))</code> Potential Error: Number to extract may be too big / negative / out of range for use in the <code>RIGHT</code> function // <code>Label</code> has insufficient characters MP4 <code>RETURN RetVal</code> Potential Error: There is no value to be returned // there is no variable named <code>RetVal</code> Mark as follows: 1 mark for each statement and description Max 3 marks	3
5(b)	MP1 Construct: A (pre/post) <u>conditional</u> loop MP2 Explanation: The terminating condition is never satisfied	2

32. O/N 24/P22/Q6/b

6(b)	Answers include: MP1 For each filling, create a list of <u>acceptable</u> / <u>incompatible</u> fillings/indexes MP2 When selecting the second filling, (as well as checking for an unused element) <u>check</u> that the filling / index is / is not on the list ALTERNATIVE: MP1 Create a list of 'good' combinations MP2 <u>Randomly select</u> from this list	2
------	---	----------

33. O/N 24/P22/Q7/c,d

7(c)	MP1 Data structures // data dictionary // identifier table(s) // validation rules MP2 Data-flow diagram // state-transition diagram MP3 User interface // Format for the email MP4 Testing method / Test plan / Test data / Trace tables MP5 Choice of email protocol to be used // Programming language to be used // Development environment MP6 Use of library routines // program to send the email Max 3 marks	3
------	---	----------

7(d)	 MP1 Three boxes correctly labelled and correct hierarchy MP2 Parameter and return values MP3 Iteration arrow	3
------	--	----------

34. O/N 24/P23/Q5/a,b

5(a)	<table><tr><th>Activity</th><th>Name of life cycle stage</th></tr><tr><td>A compiler is be used.</td><td>Coding</td></tr><tr><td>A program that has been released for general use is modified</td><td>Maintenance</td></tr><tr><td>The dry run method is used.</td><td>Testing</td></tr><tr><td>The program structure is specified.</td><td>Design</td></tr></table> One mark per row	Activity	Name of life cycle stage	A compiler is be used.	Coding	A program that has been released for general use is modified	Maintenance	The dry run method is used.	Testing	The program structure is specified.	Design	4
Activity	Name of life cycle stage											
A compiler is be used.	Coding											
A program that has been released for general use is modified	Maintenance											
The dry run method is used.	Testing											
The program structure is specified.	Design											
5(b)	One mark for method – two marks for the description MP1 Stub testing MP2 The modules <u>Test A()</u> and <u>Test B()</u> are replaced by dummy modules MP3 ... which return a known result // An output statement is displayed when called (to check it works) // gives expected output	3										

35. O/N 24/P23/Q7/a

7(a)	One mark per answer: MP1 Analysis MP2 Named document, examples include: • A <u>requirement specification</u> • Definition of System Objectives • List of problems with existing system • Survey results • Feasibility study • Interview notes Max 2 marks	2
------	---	----------

36. M/J 24/P21/Q5/a,b

5(a)	<table><tr><th>Count</th><th>C</th><th>L</th><th>Index</th><th>Result</th><th>Data[1]</th><th>Data[2]</th><th>Data[3]</th></tr><tr><td></td><td></td><td>1</td><td></td><td>"*****"</td><td>"aaaaaaa"</td><td>"bbbbbbb"</td><td>"ccccccc"</td></tr><tr><td>1</td><td>'X'</td><td>3</td><td>1</td><td>"AAAAA A"</td><td>"AAA"</td><td></td><td></td></tr><tr><td>3</td><td>'Y'</td><td>2</td><td>2</td><td>"bbbbb b"</td><td></td><td>"bb"</td><td></td></tr><tr><td>5</td><td>'W'</td><td>4</td><td>3</td><td></td><td></td><td></td><td>"bbbb"</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> One mark per zone.	Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]			1		"*****"	"aaaaaaa"	"bbbbbbb"	"ccccccc"	1	'X'	3	1	"AAAAA A"	"AAA"			3	'Y'	2	2	"bbbbb b"		"bb"		5	'W'	4	3				"bbbb"																																																	6
Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]																																																																																			
		1		"*****"	"aaaaaaa"	"bbbbbbb"	"ccccccc"																																																																																			
1	'X'	3	1	"AAAAA A"	"AAA"																																																																																					
3	'Y'	2	2	"bbbbb b"		"bb"																																																																																				
5	'W'	4	3				"bbbb"																																																																																			
5(b)(i)	<u>OTHERWISE : CALL Error(C)</u>	1																																																																																								
5(b)(ii)	After the 'z' clause in the CASE construct // before the ENDCASE	1																																																																																								

37. M/J 24/P21/Q6/b

6(b)	1 mark for statement of problem: Problem: - The function will return an incorrect value // the test will fail 1 mark for solution: Solution: - Round the calculated values (to a known number of decimal places) - Define a threshold below which any difference can be ignored	2
6(a)	<pre> FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR___ (Len2 = Len1 + Len3) OR___ (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: 1 Calculate the square of the length of at least one side 2 Calculation of all three lengths squared correctly 3 One correct comparison of square of three lengths 4 All three comparisons... 5 combined using logical operators / nested IF // completely correct selection 6 Return result correctly in both cases	6

38. M/J 24/P21/Q7/b

7(b)(i)	Means that Update calls (one of) either Sub-A, Sub-B or Sub-C One mark for each point: - reference to selection / decision / if - naming all four modules correctly	2
7(b)(ii)	<u>PROCEDURE Sub-A (Name : STRING, BYREF P2 : BOOLEAN)</u> <u>FUNCTION Sub-B (P1 : REAL) RETURNS REAL</u> One mark per underlined part in each case	4

39. M/J 24/P22/Q5/a(i),b

5(a)(i)	One mark per error: Syntax: 1. <code>NEXT Index</code> (should be <code>ENDWHILE</code>) 2. <code>'&'</code> used to concatenate an integer (in <code>OUTPUT</code> statement) Other: 3. <code>Accesses element outside range</code> // <code>Accesses element 0</code>	3
5(b)	Run-time	1

40. M/J 24/P22/Q7/b

7(b)(i)	<table border="1"> <thead> <tr> <th>Input</th><th>Output</th><th>Next state</th></tr> </thead> <tbody> <tr> <td></td><td></td><td>S1</td></tr> <tr> <td>Input-A</td><td>none</td><td>S3</td></tr> <tr> <td>Input-A</td><td>Output-W</td><td>S3</td></tr> <tr> <td>Input-B</td><td>none</td><td>S2</td></tr> <tr> <td>Input-B</td><td>none</td><td>S5</td></tr> <tr> <td>Input-A</td><td>none</td><td>S2</td></tr> <tr> <td>Input-A</td><td>Output-X</td><td>S4</td></tr> </tbody> </table> One mark per row 3 to 7	Input	Output	Next state			S1	Input-A	none	S3	Input-A	Output-W	S3	Input-B	none	S2	Input-B	none	S5	Input-A	none	S2	Input-A	Output-X	S4	5
Input	Output	Next state																								
		S1																								
Input-A	none	S3																								
Input-A	Output-W	S3																								
Input-B	none	S2																								
Input-B	none	S5																								
Input-A	none	S2																								
Input-A	Output-X	S4																								
7(b)(ii)	Input-B, Input-A	1																								

41. M/J 24/P23/Q6/b(ii)

6(b)(ii)	The progress display will more accurately show the progress of the task	1
----------	---	----------

42. M/J 24/P23/Q7/a(i),b

7(a)(i)	Mod-B() and Mod-E()	1
---------	---------------------	----------

7(b)	One mark per bullet: 1 All modules correctly labelled and interconnected 2 Parameters between Mod-A and Mod-B and return value from Mod-B 3 Parameters between Mod-A and Mod-C 4 Diamond applied to Mod-B only 5 Iteration arrow applied to Mod-C only 6 All parameters at lower level and return value	6
------	---	---

43. O/N 23/P21/Q5/a,b

5(a)	MP1 Num > Min should be Num < Min MP2 Count & 1 should be Count + 1 MP3 NextInput ← "END" should be NextInput = "END"	3
5(b)(i)	MP1 If all the numeric input values are greater than 999 // If there are no numeric values in the sequence MP2 then the minimum will be given as <u>999</u> (and not one of the input values)	2
5(b)(ii)	Many possible correct answers, for example: MP1 Mixture non-numeric and numeric with 3 or 4 values - with all numerics greater than 999 Examples: 1325, DOG, 7868, 7615 // SNAKE, 3478, SPIDER MP2 Final value: END	2

44. O/N 23/P21/Q7

7	MP1 S2 labelled MP2 S3, S4 and S5 added MP3 Line from S1 to S3 MP4 All three lines between S2 and S5 (including S5 to S5) MP5 Line from S5 to S3 AND from S3 to S4	5
---	---	---

45. O/N 23/P21/Q8/c

8(c)(i)	MP1 A message cannot contain a zero-length data field MP2 ... so a blank line cannot be sent // there is no way to send a blank line	2
8(c)(ii)	MP1 Append a (special) character to the start of the message text MP2 interpret the new field data as a blank line ALTERNATIVE MP1 Change the message protocol and use an additional field to act as an indicator MP2 Interpret the new field data	2

46. O/N 23/P22/Q4/b

4(b)	One mark per set, including stated purpose. Max 3 marks Example answers: 1 data set with (only) odd values, terminated with 99 2 data set with (only) even values, terminated with 99 3 data sets with same number of odd and even values, terminated with 99 4 data sets with all even / all odd with just one odd/even value, terminated with 99 5 data sets with no values just final 99 6 data sets without (terminating) 99 // missing or incorrectly placed 99	3
------	---	---

47. O/N 23/P22/Q5**5**

Index	Value	Count	Mix[1]	Mix[2]	Mix[3]	Mix[4]
3		0	1	3	4	2
	4				3	
4		1				
	2					1
2		2				
	3			2		
3		3				
	3				2	
3		4				
	2				1	
2		5				
						10

6**One mark for:**

- Initialisation row
- Each iteration of Count 1 to 4 with correct Index, Count and array Mix as shown
- Final iteration, Count = 5, correct Index, Count and array Mix as shown plus Mix[4] assignment

48. O/N 23/P22/Q7/a,b

7(a)	One mark per point: 1 Module names 2 Parameters with labels to Module-X and between Module-X and Reset 3 Parameter (with label) to Module-Z and return from Module-Z 4 Parameters with labels to Module-Y and Restore and return values from Module-Y	4
7(b)	Means that Module-A calls either one of Module-X, Module-Y or Module-Z (which one is called is decided at runtime). One mark for reference to selection One mark for naming all four modules correctly	2

49. O/N 23/P23/Q1/d

1(d)	MP1 Type: Adaptive MP2 Reason: The (user) requirement(s) changes // to accommodate legislative changes	2
------	--	---

50. O/N 23/P23/Q5

5

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]
2		0	4	2	3	5
	2	2				
				5		
3						
	3	5				
					8	
4						
	5	10				
						9
2						
	5	15				
				13		
2						
	13	28				
				21		
2						
			56			

6

MP1 Row 1 (initialisation)

Each iteration (1 – 5):

MP2 1 – Total 2

MP3 2 – Total 5

MP4 3 – Total 10

MP5 4 – Total 15

MP6 5 – Total 28 **and** final Mix[1] = 56

51. O/N 23/P23/Q7/a

7(a)	MP1 iteration / looping MP2 naming all four modules correctly in the correct sequence //_e.g. Module-A repeatedly calls Sub-Y1, then SubY2 then Sub-9	2
------	---	---

52. O/N 23/P23/Q8/b,d

8(b)	Max 3 marks Problem: • If the file being sent contains a line of the string "****" • then the file being written by <code>ReceiveFile()</code> will end at this point // subsequent file lines will be lost Solution: • Read the file (at the sending end) to find the number of lines it contains • Send an initial message which defines the number of lines in the file ALTERNATIVE SOLUTION: • (Transmitter program) chooses a different terminator string / character that doesn't occur in the file • Transmitter program sends the terminator string / character before first line of file / before the transfer begins	3
8(d)	Note: Max 3 marks (from either limitation or modification list) Limitation: 1 <code>GetData()</code> does not return a value until a message has been received 2 So once a message has been sent the user has to wait for a reply // chat is half-duplex Modification: 3 If no response allow the receiver to exit chat at any time ... 4 <code>GetData()</code> should immediately return a suitable message // set a time limit 5 ... which <code>Chat()</code> can detect and respond by allowing the conversation to continue	3

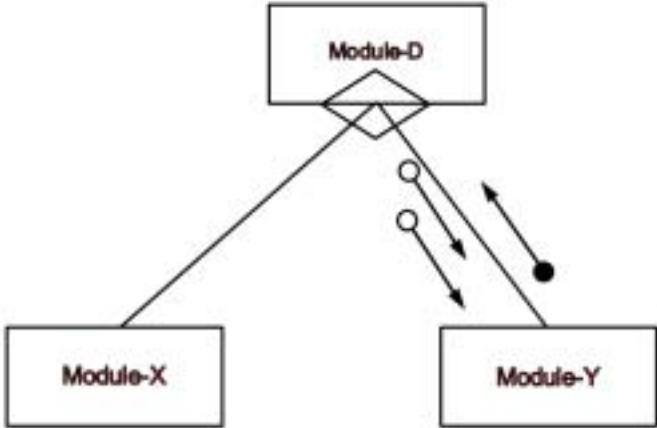
8(c)	<pre> PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE </pre> Conditional loop MP1 Conditional loop MP2 Test for terminator in both cases MP3 Use <code>GetData()</code> to get the data from the message MP4 <code>OUTPUT</code> the data in a loop MP5 <code>INPUT</code> the data reply MP6 'Attempted' use of <code>Transmit</code> to send it in a loop MP7 Correct formation of parameters to <code>Transmit()</code>	7
------	---	---

53. M/J 23/P21/Q5/a,b

5(a)(i)	Reasons include: 1 No working software until late in the life cycle so slower to market than competitors // Does not allow the creation of early versions/prototypes (which can be updated later) 2 More difficult/slower to cope with changes to the requirements // website slower to be updated to reflect new requirements 3 Needs high involvement/feedback of the stake holders /customer / client One mark per point Max 2 marks	2
5(a)(ii)	Iterative / Rapid Application Development / RAD	1

5(b)	One mark for Stage Stage: Beta testing Max 2 marks for Description Description: 1 Testing carried out by a small group of (potential) users 2 Users will check that the website/software works as required / works in the real world // User will identify errors in the website/software 3 Users will feedback (problems) / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	3
------	---	----------

54. M/J 23/P21/Q7/b

7(b)(i)	One mark per point: • Module-A calls the other three modules • The process is repeated	2
7(b)(ii)	 One mark per bullet point: • All rectangles correctly labelled and interconnected • All and only parameters as shown • Selection diamond	3

55. M/J 23/P21/Q8/c

8(c)(i)	Integration testing	1
8(c)(ii)	Two marks for the description: • A dummy/simple module is written to replace the module that does not work properly • The dummy/simple module will return an expected value // will output a message to show it has been called	2

56. M/J 23/P22/Q5/a,b,d

5(a)	One mark per point: - parameter / Num has been passed by reference // should have been passed by value - so when the value / ThisNum is modified (in procedure DisplaySqrt()) - the new value will be used in the loop (lines 40–43) // Num will be changed to modified value	3
5(b)	The rules of the language have not been broken // there are no syntax errors	1
5(d)	Answers include: - Change the statement into a comment - Change the statement to a string representing a literal value and assign it to a variable / output it Note: max 1 mark	1

57. M/J 23/P22/Q7/b

7(b)	<pre> graph LR START((START)) --> S1((S1)) S1 -- "Input PIN" --> S2((S2)) S2 -- "Re-input PIN Display error" --> S2 S2 -- "Too many tries Block Account" --> S3((S3)) S2 -- "Valid PIN Enable payment" --> S4((S4)) S2 -- "Cancel Re-prompt" --> S1 </pre> One mark for each: - Line from S1 to S2 and label - S2 loop label - S3 circle and label on incoming - S4 circle and label on incoming	4
------	---	----------

58. M/J 23/P22/Q8/c

8(c)	One mark for method: Method: Stub testing Two marks for description: • The modules <code>SuppExists()</code> and <code>CheckSupplier()</code> are replaced by dummy modules • ...which return a known result / contain an output statement to show they have been called	3
------	--	----------

59. M/J 23/P23/Q1/b

1(b)	One mark per point: • The (second dimension/index of the) array is declared from 1 to 50 but the loop runs from 0 to 49 • Line number: 10 / 100 / 101 / 102	2
------	---	----------

60. M/J 23/P23/Q5/a,b

5(a)	Max 3 marks Additional Information: MP1 The (program/source) code/specification MP2 test plan // inputs/test data <u>and</u> expected outputs Explanation: MP3 The structure / design / algorithm of the program of the program needs to be known MP4 so that all paths through the program can be tested	3
5(b)	Perfective	1

61. M/J 23/P23/Q7/b

7(b)	Max 3 marks MP1 All modules correctly labelled and interconnected. MP2 Correct parameters and return type to Module-X and Reset MP3 Correct parameters and return type to Module-Y and Replace MP4 Correct parameters and return type to Overlay and Module-Y	3
------	--	----------

62. M/J 23/P23/Q8/c

8(c)(i)	Max 2 marks MP1 Must 'calculate' the count before any item + description output / after the file is read once MP2 Lines to be output have to be <u>stored</u> ... MP3 The file has to be read twice	2
8(c)(ii)	One mark per point: MP1 Loop through the file calculating the count MP2 Save 'selected' items in <u>an array</u> MP3 (After all lines have been read), output the header lines / count MP4 Loop through <u>the array</u> to output each array element	3

8(b)	<pre> PROCEDURE Report_1(Supp : STRING) DECLARE Count : INTEGER DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode : STRING Count ← 0 OPENFILE "Stock.txt" FOR READ OUTPUT "Report for Supplier:" & Supp OUTPUT "" //Blank line as per example OUTPUT "Item Description" OUTPUT "" //Blank line as per example WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) ThisCode ← Mid(ThisLine, 5, 3) IF ThisCode = Supp THEN ThisItemNum ← LEFT(ThisLine, 4) ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7) OUTPUT ThisItem & " " & ThisDesc Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE "Stock.txt" OUTPUT "" //Blank line as per example OUTPUT "Number of items listed: ", Count ENDPROCEDURE </pre> MP1 Output report header (blank lines optional) – Must contain the parameter code MP2 Conditional loop until EOF("Stock.txt") MP3 Read a line from Stock.txt AND extract SupplierCode in a loop MP4 Test if SupplierCode = Supp then increment count (must have been Initialised) MP5 Extract AND output item and description in a loop MP6 Output the final line with count	6
------	--	---

63. O/N 22/P21/Q6/b

6(b)(i)	One mark for each area as outlined:	5																																																																																										
	<table><tr><th>Index</th><th>NextChar</th><th>Dots</th><th>Ats</th><th>Others</th><th>Valid</th></tr><tr><td></td><td></td><td>0</td><td>0</td><td>0</td><td>TRUE</td></tr><tr><td>1</td><td>'L'</td><td></td><td></td><td>1</td><td></td></tr><tr><td>2</td><td>'i'</td><td></td><td></td><td>2</td><td></td></tr><tr><td>3</td><td>'z'</td><td></td><td></td><td>3</td><td></td></tr><tr><td>4</td><td>'.'</td><td>1</td><td></td><td></td><td></td></tr><tr><td>5</td><td>'1'</td><td></td><td></td><td>4</td><td></td></tr><tr><td>6</td><td>'2'</td><td></td><td></td><td>5</td><td></td></tr><tr><td>7</td><td>'3'</td><td></td><td></td><td>6</td><td></td></tr><tr><td>8</td><td>'@'</td><td></td><td>1</td><td></td><td></td></tr><tr><td>9</td><td>'b'</td><td></td><td></td><td>7</td><td></td></tr><tr><td>10</td><td>'i'</td><td></td><td></td><td>8</td><td></td></tr><tr><td>11</td><td>'g'</td><td></td><td></td><td>9</td><td></td></tr><tr><td>12</td><td>'@'</td><td></td><td>2</td><td></td><td>FALSE</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Index	NextChar	Dots	Ats	Others	Valid			0	0	0	TRUE	1	'L'			1		2	'i'			2		3	'z'			3		4	'.'	1				5	'1'			4		6	'2'			5		7	'3'			6		8	'@'		1			9	'b'			7		10	'i'			8		11	'g'			9		12	'@'		2		FALSE							
Index	NextChar	Dots	Ats	Others	Valid																																																																																							
		0	0	0	TRUE																																																																																							
1	'L'			1																																																																																								
2	'i'			2																																																																																								
3	'z'			3																																																																																								
4	'.'	1																																																																																										
5	'1'			4																																																																																								
6	'2'			5																																																																																								
7	'3'			6																																																																																								
8	'@'		1																																																																																									
9	'b'			7																																																																																								
10	'i'			8																																																																																								
11	'g'			9																																																																																								
12	'@'		2		FALSE																																																																																							
6(b)(ii)	FALSE	1																																																																																										

64. O/N 22/P21/Q7/b

7(b)	Example string: "23/0" (Any divide by zero example) Reason: The result is infinity / cannot be represented / is undefined // will cause the program to crash	2
------	---	----------

65. O/N 22/P21/Q8/d

8(d)	One mark for each (Max 2): Examples: 1 Incorrect block structure. Missing keyword denoting part of block (for example <code>ENDPROCEDURE</code> , <code>ENDFUNCTION</code> , <code>ENDTYPE</code>) 2 Data type errors, for example, assigning an integer value to a string 3 Identifier used before it is declared 4 Incorrect parameter use	2
------	---	----------

66. O/N 22/P22/Q1/b

1(b)	One mark for test stage, one mark for each description point (Max 3 for Description) Test stage: Beta testing Description: 1 Testing carried out by a small group of (potential) users 2 Users will check that the software works as required / works in the real world / does not contain errors 3 Users will feedback problems / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	4
1(a)(i)	One mark for each point (Max 2): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	2
1(a)(ii)	One mark for each part: Term: Parameter(s) Use: to pass values / arguments to the procedure	2

67. O/N 22/P22/Q2/b

2(b)	One mark per row (row 2 to 5): <table border="1"> <thead> <tr> <th>Input</th><th>Output</th><th>Next state</th></tr> </thead> <tbody> <tr> <td></td><td></td><td>S1</td></tr> <tr> <td>Input-A</td><td>Output-X</td><td>S2</td></tr> <tr> <td>Input-A</td><td>(none)</td><td>S2</td></tr> <tr> <td>Input-B</td><td>Output-W</td><td>S3</td></tr> <tr> <td>Input-A</td><td>Output-W</td><td>S4</td></tr> </tbody> </table>	Input	Output	Next state			S1	Input-A	Output-X	S2	Input-A	(none)	S2	Input-B	Output-W	S3	Input-A	Output-W	S4	4
Input	Output	Next state																		
		S1																		
Input-A	Output-X	S2																		
Input-A	(none)	S2																		
Input-B	Output-W	S3																		
Input-A	Output-W	S4																		

68. O/N 22/P22/Q6/b

6(b)	One mark per row. Examples of invalid strings: 1 Non-numeric but not "End" // contains space or other non-numeric characters 2 Real number 3 Integer value out of range (i.e. ≤ 0) 4 Empty string 5 Correct word but wrong case e.g. "end" rather than "End" <table><tr><th>Input</th><th>Reason for choice</th></tr><tr><td>"Aardvark"</td><td>Non-numeric (and not "End")</td></tr><tr><td>"27.3"</td><td>Numeric but not an integer</td></tr><tr><td>"-3" // "0"</td><td>A non-positive integer</td></tr><tr><td>""</td><td>An empty string</td></tr></table>	Input	Reason for choice	"Aardvark"	Non-numeric (and not "End")	"27.3"	Numeric but not an integer	"-3" // "0"	A non-positive integer	""	An empty string	4
Input	Reason for choice											
"Aardvark"	Non-numeric (and not "End")											
"27.3"	Numeric but not an integer											
"-3" // "0"	A non-positive integer											
""	An empty string											

69. O/N 22/P23/Q1/b,c,d

1(b)(i)	Easier to manage/plan/cost // Clear deliverables produced at (end of) each stage	1
1(b)(ii)	One mark per point (Max 1): • The problem definition • Requirements specification // Client requirements • Documentation related to current system (e.g. ER diagram of current system, DFD of current system, feasibility study)	1
1(c)(i)	One mark per point (Max 1): • Modules are developed in parallel / as prototypes • Minimal / no detailed planning is carried out // Allows for changes to requirements • Flexible development process • Small incremental releases are made, each adding functionality • Used for time critical development • Client involved during (all stages) of development	1

1(c)(ii)	Examples include: Benefits: (Max 2 marks) • Quicker development possible / Multiple areas can be worked on at same time • Prototype produced (at early stage in process) • Easier to change requirements / quicker delivery of usable modules • Early review possible / closer cooperation between client and developers Drawback: (Max 1 mark) • Difficult to estimate cost / time to complete project • Documentation often omitted • Lack of client availability throughout life cycle // too easy for client to keep changing their mind	3
1(d)	One mark for each point (Max 2) Examples include: 1 Change to website requirements 2 New technologies available to host website // changes made to library modules used 3 Change in relevant legislation	2

70. O/N 22/P23/Q5/d

5(d)	One mark per point (Max 3) 1 The program is checked by creating a trace table / going through the program a line at a time 2to record/check variable (values) as they change 3 Error may be indicated when variable given an unexpected value 4 Error may be indicated by an unexpected path through the program // Faults in the logic of the program can be detected	3
------	--	----------

71. O/N 22/P23/Q6/a

6(a)	One mark per row	5																														
	Examples:																															
	<table><tr><th>Test number</th><th>Component weight</th><th>Min</th><th>Max</th><th>Expected return value</th></tr><tr><td>1</td><td>300</td><td>290</td><td>315</td><td>'A'</td></tr><tr><td>2</td><td>> 317</td><td>290</td><td>315</td><td>'R'</td></tr><tr><td>3</td><td>317</td><td>290</td><td>315</td><td>'C'</td></tr><tr><td>4</td><td>288</td><td>290</td><td>315</td><td>'C'</td></tr><tr><td>5</td><td>< 288</td><td>290</td><td>315</td><td>'R'</td></tr></table>	Test number	Component weight	Min	Max	Expected return value	1	300	290	315	'A'	2	> 317	290	315	'R'	3	317	290	315	'C'	4	288	290	315	'C'	5	< 288	290	315	'R'	
Test number	Component weight	Min	Max	Expected return value																												
1	300	290	315	'A'																												
2	> 317	290	315	'R'																												
3	317	290	315	'C'																												
4	288	290	315	'C'																												
5	< 288	290	315	'R'																												

72. O/N 21/P21/Q1/a

1(a)(i)	One from: - The program obeys the rules / grammar of the programming language used - The program will run // it can be compiled / interpreted - Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	1
1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: - An error in the algorithm / design of the solution - the program does not behave as expected / give the expected output. - Accept by example e.g. wrong arithmetic operator used / wrong loop count OR Type of error: Run-time error Description: - The program performs an illegal instruction / invalid operation - Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	2

73. O/N 21/P21/Q4/a,b

4(a)

Test	Test data value	Explanation	Expected Outcome
1	23	Normal Data	Data is accepted
2	0	Boundary Data	Data is accepted
3	40	Boundary Data	Data is accepted
4	>= 41	Abnormal Data	Data is rejected
5	<= -1	Abnormal Data	Data is rejected

One mark per row for rows 2 to 5.

4

4(b)	<pre> graph LR Start((Start)) --> Off((Heaters Off)) Off -- "Temp >= 10" --> Off Off -- "Temp < 10" --> On((Heaters On)) On -- "Temp <= 20" --> On On -- "Temp > 20" --> Off </pre> One mark for each label: • Temp < 10 (Heaters Off to Heaters On) • Temp > 20 (Heaters On to Heaters Off) • on BOTH loops (non-contradictory values)	3
------	--	---

74. O/N 21/P21/Q5/b

5(b)	Design	1
------	--------	---

75. O/N 21/P22/Q2/b

2(b)(i)	One mark for each: Life cycle method: Iterative // Rapid Application Development (RAD) Reason: Provides a working model / prototype at an early stage for the principal to approve / review	2
2(b)(ii)	Decisions will be made regarding: • Data structures • Algorithms / flowcharts / pseudocode • Program structure (modules) / use of library routines / module - team allocation • User interface // Web-page layout / content (for given scenario) • Testing method / plan • Choice of programming language / program environment Max 3	3

76. O/N 21/P22/Q6/c(i)

6(c)(i)	One mark for: • Parameter(s) need to be passed to the module to identify the type of search • Search algorithm is controlled by (global) variables / parameters Alternative: • The search algorithms from the original modules are included in the new module • The new module needs to return / store the four values (the results of the four searches)	2
---------	---	----------

77. O/N 21/P23/Q1/a

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	1
1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: • An error in the algorithm / design of the solution • the program does not behave as expected / give the expected output. • Accept by example e.g. wrong arithmetic operator used / wrong loop count OR Type of error: Run-time error Description: • The program performs an illegal instruction / invalid operation • Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	2

78. O/N 21/P23/Q4/a,b

4(a)	Test	Test data value	Explanation	Expected Outcome	4
	1	23	Normal Data	Data is accepted	
	2	0	Boundary Data	Data is accepted	
	3	40	Boundary Data	Data is accepted	
	4	>= 41	Abnormal Data	Data is rejected	
	5	<= -1	Abnormal Data	Data is rejected	
One mark per row for rows 2 to 5.					

4(b)	<pre> graph LR Start((Start)) --> Off((Heaters Off)) Off -- "Temp >= 10" --> Off Off -- "Temp < 10" --> On((Heaters On)) On -- "Temp <= 20" --> On On -- "Temp > 20" --> Off </pre> One mark for each label: • Temp < 10 (Heaters Off to Heaters On) • Temp > 20 (Heaters On to Heaters Off) • on BOTH loops (non-contradictory values)	3
------	--	----------

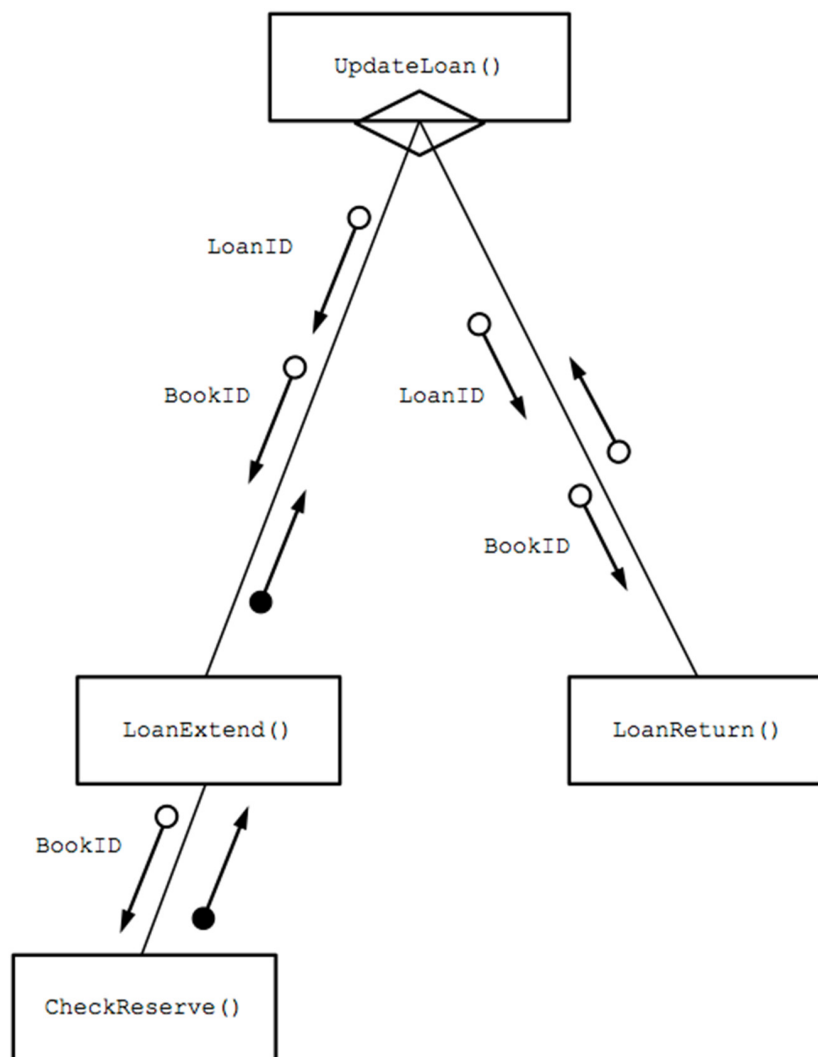
79. O/N 21/P23/Q5/b

5(b)	Design	1
------	--------	----------

80. M/J 21/P21/Q2/a

2(a)

5



One mark for:

- 1 All four boxes correctly labelled and positioned
- 2 Selection diamond (only on `UpdateLoan()` and no iteration arrows)
- 3 Parameters to and from `LoanExtend()`
- 4 Parameters to and from `CheckReserve()`
- 5 Parameters to and from `LoanReturn()`

4(c)(i)	A couple of solutions: <pre>24 ThisChar ← LCASE (MID (Name, Index, 1)</pre> ALTERNATIVE: <pre>31 NewName ← NewName & LCASE (ThisChar)</pre> Ignore line number	1
4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	2

82. M/J 21/P22/Q2/a

2(a)(i)	The number of transitions that result in a different state	3	4
	The number of transitions with associated outputs	2	
	The label that should replace 'X'	Start	
	The final or halting state	S3	
	One mark per row		
2(a)(ii)	Number of outputs: 1 Current state: S2	2	

83. M/J 21/P22/Q7/a,b

7(a)	Test string should contain: • At least one leading space (before the first word) • at least one trailing space (after the last word) • at least one instance of multiple spaces between words • At least one upper case character Mark as follows: One mark for one correct Two marks for three correct Three marks for all correct	3
------	---	----------

7(b)	One mark for each: • Dry run / produce a trace table / walk through the code • Add output statements to allow the code to be tracked • Insert a Breakpoint into the program // use single-stepping to execute instructions // monitor variables using a watch window • Try different test values to see which ones fail Note: Max 2 marks	2
------	---	----------

84. M/J 21/P23/Q2/a

2(a)	<pre> graph TD UpdateLoan([UpdateLoan()]) LoanExtend[LoanExtend()] CheckReserve[CheckReserve()] LoanReturn[LoanReturn()] UpdateLoan -- LoanID --> LoanExtend LoanExtend -- BookID --> CheckReserve CheckReserve --> LoanExtend UpdateLoan -- LoanID --> LoanReturn LoanReturn -- BookID --> UpdateLoan </pre> The diagram shows four activity boxes: <code>UpdateLoan()</code> at the top, <code>LoanExtend()</code> at the bottom left, <code>CheckReserve()</code> at the bottom left below <code>LoanExtend()</code>, and <code>LoanReturn()</code> at the bottom right. A selection diamond is located at the bottom of the <code>UpdateLoan()</code> box. Two paths emerge from the diamond: one to <code>LoanExtend()</code> labeled <code>LoanID</code>, and another to <code>LoanReturn()</code> labeled <code>LoanID</code>. From <code>LoanExtend()</code>, an arrow labeled <code>BookID</code> points to <code>CheckReserve()</code>, which then returns to <code>LoanExtend()</code> with an empty arrow. From <code>LoanReturn()</code>, an arrow labeled <code>BookID</code> points back to the selection diamond in <code>UpdateLoan()</code>. One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on <code>UpdateLoan()</code> and no iteration arrows) 3 Parameters to and from <code>LoanExtend()</code> 4 Parameters to and from <code>CheckReserve()</code> 5 Parameters to and from <code>LoanReturn()</code>	5
------	--	----------

85. M/J 21/P23/Q4/a,c

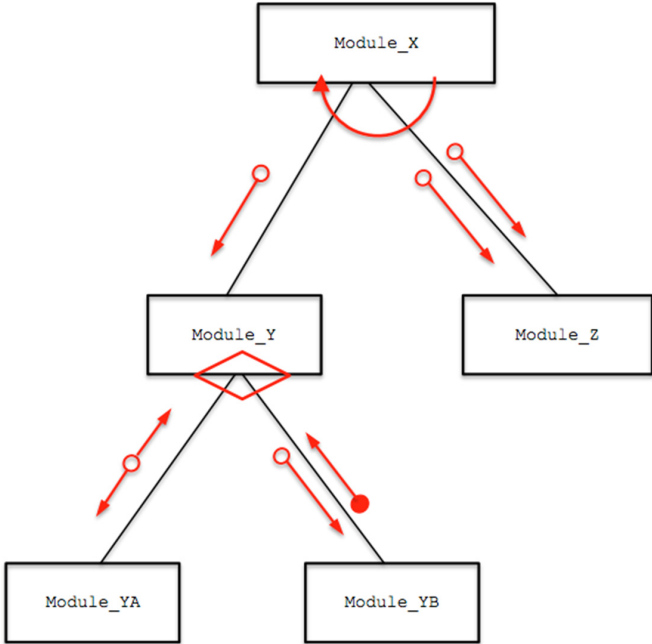
4(a)						5
	Name	Flag	Index	NewName	ThisChar	
	"VInVaVVcup"					
		TRUE	1	" "		
					'V'	
				"V"		
			2		'i'	
		FALSE		"VI"		
			3		'n'	
				"VIn"		
			4		'V'	
		TRUE		"VInV"		
			5		'a'	
		FALSE		"VInVA"		
			6		'V'	
		TRUE		"VInVAV"		
			7		'V'	
				"VInVAVV"		
			8		'C'	
		FALSE		"VInVAVVC"		
			9		'u'	
				"VInVAVVCu"		
			10		'p'	
				"VInVAVVCup"		
		FALSE	11	"VInVAVVCup"	'p'	

Mark as follows:

- One mark for each of columns 2 to 5 (condone missing final 11)
- One mark for final row all correct (including final 11)

4(c)(i)	A couple of solutions: <pre>24 ThisChar ← LCASE(MID(Name, Index, 1))</pre> ALTERNATIVE: <pre>31 NewName ← NewName & LCASE(ThisChar)</pre> Ignore line number	1
4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	2

86. 9608 M/J 21/P21/Q3/a,b

3(a)	 1 mark for each of: - Iteration arrow - Selection diamond - Both sets of parameters from Module_X - Parameter ByReference to Module_YA - Parameter (ByValue) and return Boolean from Module_YB	5
3(b)(i)	One mark for each statement: - it is a function - because it returns a value	2

3(b)(ii)	<u>PROCEDURE</u> Module <u>ZB</u> (BYVALUE ParX : REAL, BYREF ParZ : STRING) One mark for: • Procedure declaration • ParX : REAL and ParZ : STRING • ByRef for ParZ Condone missing BYVALUE for ParX	3
----------	---	----------

87. 9608 M/J 21/P21/Q4/c

4(c)(i)	One mark for Name, max 2 for Tasks (one per underlined term): Name: Design Tasks: To define the <u>data structures</u> and <u>algorithms</u> (of the solution) ALTERNATIVE Name: Analysis Tasks: Feasibility study // Problem definition / investigation // Requirement spec	3
4(c)(ii)	Coding / Implementation / Programming	1

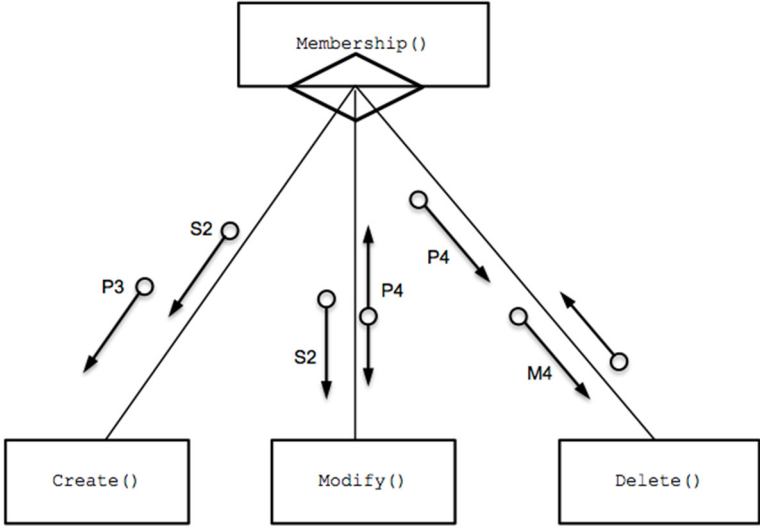
88. 9608 M/J 21/P21/Q5/b(i,ii)

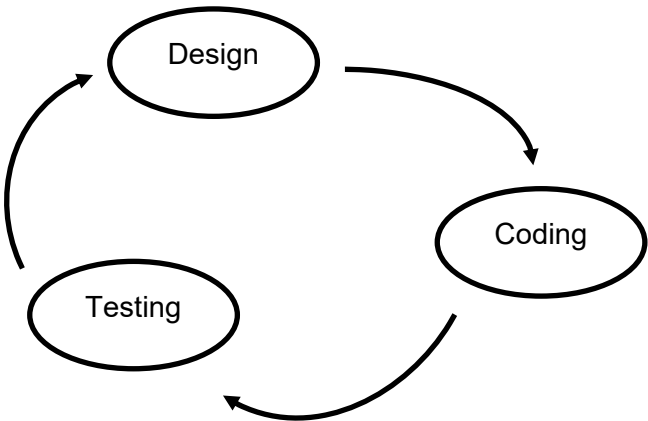
5(b)(i)	• Stub testing	1
5(b)(ii)	One mark for each: • A simplified version of <code>Status()</code> / a dummy function is written • that returns a typical / expected value.	2

89. 9608 M/J 21/P22/Q1/a

1(a)	One mark for each type and one mark for corresponding reason (not dependant) Type: Corrective Reason: Because the program does not function as intended / contains a bug Type: Adaptive Reason: Change the program due to a requirement / specification / legislative change Type: Perfective Reason: Improve the performance of the program / to enhance the program Type: Preventive Reason: Improve maintainability or reliability	4
------	---	----------

90. 9608 M/J 21/P22/Q3/a,b

3(a)(i)	One mark for each: Module... • Hierarchy / relationships • Selection • Repetition / Iteration • Sequence Note: Max 2 marks	2
3(a)(ii)	 One mark for each of: 1 Diagram with all boxes correctly labelled, positioned as shown 2 Selection diamond as shown 3 (P3 and S2) and (P4 and M4) (<i>Parameters to Create and Delete</i>) 4 S2 and P4 (double arrow) (<i>Parameters to Modify</i>) 5 Return parameter from Delete()	5

3(b)	 <pre> graph TD Design([Design]) --> Coding([Coding]) Coding --> Testing([Testing]) Testing --> Design </pre> One mark for: • Design, Coding and Testing (in sequence) • Arrows as shown // Alternative 'waterfall model' where each stage loops back to the previous stage	2
------	---	----------

91. 9608 M/J 21/P23/Q2/a,c

2(a)	• (A program fault is) when the program does not do what it is supposed to do / expected to do • ... under certain circumstances One mark per point or equivalent	2
2(c)	One mark per point: • white box • dry-run testing / use of trace table / walk through Max 1 mark	1

92. 9608 M/J 21/P23/Q4/a,b,c

4(a)(i)	<table><thead><tr><th>Result</th><th>Count</th><th>Index</th><th>NextChar</th></tr></thead><tbody><tr><td>0</td><td>1</td><td></td><td></td></tr><tr><td>1</td><td></td><td>1</td><td>'7'</td></tr><tr><td>2</td><td></td><td>2</td><td>'4'</td></tr><tr><td>3</td><td></td><td>3</td><td>'.'</td></tr><tr><td>4</td><td></td><td>4</td><td>'0'</td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>5</td><td>'.'</td></tr><tr><td></td><td>2</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td>5</td><td></td><td>6</td><td>'4'</td></tr><tr><td>6</td><td></td><td>7</td><td>'.'</td></tr><tr><td>7</td><td></td><td>8</td><td>'6'</td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>9</td><td>'.'</td></tr><tr><td></td><td>3</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td></td><td>10</td><td>'3'</td></tr><tr><td>-1</td><td></td><td>11</td><td>'x'</td></tr><tr><td>0</td><td></td><td>12</td><td>'2'</td></tr></tbody></table> Note: One mark per region indicated If no marks by zone then mark by column (max 3) Values in column 4 must be in quotes	Result	Count	Index	NextChar	0	1			1		1	'7'	2		2	'4'	3		3	'.'	4		4	'0'							5	'.'		2							5		6	'4'	6		7	'.'	7		8	'6'							9	'.'		3							8		10	'3'	-1		11	'x'	0		12	'2'	5
Result	Count	Index	NextChar																																																																															
0	1																																																																																	
1		1	'7'																																																																															
2		2	'4'																																																																															
3		3	'.'																																																																															
4		4	'0'																																																																															
		5	'.'																																																																															
	2																																																																																	
5		6	'4'																																																																															
6		7	'.'																																																																															
7		8	'6'																																																																															
		9	'.'																																																																															
	3																																																																																	
8		10	'3'																																																																															
-1		11	'x'																																																																															
0		12	'2'																																																																															
4(a)(ii)	0 (zero) Allow FT from final value in 'Result' column	1																																																																																

4(b)(i)	Final <code>IN ... ENDIF</code> errors: • Test needs to check <code>Count AND Result</code> • Returns <code>Result</code> instead of <code>Count</code> OR <code>FOR</code> loop error: • The loop continues after the illegal character is detected • So the error condition (<code>Result = -1</code>) can be lost	2
4(b)(ii)	• Change <code>RETURN Result</code> to <code>RETURN Count</code> • Change final <code>if</code> to <code>IF Count < 3 and Result <> -1</code> • Terminate the loop as soon as an illegal character is encountered Max 1 mark	1
4(c)	• Syntax error • Rules of the language are not followed OR • Run-time error • Program performs an illegal operation or enters an infinite loop	2

93. 9608 M/J 21/P23/Q5/b

5(b)	One mark for check plus one for corresponding test data example. Test data must be invalid to prove that the check is working. Telephone number: • Length check // Check that the telephone number string is at least six characters e.g. number of "127" OR • Check it is a number // Check that the telephone number string only contains characters from '0' to '9' e.g. number of "12A" Template: • Check it is a number in range 1 to 5 // Check that the template string only contains characters from '1' to '5' e.g. template of "127" OR • Check that there are enough characters in the <code>TelNum</code> string so that the template can be applied e.g. <code>Telnum = "123456"</code>, <code>Template = "66"</code>	4
------	---	----------

94. M/J 20/P21/Q2/a,b

2(a)	<pre> graph TD RentCar{RentCar()} PayByCard[PayByCard()] PayByAccount[PayByAccount()] RentCar -- HireCost --> PayByCard PayByCard --> RentCar RentCar -- HireCost --> PayByAccount RentCar -- AccountNumber --> PayByAccount RentCar -- AccountLimit --> PayByAccount PayByAccount -- CurrentBalance --> RentCar </pre> Mark as follows: 1 One mark for all three boxes correctly labelled 2 One mark for selection diamond 3 One mark for passing value and return Boolean from PayByCard() 4 One mark for passing Value, AccountNUmber and AccountLimit to PayByAccount () 5 One mark for passing CurrentBalance ByRef	5
2(b)(i)	Trace table shows: • 'A' is not treated as an upper case character (row 7) • NumUpper not incremented as expected • Incorrect final value for NumUpper (should be 1) Max 2	2
2(b)(ii)	One mark per point: • Line number: 33 • Correction: IF NextChar >= 'A' AND NextChar <= 'Z'	2

2(b)(iii)	CASE OF NextChar <pre> >= 'a' AND <= 'z' : NumLower ← NumLower + 1 > 'A' AND <= 'Z' : NumUpper ← NumUpper + 1 OTHERWISE NumNonAlpha ← NumNonAlpha + 1 </pre> ENDCASE One mark for CASE OF NextChar ... ENDCASE One mark for each remaining line Accept alternative range description. E.g. 'a' to 'z' Accept corrected version for the second range.	4
-----------	---	----------

95. M/J 20/P21/Q4/a,b,c

4(a)	A program fault is something that makes the program <u>not do what it is supposed to do</u> under <u>certain circumstances</u> One mark per underlined phrase or equivalent	2
4(b)	Answers include the use of: • Tried and tested (library) subroutines / code • Modular programming techniques (to break the problem down and make it easier to solve) • Good programming practice (formatting, sensible variable names, comments etc) • IDE features (parameter type-checking, auto-complete) Max 3	3
4(c)	Syntax error: A construct / statement in the source code that breaks the rules of the language Logic Error: An error in the algorithm that causes the program not to behave as intended Run-time: A program performs an invalid operation / tries to divide by zero // enters an infinite loop / stops unexpectedly	3

96. M/J 20/P21/Q6/c(ii)

6(c)(ii)	One mark for each test: For example: Test 1 Start time value "10:00", Duration value 30 Expected new time value "10:30" Test 2 Start time value "10:45", Duration value 30 Expected new time value "11:15" String values (time) must be enclosed in quotation marks, integer values (duration) must not. Penalise once then FT.	2
----------	--	----------

97. M/J 20/P22/Q2/a,b(ii)

2(a)	One mark per bullet point: • <u>Parameters</u> passed between modules // the <u>interface</u> between modules • Module Iteration • Module selection Max 2	2
------	---	----------

2(b)(ii)	<pre>graph TD; Shop[Shop()] -- SlotNumber --> ChooseSlot[ChooseSlot()]; Shop --> FillBasket[FillBasket()]; Shop --> Checkout[Checkout()]; ChooseSlot -- SlotNumber --> Shop; FillBasket --> Search[Search()]; Search -- ItemRef --> FillBasket; FillBasket --> Add[Add()]; Add -- ItemRef, Qty --> FillBasket; FillBasket --> FillBasket; Checkout --> Shop;</pre> One Mark for - Three middle row boxes correctly labelled and connected to Shop () - Two bottom row boxes correctly labelled and connected to FillBasket () - Iteration arrow on FillBasket () - Return parameters from ChooseSlot () and Checkout () - Return parameters from Search () - Two input parameters to Add () Notes: Parameter types must be as shown but ignore parameter names (if given)	6
----------	--	---

98. M/J 20/P22/Q4/b

4(b)	Adaptive Maintenance	1
------	----------------------	---

4(a)	<pre> DECLARE Random : ARRAY [1:10] OF INTEGER DECLARE NextNum, Index, Rnum : INTEGER DECLARE Exists : BOOLEAN NextNum ← 1 // index position for the next random number REPEAT Rnum ← INT(RAND(100)) + 1 // from original question Exists ← FALSE FOR Index ← 1 to NextNum - 1 // search for Rnum IF Random[Index] = Rnum THEN Exists ← TRUE ENDIF ENDFOR IF Exists = FALSE THEN Random[NextNum] ← Rnum // store Rnum NextNum ← NextNum + 1 // increment index ENDIF UNTIL NextNum > 10 1 mark for each of the following: 1 Conditional (outer) loop to generate 10 values 2 Inner loop to search array for duplicate number 3 Check for duplicate by comparing number generated with array element in a loop 4 Avoid checking uninitialised elements // array initialisation to rogue value at start of algorithm 5 If Rnum is a duplicate then repeat outer loop 6 If Rnum not a duplicate then assign to array element and Increment index Notes: Max 5 if statement to generate random number (as given in Q) not present or incorrectly placed. </pre>	6
------	---	----------

99. M/J 20/P22/Q6/a(ii,iii),b

6(a)(ii)	To test every path through the algorithm	1
6(a)(iii)	- Logical error - Algorithm is incorrect // program produces unexpected result / incorrect calculation is performed	2

6(b)	Test data: Any string value where hours are > “24” or minutes > “59” Explanation: Suitable explanation Note: Accept times that would also be invalid for the given scenario.	2
------	---	---

100. M/J 20/P23/Q2/a,b

2(a)	- Sequence of (module) execution - Module Iteration - Module selection One mark per item Max 2	2
2(b)(i)	<u>FUNCTION ModuleB</u> (<u>ParX : INTEGER</u>) <u>RETURNS BOOLEAN</u> One mark for each underlined part Ignore BYVALUE for ParX	3
2(b)(ii)	<u>PROCEDURE ModuleC</u> (<u>BYREF ParW: REAL</u>, <u>BYVALUE ParZ :</u> <u>STRING</u>) One mark for each underlined part BYVALUE not essential for ParZ	3

101. M/J 20/P23/Q3/b,c

3(b)(i)	Error 1 - Index stops at 5 – should go to 6. Fails to loop correctly through all characters of string (lines 26 to 38) // Final values for NumUpper is not as expected (because of loop error) Error 2 - Trace table row 12 – NumOther assigned wrong value. 0 expected not 3 One mark per bullet	2
3(b)(ii)	Error 1 Line Number: 26 Correction: FOR Index ← 1 to StrLen Error 2 Line Number: 41 Correction: NumOther ← StrLen - (NumDigit + NumUpper) One mark for each line number with correction	2
3(c)	Data Structure	1

102. M/J 20/P23/Q5/b(ii),d(i,ii)

5(b)(ii)	Three <i>different</i> tests String example 1 - String: "007" - Reason: Too many characters String example 2 - String: "4x" - Reason: String does not represent a numeric value / contains an illegal character String example 3 - String: "25" - Reason: number outside range 1 to 20 Illegal character can only be used in one test One mark for string plus one for corresponding explanation	6
5(d)(i)	One mark per bullet point - The use of tried and tested (library) subroutines - The use of modular programming (to break the problem down and make it easier to solve) - The use of programming practice to make the code easier to read (e.g. format, use of sensible variable names) Max 2	2
5(d)(ii)	One mark per bullet point - Testing may be carried out before the modules are developed // not ready for full testing - Module stubs contain simple code to provide a known response // temporary replacement for a called module / return a fixed value / output a message to confirm the module has been called	2

103. O/N 19/P21/Q2/a,d

2(a)	One mark for each feature: - Module <u>hierarchy</u> - The <u>parameters</u> that are passed (between modules) // the module <u>interface</u> - Selection / Decisions (which modules are executed) - Iteration / Repetition	4
------	---	----------

2(d)	One mark per bullet point: - Changes made to // Updating // Editing a program / algorithm / data structure / software / system ...as a result of changes to requirements / specification / legislation / available technology	2
------	---	----------

104. O/N 19/P21/Q5/a,b

5(a)	One mark for each point. A valid string must contain: - At least two // more than one upper case character(s) - At least five // more than four lower case character(s) - More digit characters than 'other' characters	3
------	---	----------

5(b)(i)

One mark for each area as outlined:

Index	NextChar	Upper	Lower	Digit	Other
		0	0	0	0
1	'J'	1			
2	'i'		1		
3	'm'		2		
4	'+'				1
5	'S'	2			
6	'm'		3		
7	'i'		4		
8	't'		5		
9	'h'		6		
10	'*'				2
11	'9'			1	
12	'9'			2	

5

5(b)(ii)	One mark per bullet point: - Returned value is FALSE - Digit - Other is not greater than zero // Number of Digit same as Other	2
----------	--	----------

105. O/N 19/P22/Q2/a

2(a)	One mark per point: 1. The source code represents a solution / design / algorithm expressed in a high-level language 2. The Object code is produced (by the compiler) during the translation stage // The Object code is produced by translating the source code (NOT produced by Interpreter) 3. Corrective maintenance occurs when testing reveals a fault (or error) in the program and this is corrected // Corrective maintenance is when errors are found and fixed // Corrective maintenance is when a program is debugged Accept alternative answers provided they relate to the program development cycle	3
------	--	----------

106. O/N 19/P22/Q4/b,c

4(b)	2 independent marks for each test: Test 1 MaxVol: 0/49 VolLevel expected: 49 Test 2 VolLevel before: 34 VolLevel expected: 34 Test 3 Parameter: 20 VolLevel expected: 0	6
4(c)(i)	One mark for type, one for description: • Type: Logical Error • Description: Program does not perform as expected • Type: Run-time error • Description: Program executes an invalid instruction / out of bounds error / attempts to divide by zero // program crashes	2
4(c)(ii)	One mark per bullet point: • Tests carried out before all the modules / subroutines have been written • Simple / dummy module written to simulate / model / replace the actual module / subroutine / object • Contains an output statement // returns a fixed value to indicate that the call has been made	3

107. O/N 19/P22/Q5

5	<pre>sequenceDiagram participant Renew as Renew() participant Validate as Validate() participant Lookup as Lookup() participant Update as Update() Renew->>Validate: P3 Validate-->>Renew: S2 Renew->>Lookup: P4 Lookup-->>Renew: M4 Renew->>Update: T4 Update-->>Renew: </pre> One mark for each of: - Diagram with boxes as above correctly labelled - P3 and S2 - Return Boolean from <code>Validate()</code> - P4 - M4 - T4 - Return parameter from <code>Update()</code>	7
---	---	---

108. O/N 19/P23/Q1/e

1(e)	White-box	1
------	-----------	---

109. O/N 19/P23/Q3

3	<pre> sequenceDiagram participant CP as ChangePassword() participant GP as GetPassword() participant UF as UpdateFile() CP->>GP: OldPassword, AccountID GP-->>CP: CP->>UF: AccountID, NewPassword UF-->>CP: </pre> One mark for each of the following: • all three boxes correctly labelled • parameters in to GetPassword() • value back from GetPassword() • parameters in to UpdateFile() • BOOLEAN value back from UpdateFile()	5
---	--	---

110. O/N 19/P23/Q4/a,b,c

4(a)	One mark for each point. Valid string must contain: • at least one '.' characters • one '@' character • more than 5 other characters.	3
------	---	---

4(b)(i)	One mark for each area as outlined:	5																																																																																																														
	<table><tr><th>Index</th><th>NextChar</th><th>NumDots</th><th>NumAts</th><th>NumOthers</th></tr><tr><td></td><td></td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>'J'</td><td></td><td></td><td>1</td></tr><tr><td>2</td><td>'i'</td><td></td><td></td><td>2</td></tr><tr><td>3</td><td>'m'</td><td></td><td></td><td>3</td></tr><tr><td>4</td><td>'.'</td><td>1</td><td></td><td></td></tr><tr><td>5</td><td>'9'</td><td></td><td></td><td>4</td></tr><tr><td>6</td><td>'9'</td><td></td><td></td><td>5</td></tr><tr><td>7</td><td>'@'</td><td></td><td>1</td><td></td></tr><tr><td>8</td><td>'s'</td><td></td><td></td><td>6</td></tr><tr><td>9</td><td>'k'</td><td></td><td></td><td>7</td></tr><tr><td>10</td><td>'a'</td><td></td><td></td><td>8</td></tr><tr><td>11</td><td>'i'</td><td></td><td></td><td>9</td></tr><tr><td>12</td><td>'l'</td><td></td><td></td><td>10</td></tr><tr><td>13</td><td>'.'</td><td>2</td><td></td><td></td></tr><tr><td>14</td><td>'c'</td><td></td><td></td><td>11</td></tr><tr><td>15</td><td>'o'</td><td></td><td></td><td>12</td></tr><tr><td>16</td><td>'m'</td><td></td><td></td><td>13</td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	Index	NextChar	NumDots	NumAts	NumOthers			0	0	0	1	'J'			1	2	'i'			2	3	'm'			3	4	'.'	1			5	'9'			4	6	'9'			5	7	'@'		1		8	's'			6	9	'k'			7	10	'a'			8	11	'i'			9	12	'l'			10	13	'.'	2			14	'c'			11	15	'o'			12	16	'm'			13																					
Index	NextChar	NumDots	NumAts	NumOthers																																																																																																												
		0	0	0																																																																																																												
1	'J'			1																																																																																																												
2	'i'			2																																																																																																												
3	'm'			3																																																																																																												
4	'.'	1																																																																																																														
5	'9'			4																																																																																																												
6	'9'			5																																																																																																												
7	'@'		1																																																																																																													
8	's'			6																																																																																																												
9	'k'			7																																																																																																												
10	'a'			8																																																																																																												
11	'i'			9																																																																																																												
12	'l'			10																																																																																																												
13	'.'	2																																																																																																														
14	'c'			11																																																																																																												
15	'o'			12																																																																																																												
16	'm'			13																																																																																																												
4(b)(ii)	TRUE	1																																																																																																														
4(c)	One mark for string and one mark for correct explanation. Same for second answer providing it results in a different path through the algorithm. Correct answers may be: • without the correct number of '.' • without the correct number of '@' • without the correct number of 'other characters'	4																																																																																																														

111. M/J 19/P21/Q1/c

1(c)	Data is chosen: • to test that the program does what it is supposed to do / to check that the results are as expected • to use known valid, boundary and erroneous values	2
------	---	---

112. M/J 19/P21/Q4/a,c

4(a)(i)	<table><tr><th>Index</th><th>AfterSpace</th><th>NextChar</th><th>NewString</th></tr><tr><td></td><td>FALSE</td><td></td><td>""</td></tr><tr><td>1</td><td></td><td>'X'</td><td>"X"</td></tr><tr><td>2</td><td>TRUE</td><td>'∇ '</td><td>"X∇"</td></tr><tr><td>3</td><td></td><td>'∇ '</td><td></td></tr><tr><td>4</td><td></td><td>'∇ '</td><td></td></tr><tr><td>5</td><td>FALSE</td><td>'Y'</td><td>"X∇Y"</td></tr><tr><td>6</td><td>TRUE</td><td>'∇ '</td><td>"X∇Y∇"</td></tr><tr><td>7</td><td>FALSE</td><td>'a'</td><td>"X∇Y∇a"</td></tr><tr><td>8</td><td></td><td>'n '</td><td>"X∇Y∇an"</td></tr><tr><td>9</td><td></td><td>'d'</td><td>"X∇Y∇and"</td></tr><tr><td>10</td><td>TRUE</td><td>'∇ '</td><td>"X∇Y∇and∇"</td></tr><tr><td>11</td><td></td><td>'∇ '</td><td></td></tr><tr><td>12</td><td>FALSE</td><td>'Z'</td><td>"X∇Y∇and∇Z"</td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr></table> One mark for each area as outlined. If no row marks then mark by column for columns 2, 3 and 4	Index	AfterSpace	NextChar	NewString		FALSE		""	1		'X'	"X"	2	TRUE	'∇ '	"X∇"	3		'∇ '		4		'∇ '		5	FALSE	'Y'	"X∇Y"	6	TRUE	'∇ '	"X∇Y∇"	7	FALSE	'a'	"X∇Y∇a"	8		'n '	"X∇Y∇an"	9		'd'	"X∇Y∇and"	10	TRUE	'∇ '	"X∇Y∇and∇"	11		'∇ '		12	FALSE	'Z'	"X∇Y∇and∇Z"																																					6
Index	AfterSpace	NextChar	NewString																																																																																											
	FALSE		""																																																																																											
1		'X'	"X"																																																																																											
2	TRUE	'∇ '	"X∇"																																																																																											
3		'∇ '																																																																																												
4		'∇ '																																																																																												
5	FALSE	'Y'	"X∇Y"																																																																																											
6	TRUE	'∇ '	"X∇Y∇"																																																																																											
7	FALSE	'a'	"X∇Y∇a"																																																																																											
8		'n '	"X∇Y∇an"																																																																																											
9		'd'	"X∇Y∇and"																																																																																											
10	TRUE	'∇ '	"X∇Y∇and∇"																																																																																											
11		'∇ '																																																																																												
12	FALSE	'Z'	"X∇Y∇and∇Z"																																																																																											
4(a)(ii)	To remove repeated space characters	1																																																																																												
4(a)(iii)	- All leading spaces / spaces at the beginning - will be removed (from the input string)	2																																																																																												

4(c)	Adaptive Maintenance	1
------	----------------------	---

113. M/J 19/P22/Q3/a,b

3(a)(i)	<u>PROCEDURE SubA</u> (<u>A : STRING</u> , <u>B : INTEGER</u> , <u>BYREF C : CHAR</u>) One mark for each underlined part Ignore BYVAL for parameter A and/or parameter B Parameter order / names not important but must be correct data types	3
3(a)(ii)	<u>Function SubB</u> (<u>D : STRING</u> , <u>E : INTEGER</u>) <u>RETURNS BOOLEAN</u> One mark for each underlined part Ignore BYVAL for parameter D and/or parameter E Parameter order / names not important but must be correct data types	3
3(b)	• Selection • Iteration • Sequence One mark per bullet to max. 2	2

114. M/J 19/P22/Q4/a,b

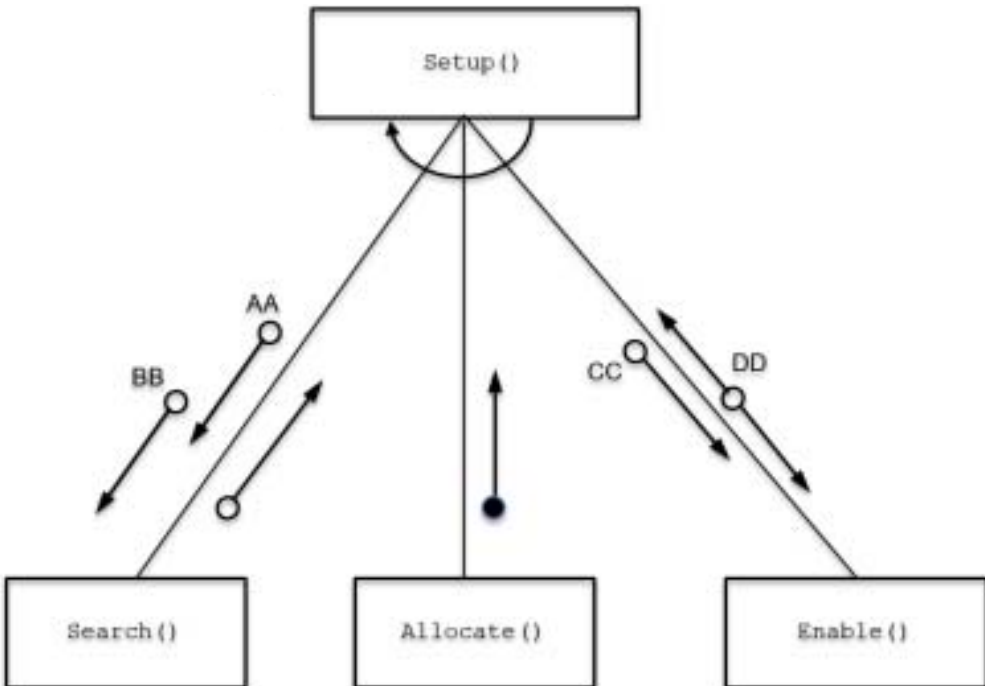
4(a)(i)	Index	NextChar	Selected	NewValue	NewString	5
			0		"0"	
	1	'1'			"01"	
	2	'2'			"012"	
	3	'▽'		12		
			12			
					"0"	
	4	'3'			"03"	
	5	'4'			"034"	
	6	'▽'		34		
			34			
					"0"	
	7	'5'			"05"	
	8	'▽'		5		
					"0"	
	9	'▽'		0		
					"0"	
	10	'3'			"03"	
	11	'9'			"039"	
	One mark for each column.					
	If no mark for columns, award one mark for initialisation of <code>Selected</code> to 0 and <code>Newstring</code> to '0' (single or double quotes).					

4(a)(ii)	34	1
----------	----	---

4(b)(i)	- The final value (in the string) is the largest value (39) and is not considered // the final comparison with variable <code>Selected</code> is not made - The loop terminates at the end of the string (the character 9) // there wasn't a final space / non-numeric digit One mark per bullet.	2
---------	--	---

4(b)(ii)	- Check the (final) value of <code>NewString</code> after the loop... ...and see if it is greater than <code>Selected</code> (repeat the existing conditional clause) OR - Amend the algorithm to add a space character / non-numeric character to the end of the string... ...before the <code>FOR</code> loop / at the start of the function One mark per bullet point Accept alternative workable solution	2
----------	---	----------

115. M/J 19/P23/Q3/a

3(a)	 Mark as follows: - One mark for four boxes connected as shown (search, allocate and enable in the correct order) - One mark for each of: - Arrows labelled AA, BB and CC with correct symbols - Return parameter from Search with correct symbol - Return boolean from Allocate with correct symbol - Double-headed arrow for DD - One mark for repetition arrow (either direction)	6
------	---	----------

116. M/J 19/P23/Q5/a,b

5(a)(i)	To test <u>every path</u> through the code / algorithm Accept phrase with equivalent meaning	1
5(a)(ii)	A trace table	1

5(b)(i)	String: - three possible formats for string containing two words: <code>"Cat Dog"</code> // <code>"Cat Dog"</code> // <code>"CatDog"</code> Explanation: - When a space character is encountered <code>NumWords</code> is incremented by 1 OR: - The algorithm counts the spaces - and not the words 1 mark for a string that would give the correct result 2 marks for explanation	3
5(b)(ii)	String 1: - Output: <code>Number of words : 2</code> Description 1: - Check character at end of string - If not a character, increment variable <code>NumTotal</code> OR: - Add a space at the beginning / end of string - At the start of the algorithm OR: - Change Initialisation of <code>NumWords</code> - to 1 OR: - After the loop - Add 1 to <code>NumWords</code> String 2: - Output: <code>Number of words : 5</code> Description 2: - Detect a space followed by a space - Count as a single space / only increment variable <code>NumTotal</code> once OR: - Replace all double spaces with a single space - Before the loop OR: - After the loop - Subtract 2 from <code>NumWords</code> Many possible solutions. One mark for each correct output One mark for each description bullet point	6

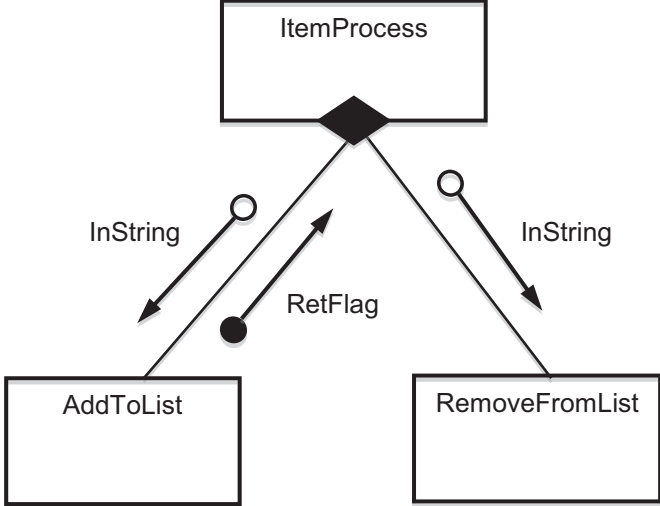
117. O/N 18/P22/Q2/b

2(b)(i)	1 mark for name, 1 mark for description Accept by example for description Name: Run-time Description: The program executes an illegal instruction // performs an illegal operation that is trapped by the OS	2
2(b)(ii)	One specific value from within each of the following ranges: For example: 73 (a value ≤ 100) 145.3 (a value > 100) 1 mark for the value, plus one for a meaningful description.	4

118. O/N 18/P23/Q2/b,c

2(b)(i)	Name: Logic (error) Description: Where the program does not behave as expected / Does not give expected result / An error in the logic of the algorithm OR Name: Run-time // execution (error) Description: The program performs an illegal operation One mark for name + one mark for corresponding description	2
2(b)(ii)	Values: any <code>Spend</code> value and <code>Points > 2000</code> Justification: <code>Bonus</code> should be 100 Values: <code>Spend > 1000</code> and <code>Points ≤ 2000</code> Justification: <code>Bonus</code> should be 50 Values: <code>Spend ≤ 1000</code> and <code>Points ≤ 2000</code> Justification: <code>Bonus</code> should be 10 2 marks for values 2 marks for relevant and appropriate reasons	4
2(c)	Name: Corrective Reason: Amend the algorithm to 'eliminate errors' Name: Adaptive Reason: In response to specification change arising from changes to business rules or environment (regulatory) Name: Perfective Reason: To make improvements to the program One mark for each name plus one mark for corresponding reason up to max 4 marks	4

119. M/J 18/P21/Q4/a,b

4(a)	Features: • The <u>hierarchy</u> of modules • The <u>parameters</u> that are passed between modules // the <u>interface</u> between the modules • The <u>sequence</u> of module execution One mark per item	3
4(b)	 Mark as follows: One mark for top box One mark for both lower boxes One mark for diamond 'decision' symbol One mark for each parameter (3 parameters)	6

120. M/J 18/P22/Q2/c(i)

2(c)(i)	• Mistake: function header returns a <code>CHAR</code> but last line of code returns an <code>INTEGER</code> • Correction: Function should return an <code>INTEGER</code> // Change line 26 to <code>return c</code> as <code>CHAR</code>	2
---------	--	----------

121. M/J 18/P22/Q4/a

4(a)(i)

```

graph TD
    Start(( )) --> PB[Process basket]
    PB --> A((A))
    PB --> B((B))
    PB --> E((E))
    A --> DB[Discard basket  
(remove all items)]
    B --> C((C))
    C --> DIQ[Change item  
quantity]
    E --> F((F))
    F --> G((G))
    G --> CO[Checkout  
(pay and despatch)]
    DB --> X1((X))
    DIQ --> X2((X))
    CO --> X3((X))
    X1 --> A
    X2 --> B
    X3 --> E
          
```

The diagram shows a central 'Process basket' activity box. Three arrows originate from it: one to 'A', one to 'B', and one to 'E'. 'A' leads to 'Discard basket (remove all items)', 'B' leads to 'C', which leads to 'Change item quantity', and 'E' leads to 'F', which leads to 'G', which leads to 'Checkout (pay and despatch)'. Three arrows labeled 'X' (each with a filled circle) point back to the start of the paths: from 'Discard basket' back to 'A', from 'Change item quantity' back to 'B', and from 'Checkout' back to 'E'.

2

Mark as follows:

- One mark for three arrows all labelled X (with filled circles)
- One mark for diamond symbol

4(a)(ii)

Parameter	Information
A	BasketID
B	BasketID, ItemID, Quantity (In any order)
C	
D	
E	BasketID, DeliveryAddress, PaymentDetails (In any order)
F	
G	

3

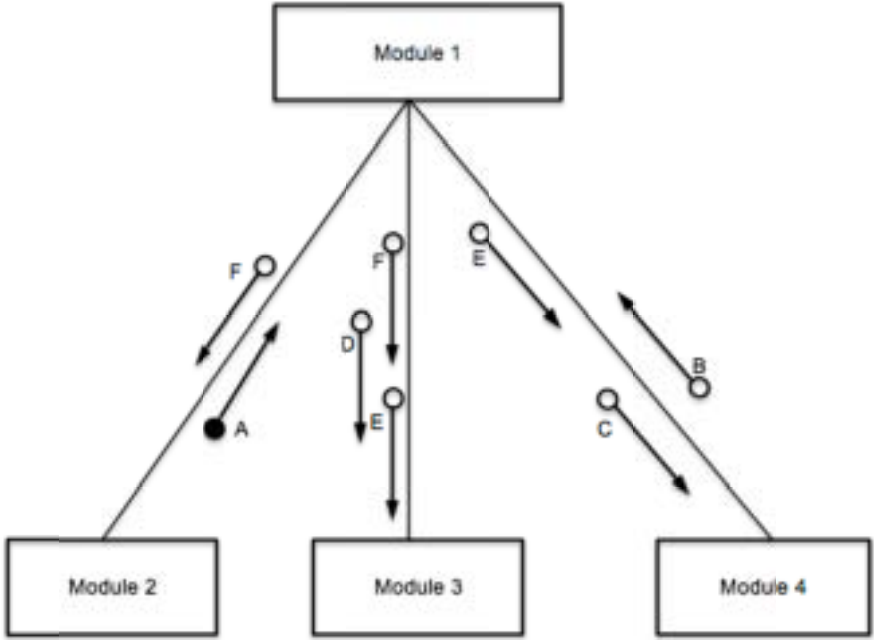
Mark as follows:

- One mark for parameter **A**
- One mark for parameters **B, C & D**
- One mark for parameters **E, F & G**

122. M/J 18/P23/Q2/c(i)

2(c)(i)	Either: - Mistake: function header specifies return of an <code>INTEGER</code> but line 37 returns a <code>STRING</code> // pseudocode returns <code>Grade</code> but should have returned <code>DGradeCount</code> - Correction: <code>RETURN DGradeCount</code> (as per code pseudocode comment) Or: - Mistake: Statement on line 32 uses <code>'&'</code> operator which concatenates <code>STRINGS</code>, but variable <code>n</code> is an <code>INTEGER</code> - Correction: Convert <code>n</code> to a <code>STRING</code> before concatenating	2
---------	--	----------

123. M/J 18/P23/Q3/b

3(b)	 Mark as follows: - One mark for all four modules - One mark for each set of interface parameters	4
------	--	----------

124. O/N 17/P21/Q2/a,b

2(a)	<table><tr><th>StartNumber</th><th>EndNumber</th><th>Divisor</th><th>NumberFound</th><th>Number</th><th>Remainder</th><th>Output</th></tr><tr><td>11</td><td>13</td><td>2</td><td>0</td><td>11</td><td>1</td><td></td></tr><tr><td></td><td></td><td></td><td></td><td>12</td><td>0</td><td>12</td></tr><tr><td></td><td></td><td></td><td>1</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td>13</td><td>1</td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td>Count: 1</td></tr></table> One mark for correct Number column One mark for correct Remainder column One mark for correct Output	StartNumber	EndNumber	Divisor	NumberFound	Number	Remainder	Output	11	13	2	0	11	1						12	0	12				1								13	1								Count: 1	3
StartNumber	EndNumber	Divisor	NumberFound	Number	Remainder	Output																																						
11	13	2	0	11	1																																							
				12	0	12																																						
			1																																									
				13	1																																							
						Count: 1																																						
2(b)	Mark as follows: • For a (given) range of values • Counts the number of times one number (numerator) is an exact divisor of the other • Outputs each numerator (only) • Outputs the count Accept by example No mark for: • ...calculate the remainder • ...add one to NumberFound	3																																										

125. O/N 17/P21/Q5/b

5(b)	Invalid string examples: A string with nothing before '@' A string with nothing after '@' A string with 1 or 2 characters after '@' A string with no '@'symbol A string with more than one '@' symbol Explanation Sensible explanation mapping each given string to an individual rule One mark for string One mark for explanation Each rule should be tested once only	6
------	--	---

126. O/N 17/P22/Q3

3				Max 8
	Line number	Error	Correction	
	01	Wrong procedure name – "SortArray"	PROCEDURE ArraySort	
	02	Wrong data type - CHAR	DECLARE Temp: STRING	
	03	Variables undefined	DECLARE FirstID, SecondID, I, J : INTEGER	
	04	Wrong 'Value2' of 100	FOR I ← 1 TO 99	
	05	Wrong range	FOR J ← 1 TO (100 - I)	
	06/07	Wrong function - MODULUS	Replace MODULUS with TONUM: FirstID ← TONUM (LEFT(Product[J],	
	06/07	Wrong value of 6	Should be 4: FirstID ← TONUM(LEFT(Product[J],	
	10	Assigning wrong value to Temp	Temp ← Product[J]	
	11	Assigning wrong value to Product[I]	Product[J] ← Product[J + 1]	
	13/14	Lines reversed	13 ENDIF 14 ENDFOR	
One mark for each correct row				

127. O/N 17/P22/Q4/c

4(c)	Program is probably working correctly if: • Header is present giving frequency as 20 • Column headers are present • All rows are present (1 to 10) • The difference is calculated correctly • Output is formatted correctly • Total differences should be zero • Sum of Frequencies should be 200	Max 2
------	--	--------------

128. O/N 17/P23/Q2/a,b

2(a)

StartNumber	EndNumber	Divisor	NumberFound	Number	Remainder	Output
11	13	2	0	11	1	
				12	0	12
			1			
				13	1	
						Count: 1

One mark for correct Number column
One mark for correct Remainder column
One mark for correct Output

3

2(b)	Mark as follows: - For a (given) range of values - Counts the number of times one number (numerator) is an exact divisor of the other - Outputs each numerator (only) - Outputs the count Accept by example No mark for: ...calculate the remainder ...add one to <code>NumberFound</code>	3
------	--	---

129. O/N 17/P23/Q5/b

5(b)	Invalid string examples: A string with nothing before '@' A string with nothing after '@' A string with 1 or 2 characters after '@' A string with no '@' symbol A string with more than one '@' symbol Explanation Sensible explanation mapping each given string to an individual rule One mark for string One mark for explanation Each rule should be tested once only	6
------	---	---

130. M/J 17/P21/Q4/a,b

4(a)	- The <u>hierarchy</u> of modules <u>Parameters</u> that are passed between modules // The <u>interface</u> between the modules / - The <u>sequence</u> - Iteration / selection One mark per item	3
4(b)	<u>FUNCTION</u> <u>CardPayment</u> (<u>ParamA : REAL</u>, <u>ParamB : STRING</u>) <u>RETURNS</u> <u>BOOLEAN</u> One mark per underlined part Order not significant for <code>ParamA</code> and <code>ParamB</code> Function name and parameter names not important but must be present	3

131. M/J 17/P21/Q6/b

6(b)(i)	String1: (e.g. "AAbb123") One mark for a valid string having: • at least 2 uppercase alphabetic • at least 2 lowercase alphabetic • at least 3 numeric characters • No other character String2 – String5: One mark for correct string and explanation (testing different rules of the function) Test strings breaking different rules: • With incorrect numbers of: • Lower case characters • Upper case characters • Numeric characters • Containing an invalid character	5
6(b)(ii)	White Box	1
6(b)(iii)	• Testing may be carried out before the modules are developed // not ready for full testing • Module stubs contain simple code to provide a known response // temporary replacement for a called module	2

132. M/J 17/P22/Q6/b

6(b)	String1: (for example, "ABC12XYZ") One mark for a valid string having: • Correct length (between 6 and 9 characters) • 3 capital letters followed by... • 2 numeric characters followed by... • between 1 and 4 capital letters String2 to String5: 1 mark for each string and explanation (testing different rules of the function) Test strings breaking one different rules: • Incorrect length • With incorrect number of capital letters at the start • With non-numeric characters in positions 4 and 5 • With incorrect number of capital letters at the end • Containing an invalid character (not alpha-numeric)	5
------	---	----------

133. M/J 17/P23/Q4/a,b

4(a)	• The <u>hierarchy</u> of modules • <u>Parameters</u> that are passed between modules // The <u>interface</u> between the modules / • The <u>sequence</u> • Iteration / selection One mark per item	3
4(b)	<u>FUNCTION CardPayment</u> (<u>ParamA : REAL, ParamB : STRING</u>) <u>RETURNS BOOLEAN</u> One mark per underlined part Order not significant for ParamA and ParamB Function name and parameter names not important but must be present	3

134. M/J 17/P23/Q6/b

6(b)(i)	String1: (e.g. "AAbb123") One mark for a valid string having: • at least 2 uppercase alphabetic • at least 2 lowercase alphabetic • at least 3 numeric characters • No other character String2 – String5: One mark for correct string and explanation (testing different rules of the function) Test strings breaking different rules: • With incorrect numbers of: • Lower case characters • Upper case characters • Numeric characters • Containing an invalid character	5
6(b)(ii)	White Box	1
6(b)(iii)	• Testing may be carried out before the modules are developed // not ready for full testing • Module stubs contain simple code to provide a known response // temporary replacement for a called module	2

135. O/N 16/P21/Q2/a**2 (a)**

Status2	ReadingCount	ThisBit	BitCount	OUTPUT
			0	
1	1	1	1	
	2	0	1	
	3	1	2	
	4	1	3	
	5	1	4	
	6	0	4	
1				
	1	1	5	Error – Investigate
			0	
	2	1	1	
	3	0	1	
	4	0	1	
	5	1	2	
	6	1	3	
0				

1 must 'follow' 6 as shown by arrow. Can have only 1 or nothing above.

1

Error – Investigate

0 must 'follow' 6 as shown by arrow. Can have only 1 or nothing above.

0

One mark per area outlined

[7]**136. O/N 16/P22/Q4/d**

- (d) (i) • Program code is modified
• following a change to the requirements

[2]

- (ii) • Use an array / list / file to store each number generated // a flag value
• Check the array / list / file to see if the new random number has already been drawn
• If YES, generate another number
• If NO, output the number and update the array / list / file

[Max. 3]

- (b) 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

Expected Loop-based solution:

```
DECLARE i , NextNumber : INTEGER
FOR i ← 1 TO 4
    NextNumber ← 1 + INT(Rnd() * 150)
    OUTPUT NextNumber
ENDFOR
```

Mark as follows:

- Declaration of all variables used including data types
- Loop
- Assignment / calculation of (four) different random numbers (0 to 150) in a loop
- Output of four values

ALTERNATIVE Non-Loop version

```
DECLARE Num1, Num2, Num3, Num4 : INTEGER

Num1 ← INT(RND() * 150) + 1
Num2 ← INT(RND() * 150) + 1
Num3 ← INT(RND() * 150) + 1
Num4 ← INT(RND() * 150) + 1

OUTPUT Num1, Num2, Num3, Num4
```

Mark as follows:

- Declaration of all variables used including data types
- Assignment of four different random numbers (0 to 150)
- Assignment to four separate variables
- Output of four values

[4]

Q4 (b): Visual Basic

```
Randomize()  
Dim i As Integer  
Dim NextNumber As Integer  
For i = 1 To 4  
    NextNumber = 1 + Int(Rnd() * 150)  
    Console.WriteLine(NextNumber)  
Next
```

OR

```
Randomize()  
Dim Num1, Num2, Num3, Num4 As Integer  
Num1 = 1 + Int(Rnd() * 150)  
Num2 = 1 + Int(Rnd() * 150)  
Num3 = 1 + Int(Rnd() * 150)  
Num4 = 1 + Int(Rnd() * 150)  
Console.WriteLine(Num1, Num2, Num3, Num4)
```

Q4 (b): Pascal

```
Var i : Integer;  
    NextNumber : Integer;  
Begin  
    Randomize;  
    For i := 1 To 4 Do  
        Begin  
            NextNumber := 1 + Random(150);  
            Writeln(NextNumber);  
        End;  
    Readln;  
End.
```

OR

```
Var Num1, Num2, Num3, Num4 : Integer;  
Begin  
    Randomize;  
    Num1 := 1 + Random(150);  
    Num2 := 1 + Random(150);  
    Num3 := 1 + Random(150);  
    Num4 := 1 + Random(150);  
    Writeln(Num1, Num2, Num3, Num4);  
    Readln;  
End.
```

Q4 (b): Python

```
import random
# i : Integer
# NextNumber : Integer

for i in range(1, 5) :
    NextNumber = 1 + int(150 * random.random())
    print(NextNumber)
```

Alternative:

```
import random
# i Integer
# NextNumber Integer
for i in range(1, 5) :
    NextNumber = random.randint(1, 150)
    print(NextNumber)
```

OR

```
import random
# i Integer
# Num1, Num2, Num3, Num4 Integer

Num1 = random.randint(1, 150)
Num2 = random.randint(1, 150)
Num3 = random.randint(1, 150)
Num4 = random.randint(1, 150)

print(Num1, Num2, Num3, Num4)
```

137. O/N 16/P22/Q5/c(iii)

(iii) Error line number 5, 9 or 11 as follows:

Line 5:

The boundary value must be included //

IF PlayerScore[GameIndex, PlayerIndex] >= 100 // > 99

Line 9:

The boundary value must be included //

IF PlayerScore[GameIndex, PlayerIndex] >= 50 // > 49

Line 11:

One should be added to Total50 (not GameIndex) //

Total50 ← Total50 + 1

One mark for line number + explanation

[1]

138. O/N 16/P23/Q2/a**2 (a)**

Status2	ReadingCount	ThisBit	BitCount	OUTPUT
			0	
1	1	1	1	
	2	0	1	
	3	1	2	
1 must 'follow' 6 as shown by arrow. Can have only 1 or nothing above.	4	1	3	
	5	1	4	
	6	0	4	
1				
	1	1	5	Error – Investigate
			0	
	2	1	1	
0 must 'follow' 6 as shown by arrow. Can have only 1 or nothing above.	3	0	1	
	4	0	1	
	5	1	2	
	6	1	3	
0				

One mark per area outlined

[7]

139. M/J 16/P21/Q4/b

(b) (i)	<pre>graph TD; A[On-line shopping] --> B[Select Item]; A --> C[Checkout]; A --> D[Dispatch]; B --> E[Search]; B --> F[Add to basket]; B --> B; C --> G[Card payment]; C --> H[Account payment]; D --> I[Print dispatch label]; D --> J[Print address label]; D --> D;</pre> The flowchart illustrates the 'On-line shopping' process. It starts with a central box 'On-line shopping' which branches into three main stages: 'Select Item', 'Checkout', and 'Dispatch'. 'Select Item' leads to 'Search' and 'Add to basket', with a feedback loop back to 'Select Item'. 'Checkout' leads to 'Card payment' and 'Account payment'. 'Dispatch' leads to 'Print dispatch label' and 'Print address label', with a feedback loop back to 'Dispatch'. A diamond symbol is placed at the junction of the 'Checkout' branch. One mark per correct annotation as shown Arrows may be drawn clockwise or anticlockwise Diamond symbol may be filled or unfilled but must be in position shown	3
(ii)	A (or B) – Card details / Card number / Card info B (or A) – Cost details / amount payable / product cost / total bill C – (Flag) indicator for successful payment // payment confirmation Data items for A and B are interchangeable	3

140. M/J 16/P21/Q6/a,b

6	(a)	<table><tr><th>n</th><th>f</th><th>x</th><th>y</th><th>MID (String1, x, 1)</th><th>MID (String2, y, 1)</th></tr><tr><td>0</td><td>0</td><td></td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td>1</td><td>1</td><td>'R'</td><td>'R'</td></tr><tr><td></td><td></td><td>2</td><td>2</td><td>'E'</td><td>'A'</td></tr><tr><td>2</td><td></td><td>2</td><td>1</td><td>'E'</td><td>'R'</td></tr><tr><td>3</td><td></td><td>3</td><td>1</td><td>'T'</td><td>'R'</td></tr><tr><td>4</td><td></td><td>4</td><td>1</td><td>'R'</td><td>'R'</td></tr><tr><td></td><td></td><td>5</td><td>2</td><td>'A'</td><td>'A'</td></tr><tr><td></td><td></td><td>6</td><td>3</td><td>'C'</td><td>'C'</td></tr><tr><td></td><td>4</td><td></td><td></td><td></td><td></td></tr></table> One mark per correct column • Minus one mark if anything else on first row • Column 2 – '4' must not precede '4' in column 1 (as shown by arrow) • Letters must all be in upper case • Ignore quotation symbol	n	f	x	y	MID (String1, x, 1)	MID (String2, y, 1)	0	0					1		1	1	'R'	'R'			2	2	'E'	'A'	2		2	1	'E'	'R'	3		3	1	'T'	'R'	4		4	1	'R'	'R'			5	2	'A'	'A'			6	3	'C'	'C'		4					6
n	f	x	y	MID (String1, x, 1)	MID (String2, y, 1)																																																										
0	0																																																														
1		1	1	'R'	'R'																																																										
		2	2	'E'	'A'																																																										
2		2	1	'E'	'R'																																																										
3		3	1	'T'	'R'																																																										
4		4	1	'R'	'R'																																																										
		5	2	'A'	'A'																																																										
		6	3	'C'	'C'																																																										
	4																																																														
	(b) (i)	• to search for a string within another string / String2 within String1 • to return the position of the start of String2 within String1 // by example First mark point: allow locate / find / calculate position of	2																																																												
	(ii)	Value: 0 / zero Meaning: String2 not found in String1	2																																																												
	(iii)	Option 1 • It is possible to “fall off” the end of String1 (or by example) while string match (for example, String1 = "Retrace", String2 = "Raced") • MID (String1, x, 1) // description of 'subscript out of range' Option 2 • If either string is empty then • subscript out of range error // description Option 3 • If String1 found within String 2 • An endless loop will occur	2																																																												

141. M/J 16/P22/Q4/b

(b) (i)	<pre>graph TD; A[On-line shopping] --> B[Select Item]; A --> C[Checkout]; A --> D[Dispatch]; B --> E[Search]; B --> F[Add to basket]; B --> B; C --> G[Card payment]; C --> H[Account payment]; D --> I[Print dispatch label]; D --> J[Print address label]; D --> D;</pre> One mark per correct annotation as shown Arrows may be drawn clockwise or anticlockwise Diamond symbol may be filled or unfilled but must be in position shown	3
(ii)	A (or B) – Card details / Card number / Card info B (or A) – Cost details / amount payable / product cost / total bill C – (Flag) indicator for successful payment // payment confirmation Data items for A and B are interchangeable	3

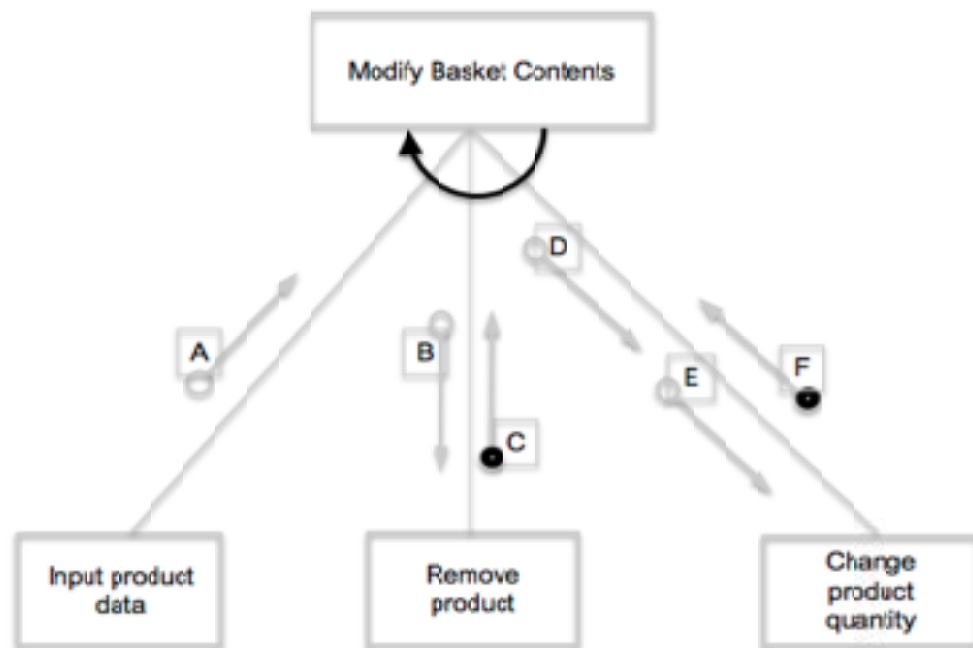
142. M/J 16/P22/Q6/a,b

6	(a)	<table><tr><th>n</th><th>f</th><th>x</th><th>y</th><th>MID (String1, x, 1)</th><th>MID (String2, y, 1)</th></tr><tr><td>0</td><td>0</td><td></td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td>1</td><td>1</td><td>'R'</td><td>'R'</td></tr><tr><td></td><td></td><td>2</td><td>2</td><td>'E'</td><td>'A'</td></tr><tr><td>2</td><td></td><td>2</td><td>1</td><td>'E'</td><td>'R'</td></tr><tr><td>3</td><td></td><td>3</td><td>1</td><td>'T'</td><td>'R'</td></tr><tr><td>4</td><td></td><td>4</td><td>1</td><td>'R'</td><td>'R'</td></tr><tr><td></td><td></td><td>5</td><td>2</td><td>'A'</td><td>'A'</td></tr><tr><td></td><td></td><td>6</td><td>3</td><td>'C'</td><td>'C'</td></tr><tr><td></td><td>4</td><td></td><td></td><td></td><td></td></tr></table> One mark per correct column • Minus one mark if anything else on first row • Column 2 – '4' must not precede '4' in column 1 (as shown by arrow) • Letters must all be in upper case • Ignore quotation symbol	n	f	x	y	MID (String1, x, 1)	MID (String2, y, 1)	0	0					1		1	1	'R'	'R'			2	2	'E'	'A'	2		2	1	'E'	'R'	3		3	1	'T'	'R'	4		4	1	'R'	'R'			5	2	'A'	'A'			6	3	'C'	'C'		4					6
n	f	x	y	MID (String1, x, 1)	MID (String2, y, 1)																																																										
0	0																																																														
1		1	1	'R'	'R'																																																										
		2	2	'E'	'A'																																																										
2		2	1	'E'	'R'																																																										
3		3	1	'T'	'R'																																																										
4		4	1	'R'	'R'																																																										
		5	2	'A'	'A'																																																										
		6	3	'C'	'C'																																																										
	4																																																														
	(b) (i)	• to search for a string within another string / String2 within String1 • to return the position of the start of String2 within String1 // by example First mark point: allow locate / find / calculate position of	2																																																												
	(ii)	Value: 0 / zero Meaning: String2 not found in String1	2																																																												
	(iii)	Option 1 • It is possible to “fall off” the end of String1 (or by example) while string match (for example, String1 = "Retrace", String2 = "Raced") • MID (String1, x, 1) // description of 'subscript out of range' Option 2 • If either string is empty then • subscript out of range error // description Option 3 • If String1 found within String 2 • An endless loop will occur	2																																																												

143. M/J 16/P23/Q4/b

(b) (i)

[1]



One mark for correct arrow as shown – accept either direction

(ii)

[4]

Data Item	Parameter					
	A	B	C	D	E	F
Product ID	✓	✓		✓	(✓)	
Quantity				(✓)	✓	
Flag Value – indicating operation success or fail			✓			✓

Mark as follows:

Row 1: One mark for tick in A **AND** B, one mark for D **OR** E

Row 2: One mark for D **OR** E (must be opposite of Row 1)

Row 3: One mark for C **AND** F

144. M/J 16/P23/Q6/a,b**6 (a) (i)****[4]**

n	x	Flag	m	NewString
–		TRUE	7	“”
1	‘B’	FALSE		“B”
2	‘i’			“Bi”
3	‘g’			“Big”
4	‘∇’	TRUE		“Big∇”
5	‘B’	FALSE		“Big∇B”
6	‘e’			“Big∇Be”
7	‘n’			“Big∇Ben”

One mark per correct column (**columns 3 and 4 count as 1**)

- Arrows indicate required sequence (i.e. “B” can’t be in row before TRUE)
- Letter case must be as shown
- Ignore quotation symbol
- Allow “ ” or ∇ for space symbol

(ii) (To return a string where:)**[2]**

- The first character of each word is capitalised / made upper case // by explanation
- The remaining characters (of each word) are made lower case

(b) (i) • The function operates normally // returns an ‘empty string’**[1]****(ii) Examples of suitable test strings:****[3]**

- Strings with all capitals
- Strings with all lower case
- Strings with first letters of words already capitalised (i.e. in correct format)
- Strings in “reverse” format – i.e. first letters lower case, the rest upper case
- String with only one word
- Strings with multiple spaces
- Strings with numbers / symbols

One mark for each string example **plus** supporting explanation.

145. O/N 15/P21/Q3/a**3 (a)**

	Inputs		Output
Test Case	InA	InB	OutZ
1	TRUE	TRUE	FALSE
2	TRUE	FALSE	TRUE
3	FALSE	TRUE	TRUE
4	FALSE	FALSE	TRUE

[1]

[1]

[1]

[1]

146. O/N 15/P21/Q8/a,b,c**8 (a) r**

[1]

Ignore inclusion of any quotes

(b) (i) 2

[1]

Ignore inclusion of any quotes for part (i), (ii) and (iii)

(ii) +

[1]

(iii) 7

[1]

(c) (i)

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT
2	5	3	8	40	-	1	1/40

[2]

(ii)

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT
3	4	1	4	16	+	16	1

[2]

(iii)

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT
7	9	2	3	27	+	39	
						12	
							1 12/27

[3]

147. O/N 15/P21/Q8/d**(d) (i)** Adaptive (maintenance)

[1]

(ii) Allow more than two fractions to be added

[1]

Numerator/denominator more than 1 digit

[1]

Multiply and division also possible

[1]

Allow brackets

[1]

Give answer as decimal number

[1]

Lowest possible denominator

[1]

Trap any fraction which has a zero numerator

[1]

Allow the input of vulgar fraction(s)

[1]

[MAX 3]

148. O/N 15/P22/Q2/a**2 (a)**

	Inputs		Output
Test Case	P	Q	X
1	1	1	1
2	1	0	0
3	0	1	0
4	0	0	0

[1]

[1]

[1]

[1]

149. O/N 15/P22/Q5/a(i)

5 (a) (i)

N	i	j	Temp		1	2	3	4	5
					11	16	13	7	8
5									
	1	1							
		2				13	16		
		3					7	16	
		4						8	16
		5							
	2	1							
		2				7	13		
		3					8	13	
		4							
	3	1			7	11			
		2				8	11		
		3							
	4	1			<i>final values are:</i>				
		2			7	8	11	13	16

Terminates at 5 and 4 respectively

Must be:

- Preceded by other entries ...
- nothing after the 1, 2 sequence

Note these final values will not be shown on the same row ...

150. O/N 15/P23/Q3/a**3 (a)**

	Inputs		Output
Test Case	InA	InB	OutZ
1	TRUE	TRUE	FALSE
2	TRUE	FALSE	TRUE
3	FALSE	TRUE	TRUE
4	FALSE	FALSE	TRUE

[1]

[1]

[1]

[1]

151. O/N 15/P23/Q8/a,b,c**8 (a) r**

[1]

Ignore inclusion of any quotes

(b) (i) 2

[1]

Ignore inclusion of any quotes for part (i), (ii) and (iii)

(ii) +

[1]

(iii) 7

[1]

(c) (i)

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT
2	5	3	8	40	-	1	1/40

[2]

(ii)

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT
3	4	1	4	16	+	16	1

[2]

(iii)

N1	N2	N3	N4	BottomAnswer	Op	TopAnswer	OUTPUT
7	9	2	3	27	+	39	
						12	
							1 12/27

[3]

152. O/N 15/P23/Q8/d

(d) (i) Adaptive (maintenance)

[1]

(ii) Allow more than two fractions to be added

[1]

Numerator/denominator more than 1 digit

[1]

Multiply and division also possible

[1]

Allow brackets

[1]

Give answer as decimal number

[1]

Lowest possible denominator

[1]

Trap any fraction which has a zero numerator

[1]

Allow the input of vulgar fraction(s)

[1]

[MAX 3]

153. M/J 15/P21/Q1/c

- (c) (i) • Choosing data/values...
 • Test every possible 'logic path' through the code
 // with knowledge of the structure/code

Ignore any reference to normal/boundary/extreme ...

[2]

- (ii) • `PersonalBest` column labelled
 • Test number 1 message: "Equals personal best time"/or similar
 • Test 2/Test 3 – data for better performance ...
 • Described with suitable message
 • Test 2/Test 3 – data for worse performance ...
 • Described with suitable message

[6]

154. M/J 15/P21/Q3/a,b

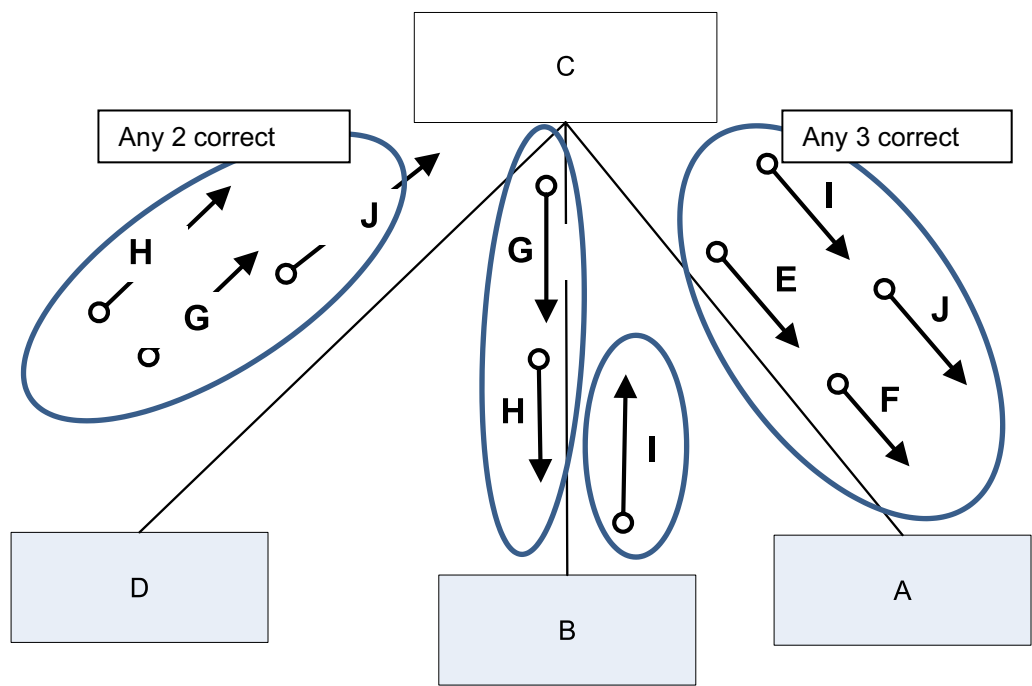
3 (a) Control box – C // Produce insurance quotation

[1]

D // Input customer details + A // Send quotation letter is correct positions

[1]

(b)



Data items	
E	CustomerName
F	CustomerEmail
G	Model
H	Age
I	PolicyCharge
J	PolicyNumber

[4]

155. M/J 15/P21/Q4/(ii)

(ii) Player scores equal // if Player1Total = Player2Total // there is no winner // a draw

[1]

156. M/J 15/P21/Q5/b**(b) (i)**

x	DayNumber	OUTPUT
0	1	
	2	
1	3	5/6/2015
	4	
2	5	7/6/2015
	6	
3	7	9/6/2015
		3

Note: 'x' and 'output' entries must be on or below the relevant 'DayNumber' entry
Mark as above

[4]

- (ii)
- ... Sales for the first seven days (1)
 - ... the number of days on which the total sales were 10 or over (1)
 - Outputs the corresponding dates (1)
 - Output the final value/total (of x) (1) [max 3]

157. M/J 15/P22/Q1/c

- (c) (i)
- Choosing data/values...
 - Test every possible 'logic path' through the code
// with knowledge of the structure/code

Ignore any reference to normal/boundary/extreme ...

[2]

- (ii)
- `PersonalBest` column labelled
 - Test number 1 message: "Equals personal best time"/or similar
 - Test 2/Test 3 – data for better performance ...
 - Described with suitable message
 - Test 2/Test 3 – data for worse performance ...
 - Described with suitable message

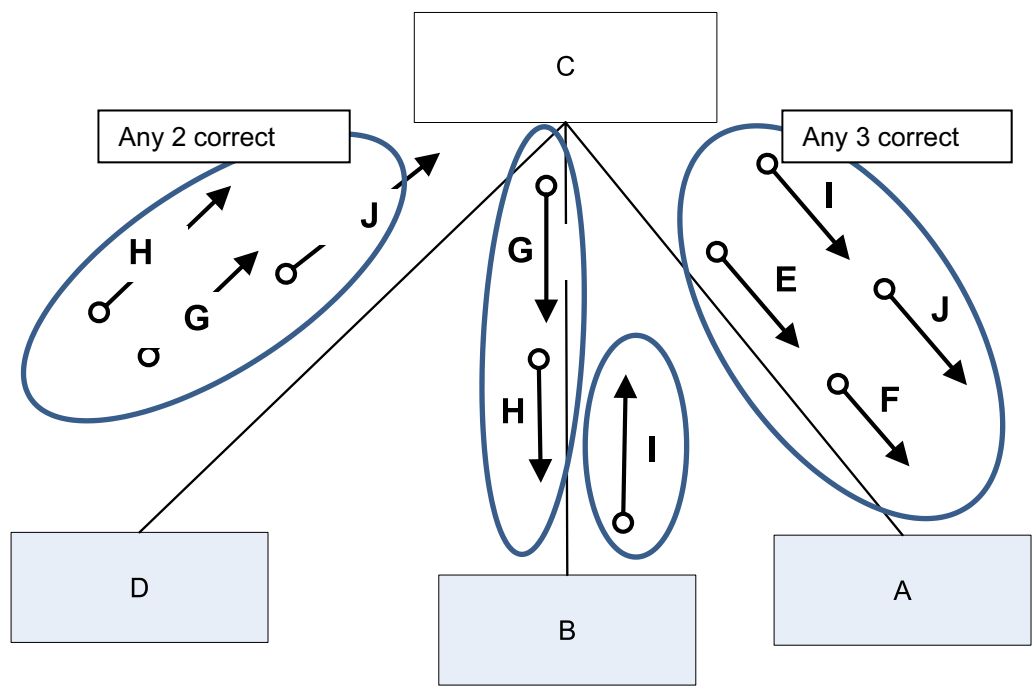
[6]

158. M/J 15/P22/Q3/a,b

- 3 (a) Control box – C // Produce insurance quotation [1]

D // Input customer details + A // Send quotation letter is correct positions [1]

(b)



Data items	
E	CustomerName
F	CustomerEmail
G	Model
H	Age
I	PolicyCharge
J	PolicyNumber

[4]

159. M/J 15/P22/Q4/(ii)

(ii) Player scores equal // if Player1Total = Player2Total // there is no winner // a draw

[1]

160. M/J 15/P22/Q5/b**(b) (i)**

x	DayNumber	OUTPUT
0	1	
	2	
1	3	5/6/2015
	4	
2	5	7/6/2015
	6	
3	7	9/6/2015
		3

Note: 'x' and 'output' entries must be on or below the relevant 'DayNumber' entry
Mark as above

[4]

- (ii)
- ... Sales for the first seven days (1)
 - ... the number of days on which the total sales were 10 or over (1)
 - Outputs the corresponding dates (1)
 - Output the final value/total (of x) (1) [max 3]

161. M/J 15/P23/Q2/b(i)**(b) (i)**

Input value	Output			Comment
	Fifty Dollar	Twenty Dollar	Ten Dollar	
70	1	1	0	Least possible number of notes
85	(0	0	0)	Error message
130	2	1	1	Least possible number of notes
600	(0	0	0)	Error message

Penalise any number entries on the 85 and 600 rows

[3]

162. M/J 15/P23/Q3/(i)**3 (i)**

A	Width	in any order
B	Length	
C	JobID	
D	CustomerName	in any order
E	JobCost	

[5]

163. M/J 15/P23/Q5/b

(b)

				WorkerTotal		
WorkerNum	DayNum	WorkerAverage	OUTPUT	1	2	3
1				0		
2					0	
3						0
1	1			10		
	2			21		
	3			31		
	4			45		
2	1				20	
	2				36	
	3				60	
	4				80	
3	1					9
	2					20
	3					33
	4					50
1		2.25				
2		2				
3		1.25	INVESTIGATE 3			